



National Technical University of Athens
School of Electrical and Computer Engineering
Data Science & Machine Learning
Post-Graduate Program

Object Detection and Semantic Segmentation of Aerial and Satellite Imagery with Deep Learning Models

Postgraduate Diploma Thesis

Panagiotis Fragkos
Rural & Surveying Engineer - Geoinformatics Engineer

Supervisor:
Athanasios Voulodimos
Assistant Prof. SECE/NTUA

Athens, 2026



National Technical
University of Athens
School of Electrical and Computer Engineering
Data Science & Machine Learning
Post-Graduate Program

Object Detection and Semantic Segmentation of Aerial and Satellite Imagery with Deep Learning Models

Postgraduate Diploma Thesis

Panagiotis Fragkos

Supervisor: Athanasios Voulodimos, Assistant Prof. SECE/NTUA

The postgraduate diploma thesis has been approved by the examination
committee on the 8th of July 2026

Athanasios Voulodimos
Assistant Prof. SECE/NTUA

Charalabos Ioannidis
Prof. SRSGE/NTUA

Georgios Stamou
Prof. SECE/NTUA

Panagiotis Fragkos

Rural & Surveying Engineer - Geoinformatics Engineer

Graduate of the Postgraduate Programme

“DATA SCIENCE and MACHINE LEARNING”

School of Electrical and Computer Engineering

National Technical University of Athens

No part of this thesis may be reproduced, stored, or distributed, in whole or in part, for commercial purposes. Reproduction, storage, and distribution for non-profit, research, or educational purposes are permitted, provided that the source is properly acknowledged by proper citation and this notice is retained. Requests regarding the use of this work for commercial purposes should be addressed to the author (panagiotisfragkos@mail.ntua.gr).

The views and conclusions presented in this document reflect the opinions of the author and do not necessarily represent the official views of the National Technical University of Athens.

Acknowledgments

This master's thesis constitutes the culmination of a demanding academic journey, through emerging & disruptive technologies and scientific domains that are transforming the traditional approaches to research and professional practice in the field of Engineering. The knowledge and skills acquired through the DSML program have provided me with the opportunity to continue my academic pursuits, equipped with the strongest possible foundations.

I feel a deep obligation to express my sincere gratitude to my supervisor, Assistant Professor Athanasios Voulodimos, for the opportunity he provided me to engage with research topics of direct interest to me, for the academic freedom he granted me in pursuing my research directions, as well as for his guidance and support throughout this work.

I would also like to thank the other members of the three-member examination committee, Professor Charalabos Ioannidis (a Mentor and a Teacher throughout my whole academic journey) and Professor Georgios Stamou, for their insightful comments, guidance and evaluation of this thesis.

Above all, my greatest thanks are owed to my wife, Anna, for her unwavering support, patience and understanding, throughout this demanding journey. Without it, none of this would have been possible...

The best is yet to come...

Περίληψη

«Ανίχνευση Αντικειμένων και Σημασιολογική Κατάτμηση Εναέριων και Δορυφορικών Απεικονίσεων με Μοντέλα Βαθιάς Μάθησης»

Η ανίχνευση αντικειμένων και η κατάτμηση στιγμιοτύπων σε εναέριες και δορυφορικές εικόνες υποστηρίζουν ένα ευρύ φάσμα εφαρμογών παρατήρησης Γης, όπως η αστική χαρτογράφηση, η παρακολούθηση υποδομών, η ναυτιλιακή επιτήρηση και η περιβαλλοντική ανάλυση. Σε αντίθεση με τις συνήθεις επίγειες εικόνες, τα εναέρια δεδομένα παρουσιάζουν αρκετές χαρακτηριστικές δυσκολίες (ακραίες μεταβολές στην οπτική γωνία λήψης, αυθαίρετο προσανατολισμό των αντικειμένων, μεγάλη διακύμανση κλίμακας, πυκνές διατάξεις αντικειμένων και περιορισμένη διαθεσιμότητα επισημειώσεων σε επίπεδο εικονοστοιχείου). Τα πρόσφατα μοντέλα βαθιάς μάθησης έχουν επιτύχει υψηλές επιδόσεις σε συμβατικά σετ δεδομένων αξιολόγησης, αλλά η συμπεριφορά τους στο πεδίο της εναέριας απεικόνισης, και ιδίως υπό συνθήκες περιορισμένων δεδομένων εκπαίδευσης, δεν έχει μελετηθεί εξίσου διεξοδικά.

Η παρούσα διπλωματική εργασία αξιολογεί ένα αρθρωτό σύστημα Ανίχνευσης→Κατάτμησης (Detection→Segmentation) για εναέριες εικόνες, το οποίο συνδυάζει ανιχνευτές αντικειμένων της οικογένειας YOLO με το μοντέλο Segment Anything Model (SAM), χρησιμοποιούμενο για την, καθοδηγούμενη από προτροπές (prompts), κατάτμηση και εξαγωγή αυτών. Η σχεδίαση αυτή αποφεύγει οποιαδήποτε εξειδικευμένη εκπαίδευση του μοντέλου κατάτμησης και στοχεύει στη μειώνοντας το κόστος δημιουργίας σετ δεδομένων ενώ διατηρεί παράλληλα υψηλή ποιότητα масκών. Για την υλοποίηση μια ενιαίας αξιολόγησης, τόσο ως προς το στάδιο ανίχνευσης όσο και ως προς την κατάτμηση, κατασκευάστηκε ένα ολοκληρωμένο σύνολο δεδομένων με την ονομασία AerialFuseCV, μέσω της αντιστοίχισης των προσανατολισμένων περικλειόντων πολυγώνων (OBB) του DOTA v1.0 με τις μάσκες στιγμιοτύπων του iSAID σε επίπεδο μεμονωμένου αντικειμένου. Το νέο σύνολο δεδομένων επιτρέπει τη συστηματική αξιολόγηση τόσο της αυτόνομης ανίχνευσης και της αυτόνομης κατάτμησης, όσο και της απόδοσης του ολοκληρωμένου συστήματος, υπό κοινές συνθήκες αξιολόγησης. Με τον τρόπο αυτό καθιερώνεται μια ενιαία βάση αξιολόγησης, για σαφώς συγκρίσιμα αποτελέσματα και τη συνακόλουθη εξαγωγή αντικειμενικών συμπερασμάτων.

Παρουσιάζονται τρία σετ πειραμάτων. Μια αξιολόγηση (benchmark) του σταδίου ανίχνευσης με μοντέλα της οικογένειας YOLO, καλύπτοντας πολλαπλές παραλλαγές, τόσο σε διαμόρφωση κατακόρυφων (axis-aligned, VBB) όσο και προσανατολισμένων (oriented, OBB) περικλειόντων πολυγώνων, ποσοτικοποιώντας την επίδρασή τους στην ποιότητα της ανίχνευσης. Μια αξιολόγηση της ικανότητας γενίκευσης (zero-shot) του μοντέλου κατάτμησης SAM, εξετάζοντας τρεις παραλλαγές του κωδικοποιητή εικόνας (ViT-H, ViT-L, ViT-B). Αξιολογούνται στρατηγικές προτροπών είτε μέσω περικλειόντων πολυγώνων είτε μέσω σημείων, με χρήση δεδομένων αληθείας, ώστε η

ποιότητα της κατάτμησης να απομονωθεί από τα σφάλματα ανίχνευσης. Τέλος, αξιολογείται το ολοκληρωμένο σύστημα Ανίχνευσης→Κατάτμησης το οποίο συντίθεται από την καλύτερη έκδοση YOLO και την καλύτερη παραλλαγή του SAM, οι οποίες προέκυψαν από τα προηγούμενα πειράματα και αξιολογείται η συμπεριφορά του συστήματος, από άκρο σε άκρο, στο σύνολο δεδομένων AerialFuseCV.

Τα αποτελέσματα αποκαλύπτουν μια σαφή ασυμμετρία. Τα μοντέλα ανίχνευσης που έχουν προεκπαιδευθεί σε επίγειες εικόνες (COCO) παρουσιάζουν μηδενική απόδοση στο πεδίο των εναέριων εικόνων, δείχνοντας αδυναμία γενίκευσης και σημαντικές διαφορές μεταξύ των δυο οπτικών (κατακόρυφη και οριζόντια), παρά τη συνάφεια των αντικειμένων. Αντιθέτως, το SAM επιδεικνύει εξαιρετική ικανότητα κατάτμησης σε συνθήκες χωρίς προεκπαίδευση (zero-shot) στην εναέρια οπτική, εφόσον τροφοδοτείται με ακριβείς χωρικές προτροπές. Η ποιότητα της προτροπής αναδεικνύεται ως ο κυρίαρχος παράγοντας που επηρεάζει την ακρίβεια της κατάτμησης, με τις προτροπές περικλείοντος πολυγώνου να αποδίδουν μέσο IoU μάσκας μεγαλύτερο κατά περίπου 20% σε σχέση με τις προτροπές σημείου. Το μέγεθος του μοντέλου διαδραματίζει συγκριτικά μικρότερο ρόλο, καθώς το SAM-ViT-B επιτυγχάνει μέσο IoU 66,9% έναντι 67,7% του SAM-ViT-H, διαφορά μικρότερη του 1% παρά την επταπλάσια αύξηση στον αριθμό παραμέτρων του μοντέλου.

Το ολοκληρωμένο σύστημα (YOLOv11x-OBB σε συνδυασμό με SAM-ViT-B και προτροπές περικλείοντος πολυγώνου) επιτυγχάνει μέσο IoU μάσκας 67,9% στη σύγκριση των προβλέψεων με τα δεδομένα αληθείας, τιμή σε γενικές γραμμές συγκρίσιμη με το άνω όριο που θέτει το ανεξάρτητο πείραμα της αξιολόγησης του μοντέλου SAM. Ο κύριος περιοριστικός παράγοντας που αναδεικνύεται, είναι η Ανάκληση (Recall) του μοντέλου ανίχνευσης, η οποία ανέρχεται σε 62,1% στις 15 κατηγορίες αντικειμένων, υποδηλώνοντας ότι περίπου το 38% των αντικειμένων αληθείας δεν λαμβάνει καμία προτροπή κατάτμησης. Η απόδοση παρουσιάζει σημαντική διακύμανση ανά κατηγορία, με εύρος από 87,1% IoU για τα γήπεδα ποδοσφαίρου έως 0% για τις πισίνες, όπου η ανίχνευση αποτυγχάνει πλήρως. Ο συνολικός χρόνος εκτέλεσης του αγωγού ανέρχεται κατά μέσο όρο σε περίπου 730 ms ανά εικόνα (σε GPU τύπου RTX 3090 Ti), με το SAM να καταναλώνει περίπου το 92% του προϋπολογισμού χρόνου, γεγονός που καθιστά τη μέθοδο κατάλληλη για μαζική επεξεργασία εναέριων συνόλων δεδομένων, όχι όμως για ανάπτυξη σε πραγματικό χρόνο (edge processing).

Συνολικά, η παρούσα διπλωματική εργασία παρέχει μια συστηματική αξιολόγηση ενός συστήματος Ανίχνευσης→Κατάτμησης για εναέριες εικόνες, διασαφηνίζει τις συνθήκες υπό τις οποίες η προσέγγιση είναι βιώσιμη επιχειρησιακά και ποσοτικοποιεί τους συμβιβασμούς μεταξύ της αποδιδόμενης ποιότητας μάσκας και της χρονικής απόδοσης (latency). Τα ευρήματα προσφέρουν πρακτικές κατευθύνσεις για την ενσωμάτωση μοντέλων κατάτμησης βασισμένων σε βασικά μοντέλα (foundation models) σε ροές εργασίας παρατήρησης Γης (EO) και προσδιορίζουν συγκεκριμένες κατευθύνσεις για μελλοντική βελτίωση. Συγκεκριμένα αναδεικνύεται η ανάγκη

βελτίωσης του μοντέλου ανίχνευσης, ιδίως για μικρά και σπάνια αντικείμενα, η ανάγκη ενσωμάτωσης δυνατότητας μαζική (batched) εκτέλεσης του SAM, καθώς και η κατασκευή συνόλων δεδομένων ανίχνευσης–κατάτμησης που να διατηρούν το πλήρες ραδιομετρικό εύρος και τη γεωαναφορά της αρχικής λήψης.

Λέξεις Κλειδιά: Εναέριες εικόνες, Σύστημα Ανίχνευσης–Κατάτμησης, Ανίχνευση αντικειμένων, Σημασιολογική κατάτμηση, YOLO, Segment Anything Model (SAM), Χωρίς προεκπαίδευση, Παρατήρηση Γης.

Abstract

“Object Detection and Semantic Segmentation of Aerial and Satellite Imagery with Deep Learning Models”

Object detection and instance segmentation in aerial and satellite imagery support a wide range of Earth observation applications, including urban mapping, infrastructure monitoring, maritime awareness, and environmental analysis. Unlike natural ground-level imagery, aerial data introduce several characteristic difficulties (extreme viewpoint changes, arbitrary object rotation, large scale variation, dense object layouts, and limited availability of pixel-level annotations). Recent deep learning models have achieved strong performance on conventional computer vision benchmarks, but their behavior in the aerial regime, particularly under restricted training data, is not as well understood.

This thesis evaluates a modular Detection→Segmentation pipeline for aerial imagery that pairs object detectors from the YOLO family with the Segment Anything Model (SAM) used as a prompt-driven segmentation backend. The design avoids any task-specific training of the segmentation stage and aims to reduce annotation cost while maintaining usable mask quality. To support a unified evaluation of detection and segmentation, a paired dataset, AerialFuseCV, is constructed by reconciling DOTA v1.0 oriented bounding box annotations with iSAID instance masks at the instance level. The reconciled dataset enables systematic assessment of standalone detection, standalone segmentation, and end-to-end pipeline performance under consistent conditions.

Three sets of experiments are reported. A YOLO detection benchmark covers multiple model variants in both axis-aligned (VBB) and oriented (OBB) configurations, isolating the contribution of rotation-aware representations. A SAM zero-shot benchmark evaluates three image-encoder variants (ViT-H, ViT-L, ViT-B) under both bounding box and point prompting strategies, using ground-truth prompts to isolate segmentation quality from detection error. The integrated pipeline benchmark connects the best-performing YOLO configuration to each SAM variant and measures end-to-end behavior on the AerialFuseCV validation split.

The results reveal a clear asymmetry between the two stages. Detectors pretrained on ground-level imagery transfer poorly to the aerial domain, with OBB-trained variants substantially outperforming their VBB counterparts on rotated objects. SAM, by contrast, demonstrates useful zero-shot segmentation when supplied with accurate spatial prompts. Prompt quality emerges as the dominant factor affecting segmentation accuracy, with bounding box prompts yielding mask IoU roughly 20 percentage points higher than point-based prompts. Model size plays a comparatively minor role, as

SAM-ViT-B reaches 66.9% mean IoU against 67.7% for SAM-ViT-H, a gap of less than one percentage point across a sevenfold parameter difference.

The integrated pipeline (YOLOv11x-OBB paired with SAM-ViT-B and box prompts) achieves 67.9% mean mask IoU on matched prediction–ground-truth pairs, broadly comparable to the SAM-only upper bound. The principal limitation is detection Recall, which sits at 62.1% across 15 object categories, indicating that around 38% of ground-truth instances receive no segmentation prompt at all. Performance varies substantially by class, ranging from 87.1% IoU for soccer ball fields to 0% for swimming pools, where detection fails entirely. Total pipeline inference time averages roughly 730 ms per image on an RTX 3090 Ti, with SAM accounting for around 92% of the budget, which makes the approach suitable for batch processing of aerial datasets but not for real-time deployment.

Overall, this thesis provides a systematic evaluation of a zero-shot Detection→Segmentation pipeline for aerial imagery, clarifies the conditions under which the approach is viable for operational use and quantifies the trade-offs between mask quality and operational latency. The findings offer practical guidance for deploying foundation-model-based segmentation in Earth observation (EO) workflows and identify directions for future improvement, namely detector adaptation for small and rare object categories, batched SAM inference and the construction of paired detection–segmentation datasets that preserve full radiometric range and georeferencing information.

Key Words: Aerial imagery, Detection–Segmentation pipeline, Object detection, semantic segmentation, YOLO, Segment Anything Model (SAM), Zero-shot segmentation, Earth observation.

Table of Contents

Acknowledgments.....	5
Περίληψη	7
Abstract	11
Table of Contents.....	13
Images' Index.....	14
Tables' Index	15
Acronyms.....	16
Introduction.....	19
Chapter 1 Background and Related Work.....	22
1.1 Computer Vision for Earth Observation.....	22
1.2 Object Detection	24
1.2.1 From Two-Stage to Single-Stage Detectors	24
1.2.2 The YOLO Family.....	26
1.2.3 Oriented Object Detection in Aerial Imagery.....	28
1.3 Image Segmentation	29
1.3.1 Semantic Segmentation	30
1.3.2 Instance Segmentation	31
1.3.3 Foundation Models: The Segment Anything Model	32
1.4 Aerial and Satellite Imagery Datasets.....	38
1.4.1 Detection Datasets	39
1.4.2 Segmentation Datasets.....	40
1.4.3 The Dataset Gap	42
1.5 Related Work	44
1.5.1 YOLO in Aerial Imagery.....	45
1.5.2 SAM in Remote Sensing	45
1.5.3 Detection-to-Segmentation Pipelines	46
1.6 Research Gaps and Thesis Positioning.....	47
Chapter 2 Relevant Public Datasets.....	49
2.1 DOTA v1.0 Dataset Exploratory Data Analysis (EDA).....	49
2.2 iSAID Dataset EDA Exploratory Data Analysis (EDA)	52
Chapter 3 Methodology	55
3.1 AerialFuseCV Dataset Creation and Analysis.....	55
3.2 YOLO Baseline Evaluation	61
3.3 SAM Baseline Evaluation.....	64
3.4 Integrated Detection→Segmentation Pipeline Evaluation	66
3.5 Evaluation Framework – Performance Metrics	70
3.5.1 Detection Stage Assessment Metrics.....	70
3.5.2 Segmentation Stage Assessment Metrics	74
3.5.3 Integrated Detection→Segmentation Pipeline Assessment Metrics	75
Chapter 4 Experimental Results.....	78
4.1 YOLO Detection Baseline Results	78
4.1.1 Vertical Bounding Box (VBB) Experiment	79

4.1.2	Oriented Bounding Box (OBB) Experiment	82
4.2	SAM Segmentation Baseline Results	87
4.2.1	Box-Prompted Segmentation.....	88
4.2.2	Point-Prompted Segmentation	92
4.2.3	Box vs Point Prompted Segmentations' Comparison.....	96
4.3	Unified Detection→Segmentation Pipeline Results.....	98
4.3.1	Pipeline Final Configuration.....	98
4.3.2	Pipeline Experimental Results	100
4.3.3	Pipeline Comparative Analysis to Baseline Experimental Results	105
Chapter 5	Discussion - Conclusions & Future Work.....	107
5.1	Discussion of Experimental Findings	107
5.1.1	Domain Transfer Without Fine Tuning: Capabilities and Boundaries	107
5.1.2	Prompt Quality as the Dominant Variable.....	108
5.1.3	The Detection Bottleneck	109
5.1.4	Pipeline Viability for Operational Deployment.....	109
5.2	Conclusions.....	110
5.3	Limitations and Work Assumptions	112
5.4	Future Research Directions.....	114
	References.....	118
	Appendices.....	125

Images' Index

Figure 1: OBB detection (a), VBB detection (b), semantic segmentation masks (c) ..	22
Figure 2: YOLOv1 architecture (source: analyticsvidhya).....	26
Figure 3: OBB (a) vs VBB (b) in condensed aerial view	29
Figure 4: Aerial Image (a), semantic segmentation masks (b) and instance segmentation masks (c)	30
Figure 5: U-NET symmetrical architecture with skip connections between encoder and decoder layers (source: Ronneberger et al., 2015b).....	31
Figure 6: SAM promptable segmentation concept. Four prompt types each produce a valid mask.	33
Figure 7: High-level architecture of the Segment Anything Model	34
Figure 8: The SAM mask decoder	36
Figure 9: SAM's segmentation candidate masks for a pair of scissors	37
Figure 10: DOTA v1.0 class distribution and statistics for training and validation splits	50
Figure 11: DOTA v1.0 example image with overlaid vertical bounding boxes – VBB (left) and oriented bounding boxes – OBB (right).....	52
Figure 12: iSAID instance segmentation and semantic segmentation masks example	53
Figure 13: iSAID class distribution and statistics per split.....	54
Figure 14: Excluded images due to annotation inconsistencies from AerialFuseCV refinement process. Segmentation masks with no corresponding detection BBoxes..	56

Figure 15: AerialFuseCV creation methodology	56
Figure 16: AerialFuseCV class distribution and statistics per split	60
Figure 17: Integrated Detection-Segmentation Pipeline Architecture	67
Figure 18: Class “Small-vehicle” and “Large-vehicle” unstable detection performance from YOLOv12x model, over similar imagery	81
Figure 19: Class “Ship” partly successful detections, with unstable results from YOLOv12x (left) and YOLOv11x (right) models	81
Figure 20: YOLOv11x-OBb best detection performance classes examples	85
Figure 21: YOLOv11x-OBb medium detection performance classes examples	85
Figure 22: YOLOv11x-OBb worst detection performance classes examples	86
Figure 23: “Baseball diamond” successful vs “swimming pool” failed detection by YOLOv11x-OBb	86
Figure 24: “Roundabout” loose annotation and tight model detection (left) and a “basketball-court” labeled as “soccer-ball-field” (right)	87
Figure 25: SAM segmentation of class “helicopter” instances	90
Figure 26: SAM segmentation of class “large-vehicle” instances	90
Figure 27: SAM segmentation of class “bridge” in small scale imagery	90
Figure 28: Failed segmentation of the aircraft carrier (class “ship”) due to VBB background inclusion	91
Figure 29: Successful (up) and failed (down) segmentation of objects of class “roundabout”	91
Figure 30: High quality segmentation cases with bbox centroid point prompting	94
Figure 31: Case of class “roundabout” that benefits from centroid point SAM prompting	95
Figure 32: Case of class “roundabout” that benefits (up) and case of class “ground-track-field” that does not benefit from centroid point SAM prompting	95
Figure 33: Detection→Segmentation pipeline implementation	98
Figure 34: Integrated Detection→Segmentation pipeline detection failure examples	103
Figure 35: AerialFuseCV annotation discrepancies, inherited from DOTA v1.0 and iSAID datasets	104

Tables’ Index

Table 1: YOLO family evolution summary	28
Table 2: SAM image encoder variants	34
Table 3: Comparison of detection datasets in the aerial and satellite domain	40
Table 4: Comparison of segmentation datasets in the aerial and satellite domain.	42
Table 5: iSAID semantic segmentation masks’ RGB color encoding for 15 classes ..	53
Table 6: AerialFuseCV refinement process overall results	58
Table 7: AerialFuseCV refinement process per class results	58
Table 8: Datasets used and role in evaluation experiments	60
Table 9: Full YOLO baseline evaluation experimental setup	61

Table 10: COCO to DOTA Class Correspondence, for YOLO VBB experimental evaluation.....	62
Table 11: SAM baseline evaluation experimental setup.....	65
Table 12: Detection Assessment Metrics - Definitions and Interpretations	71
Table 13: Segmentation Assessment Metrics	74
Table 14: Integrated Pipeline Assessment Metrics Summary	75
Table 15: Macro vs Micro Averaging Comparison	77
Table 16: YOLO-VBB Overall Performance (Top 5 Models, matching IoU>0.1).....	79
Table 17: YOLOv12l-VBB Per-Class Performance (3 Overlapping Classes, matching IoU>0.1).....	80
Table 18: Complete YOLO - OBB Model Progression (matching IoU>0.1).....	82
Table 19: YOLOv11x-OBB Per-Class Detection Performance (matching IoU>0.1) .	84
Table 20: SAM Box-Prompted Overall Performance.....	89
Table 21: SAM-ViT-B-BOX Per-Class Segmentation Performance	89
Table 22: SAM Point-Prompted Overall Performance.....	93
Table 23: SAM-ViT-L-POINT Per-Class Segmentation Performance	94
Table 24: Box vs Point Prompt Performance Gap for SAM-ViT-B variant.....	97
Table 25: Box vs Point Inference Time Comparison per SAM Variant.....	97
Table 26: Integrated Detection→Segmentation pipeline component configuration .	100
Table 27: Integrated Pipeline Overall Performance.....	101
Table 28: Integrated Pipeline Per-Class Results.....	102
Table 29: Integrated Pipeline Timing Breakdown.....	103
Table 30: Pipeline vs SAM baseline segmentation quality (Δ IoU) and detection Coverage	105

Acronyms

CNN	Convolutional Neural Network
COCO	Common Objects in Context
DOTA	Dataset for Object Detection in Aerial Images
EO	Earth Observation
EfficientSAM	Efficient Segment Anything Model
F1	F1 Score
FN	False Negative
FP	False Positive
GeoAI	Geospatial Artificial Intelligence
GPU	Graphics Processing Unit
GSD	Ground Sample Distance
IoU	Intersection over Union
iSAID	Instance Segmentation in Aerial Images Dataset
Mask R-CNN	Mask Region-Based Convolutional Neural Network
Mask2Former	Masked-attention Mask Transformer
mAP	mean Average Precision

OBB	Oriented Bounding Box
PerSAM	Personalized Segment Anything Model
RGB	Red Green Blue
RTX	NVIDIA Ray Tracing Texel eXtreme (GPU series)
SA-1B	Segment Anything 1 Billion Mask Dataset
SAM	Segment Anything Model
SAM2	Segment Anything Model v2
SAR	Synthetic Aperture Radar
SOLOv2	Segmenting Objects by Locations v2
UAV	Unmanned Aerial Vehicle
VBB	Vertical Bounding Box
ViT	Vision Transformer
YOLO	You Only Look Once
YOLACT	You Only Look At Coefficients

Introduction

Over the past two decades, computer vision has become a core enabling technology for modern perception systems. Most of the field's progress has been driven by natural image datasets, that is photographs captured at human eye level, with objects at familiar scale, perspective, and texture. Remote sensing imagery violates most of those assumptions. In aerial and satellite views objects appear at smaller spatial extents, rotations are arbitrary, backgrounds are repetitive and easily confused with targets, and illumination and atmospheric conditions vary widely. Models that perform strongly on ground-level benchmarks often degrade sharply when deployed on overhead imagery, and the literature contains many cases where detection performance drops by tens of percentage points on the domain shift alone.

At the same time, the demand for turning remote sensing imagery into actionable products has grown rapidly, driven by cheaper imagery (more satellite Earth observation sensors, the proliferation of UAVs etc.) and by improvements in modelling capacity. Aerial and satellite data now support applications such as disaster response, infrastructure monitoring, maritime awareness, precision agriculture, environmental monitoring, and dynamic mapping. In many of these scenarios, bounding boxes that generally localize objects (detection) are not sufficient. They indicate that an object exists and roughly where it is, but they do not describe its shape. For GIS workflows (area estimation, vectorization, change detection, map updates etc.), the boundary of the object is what matters and detection alone cannot supply it. Segmentation methods, on the other hand, can produce pixel-accurate object boundaries far faster than traditional manual workflows, but modern geospatial AI pipelines still depend heavily on suitable training data. For instance segmentation in particular, training a model from scratch in the aerial domain requires pixel-level masks for every object of interest, an annotation effort estimated at 10 to 100 times the cost of bounding box labeling. The result is a structural imbalance in the literature, in which detection datasets are relatively plentiful (DOTA, DIOR, FAIR1M, xView and others, each containing hundreds of thousands to millions of instances) while large-scale aerial instance segmentation datasets are scarce. iSAID, the segmentation companion to DOTA v1.0, remains the only general-purpose multi-class instance segmentation resource of comparable scope, but still covers only 15 object categories.

The recent emergence of vision foundation models, i.e. models trained on web-scale data with strong zero-shot transfer properties, offers a possible way to circumvent the annotation bottleneck. The Segment Anything Model (SAM), released by Meta AI in 2023 and trained on 1.1 billion masks across 11 million natural images, is the most prominent example. SAM is designed to segment any object indicated by a user-supplied prompt (a point, a bounding box, a rough mask, or in principle a text description) and it presents a good generalization capability to visual domains and object categories outside its training distribution. The behavior of SAM in the aerial domain has been studied

since shortly after its release, with results that are encouraging for some classes of objects and discouraging for others. The aerial domain itself imposes several specific challenges that any pipeline must contend with. Objects in aerial imagery appear at a wide range of scales (from vehicles a few pixels wide to airports spanning thousands), are viewed from an essentially nadir or near-nadir perspective rather than the perspective views that dominate ground-level training data, can be oriented at arbitrary angles relative to the image axes, often appear in dense clusters where instances touch or overlap and inhabit a different distribution of background contexts (rooftops, water, soil, vegetation etc.) than the predominantly indoor and street-level scenes, that are used to train general-purpose models. Beyond these geometric and contextual differences, the available public datasets impose their own constraints, since most are distributed as 8-bit RGB exported from consumer platforms (such as Google Earth), discarding both the multispectral richness and the georeferencing metadata that real Earth observation products carry.

This thesis investigates a specific way of combining a domain-trained detector with a zero-shot segmenter for the aerial regime. The detector (a YOLO family model) is responsible for finding objects and proposing oriented bounding boxes that respect the rotational variability of overhead views. The segmenter (SAM) consumes those boxes as spatial prompts and converts them into pixel-level instance masks without further training. The two components remain decoupled and can be updated independently, the detector benefits from the relatively abundant supply of bounding box annotations in aerial datasets and the segmenter inherits the broad generalization properties of its foundation-model pretraining. The central question is whether the combined system produces masks of practically useful quality across a representative range of aerial object categories and where its failure modes lie. Thus, this thesis focuses on answering the following research questions:

- How well do pretrained object detectors perform, out of the box, when the image domain shifts from ground-level views to aerial and satellite vertical views, and how do scale, scene density, and rotational variability affect detection quality?
- How well does a foundation segmentation model (SAM) perform on aerial imagery under zero-shot conditions and how do prompt type, model size, and image resolution affect mask quality?
- What is the current state of dataset availability for paired detection and segmentation in the aerial domain, and what gaps does it impose on rigorous end-to-end evaluation?
- How feasible is it to construct an end-to-end Detection→Segmentation pipeline that approaches near-real-time operation, and where do the principal bottlenecks lie?

Answering these questions requires thorough benchmarking. Each model must be evaluated individually to separate intrinsic model capability from pipeline integration effects and the full system must also be evaluated as an integrated whole, capturing the

behavior that a real-world deployment would observe. The objectives of this thesis, therefore are:

- To quantify the standalone performance of multiple YOLO variants on the aerial detection task, under both VBB and OBB configurations, isolating the contribution of orientation-aware modelling.
- To quantify the zero-shot performance of three SAM image-encoder variants under bounding box and point prompting, using ground-truth prompts to isolate segmentation quality from upstream noise.
- To evaluate the overall performance of a minimal Detection→Segmentation pipeline as a fully automated workflow for object extraction from aerial imagery, quantifying its processing characteristics and identifying the components most amenable to future optimization.

In support of these objectives, the thesis also constructs AerialFuseCV, a paired detection-and-segmentation dataset assembled from DOTA v1.0 and iSAID. AerialFuseCV reconciles oriented bounding box annotations with instance masks at the instance level across 15 object categories, addressing the absence of paired labels that has historically prevented systematic end-to-end evaluation of aerial pipelines.

The thesis is organized as follows. Chapter 1 provides the necessary background and reviews the relevant literature, covering the evolution of object detection and instance segmentation, the foundation-model paradigm and the SAM architecture in particular, the landscape of aerial detection and segmentation datasets as well as the structural gaps that motivate the present work. Chapter 2 presents the exploratory data analysis of the two public datasets used in this thesis (DOTA v1.0 and iSAID), characterizing their content, class distributions, image resolutions and the structural mismatches that motivate the construction of a paired dataset. Chapter 3 describes the methodology, namely the construction of the AerialFuseCV dataset, the experimental design for the YOLO detection baseline, the SAM segmentation baseline, the integrated Detection→Segmentation pipeline as well as the evaluation framework with its detection-stage, segmentation-stage and pipeline-level performance metrics. Chapter 4 reports the experimental results across three blocks (YOLO detection performance under both VBB and OBB configurations, SAM zero-shot segmentation under box and point prompting strategies and the integrated pipeline performance), including per-class behavior, error propagation analysis, and the comparison between predicted and ground-truth prompts. Chapter 5 discusses the findings in the context of the broader literature, draws the principal conclusions of the work, examines the main limitations of the approach and outlines directions for future work.

Chapter 1 Background and Related Work

1.1 Computer Vision for Earth Observation

Computer vision for Earth Observation (EO) revolves around two core tasks that guide nearly all downstream applications. Object detection identifies what is present in an image and encloses each instance in a bounding box, either axis-aligned (vertical bounding box - VBB) or rotated (oriented bounding box - OBB). Using bounding boxes is relatively cheap to annotate, efficient to run, and scales well, but they fail to provide any usable spatial information regarding the boundaries of the objects, making them semantically indistinct from the background in the image. For applications that need precise boundaries, such as building footprint extraction, area estimation, or the generation of geospatial vector layers, detection alone is not adequate. Image segmentation fills this gap, by assigning a class or instance label to every image pixel, producing dense masks that accurately describe objects and are directly usable in mapping workflows. Segmentation itself can be distinguished in two principal approaches, which are semantic segmentation and instance segmentation. Semantic segmentation labels every pixel with a class but treats all objects of the same class as one undifferentiated region (J. Long et al., 2015). That is perfectly fine for tasks like land-use/land-cover mapping, which aim to determine and quantify the earth's surface area coverage but not to analyze or quantify the above ground objects that exist (e.g., buildings, trees etc.). Instance segmentation on the other hand, separates each distinct object by assigning it its own mask (group of image pixels that fall within each distinct object's boundaries), even when two objects share the same class (He et al., 2017). This method is especially useful for downstream applications like counting ships in a harbor, inventorying vehicles in a parking lot, or tracking individual structures over time.

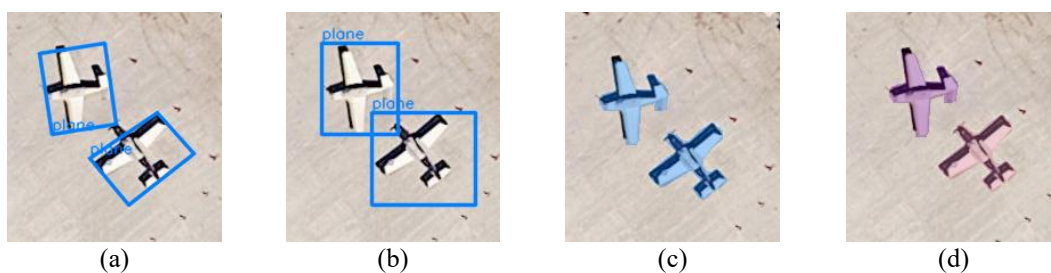


Figure 1: OBB detection (a), VBB detection (b), semantic segmentation masks (c) and instance segmentation masks (d)

Modern EO pipelines frequently adopt a two-stage paradigm, where an object detector first localizes candidate regions and a subsequent segmentation module refines each detection into a pixel-accurate mask, following the paradigm introduced by Mask R-CNN (He et al., 2017) and later extended through promptable foundation models such as SAM (Kirillov et al., 2023). Whether one task feeds the other or both run in parallel depends on the available annotation data and the requirements of the downstream

application. This detect-then-segment logic is the principle behind the pipeline proposed and investigated in this thesis.

Over the past fifteen years, deep learning has been the primary factor driving progress in both object detection and image segmentation. Convolutional Neural Networks (CNNs), including YOLO for real-time object detection (Redmon et al., 2016), ResNet for large-scale image recognition (He et al., 2015) and U-Net for dense pixel-wise segmentation (Ronneberger et al., 2015a), substantially outperformed earlier approaches based on handcrafted features and manually engineered descriptors, such as SIFT (Lowe, 2004), HOG (Dalal & Triggs, 2005), and SURF (Bay et al., 2006). In those earlier pipelines, visual descriptors were explicitly designed by researchers and typically combined with traditional classifiers (e.g., Support Vector Machines - SVMs), before the emergence of deep convolutional architectures capable of automatic feature learning (Krizhevsky et al., 2012). The strength of CNNs lies in their ability to learn hierarchical feature representations directly from the data itself (image pixels). Their shallow layers capture low-level structures such as edges and textures, while deeper layers encode progressively more abstract and semantically rich patterns. By 2020, CNN-based models had become the default paradigm for virtually all computer vision tasks, including detection, segmentation, classification, and scene interpretation, dominating most practical and research applications. Then a new architectural breakthrough soon emerged, named Transformers. The key innovation of Transformers was the introduction of the self-attention mechanism, which enables a model to weigh the relative importance of different parts of the input data when computing representations (Vaswani et al., 2017). Instead of processing information strictly through local neighborhoods, as convolution does, each element of the input can dynamically attend to all other elements. This global context modeling capability proved especially effective for capturing complex relationships in structured data. Their original application field was the natural language processing (NLP) domain, where transformers enabled context to be created by considering all past words in a sentence and not just the neighboring ones.

Very soon, the attention mechanism was adapted to computer vision through the Vision Transformer (ViT) (Dosovitskiy et al., 2021). In ViT, an image is divided into fixed-size patches, each treated as a token that represents for the image what words do for a sentence. These tokens are linearly embedded and processed through stacked self-attention layers, enabling the model to learn global spatial relationships across the entire image. ViT demonstrated that, when trained at scale, Transformer-based architectures could match and even surpass CNN performance in image classification tasks. By 2021, the Swin Transformer (Z. Liu et al., 2021) had surpassed CNNs on multiple image recognition benchmarks, while the Detection Transformer (DETR) (Carion et al., 2020a) demonstrated that end-to-end object detection, without handcrafted components like Non-Maximum Suppression (NMS) was not only possible but competitive with Faster R-CNN (Ren et al., 2016) on the COCO benchmark (Lin et al., 2015a).

For GeoAI and EO applications, the global context modeling capability of Transformers is particularly attractive. Aerial imagery often contains large scenes with distributed objects, long-range spatial dependencies, and complex background patterns. The ability of attention mechanisms to model interactions across the entire spatial extent of an image offers a promising direction for addressing some of the structural limitations of purely convolutional approaches. Both strands of this CNN-to-Transformer evolution are directly relevant to the present work, as the detection models evaluated here (the YOLO family) belong to the CNN tradition, while the segmentation model (SAM) is built on a ViT backbone.

Creating and training such models is resource intensive. It demands specialized expertise and substantial GPU infrastructure, a combination found primarily in large technology companies and far less often in academic settings. When these foundation models are released publicly, however, the broader research community can exploit them through transfer learning and fine-tuning (partially retraining a pretrained model on domain-specific data). The downfall of this approach is that domain-specific data, particularly pixel-level segmentation masks for remote sensing imagery, is expensive and slow to produce. Annotating segmentation masks is substantially more time-consuming than drawing bounding boxes, often requiring an order of magnitude more effort depending on object complexity and scene density. This cost asymmetry between detection labels and segmentation labels constitutes a secondary motivation of the present thesis, which, beyond performance benchmarking, investigates if such a workflow may also enable semi-automatic generation of segmentation masks for detection-only datasets.

1.2 Object Detection

Object detection can be considered to have evolved through the three main categories of two-stage region-based detectors, single-stage detectors and more recently Transformer-based end-to-end approaches. Study of this progression provides valuable insights for understanding the mechanisms behind the YOLO models evaluated in Chapters 3 and 4 and helps explain why certain design decisions (e.g., anchor-free heads, oriented bounding boxes etc.) can be as influential as raw performance metrics in determining real-world effectiveness, particularly in aerial imagery scenarios

1.2.1 From Two-Stage to Single-Stage Detectors

The deep learning era of object detection started with R-CNN (Girshick et al., 2014) which combined classical region proposal methods (Selective Search) with CNN based feature extraction and classification, establishing the modern two-stage detection paradigm. For each input image, Selective Search generated roughly 2000 candidate regions. Each region was warped to a fixed size, passed through an AlexNet backbone, and classified by a set of SVMs. The accuracy gains over prior handcrafted feature methods on PASCAL VOC (Everingham et al., 2010b), were substantial, but so was the computational cost, as 2000 independent CNN forward passes per image made R-CNN

literally impractical for anything approaching real-time processing. Fast R-CNN (Girshick, 2015) proposed a solution to this bottleneck. Instead of running the CNN once per region, it processed the entire image in a single forward pass, producing a shared feature map. Then, region specific features were extracted via Region of Interest (RoI) pooling and classification plus bounding box regression were trained jointly. The speed improvement was by orders of magnitude, but Selective Search still ran on the CPU, outside the trainable pipeline, and remained the slowest component. Faster R-CNN (Ren et al., 2015) replaced the Selective Search process with a Region Proposal Network (RPN), a lightweight fully convolutional network, that slides over the shared feature map and predicts, at each spatial location, a set of anchor boxes at multiple scales and aspect ratios along with probability scores for object existence (objectness). With the RPN, proposal generation became nearly free in computational terms, and the entire pipeline (feature extraction, proposal generation, classification, and regression) could be trained end-to-end for the first time. Over a decade later, Faster R-CNN remains a standard reference baseline in the detection literature, a testament to how well the RPN concept holds up.

While two-stage detectors were based on the “propose regions and then classify” method, an entirely different design philosophy emerged in parallel, which skipped the proposal stage, shifting the paradigm to single-stage detectors. According to this new approach, predicted bounding boxes and class scores derive directly from the feature map in one pass. SSD (W. Liu et al., 2015) was among the first to do this effectively, using multi-scale feature maps so that small objects could be detected from high-resolution layers and large objects from coarser ones. The main drawback of early single-stage detectors was severe class imbalance, as most spatial locations correspond to background, causing easy negative samples (patches containing only background and not object parts) to dominate the training process. RetinaNet (Lin et al., 2017) solved this issue by introducing a new loss function called Focal Loss, a modified cross-entropy loss function, that down-weights well-classified examples (easy negatives) and focuses the gradient on hard cases. The result was a single-stage detector that matched or exceeded the accuracy of two-stage models while running considerably faster. Focal Loss proved to be a highly influential contribution, widely adopted in detection, segmentation, and other tasks characterized by severe class imbalance. YOLO (Redmon et al., 2015) introduced at roughly the same time, advanced the single-stage detection paradigm. Instead of generating region proposals or relying on multi-scale anchor pyramids, YOLOv1 divided the image into an $S \times S$ grid and directly regressed bounding box coordinates and class probabilities from each grid cell in a single forward pass. This unified architecture was both conceptually simple and computationally efficient, achieving approximately 45 frames per second (FPS) on a Titan X GPU. While YOLOv1 offered lower detection accuracy compared to state-of-the-art two-stage detectors at the time, its real-time performance marked a significant shift in the field. In subsequent versions, architectural refinements progressively narrowed the accuracy gap with two-stage detectors while preserving high inference speed.

More recently, Transformers have entered the detection landscape. DETR (Carion et al., 2020b) reframed detection as a set prediction problem. According to this approach, a CNN backbone extracts features, a Transformer encoder-decoder processes them and a set of learned object queries outputs predictions in parallel. Prediction bounding boxes are matched via the Hungarian algorithm (Kuhn, 1955) with the ground-truth ones during training and the best matched bounding box is retained for each object. This replaces NMS entirely, producing a cleaner and conceptually simpler pipeline, where the model directly learns to predict one detection per object and not many ones, that will be cleaned up by a heuristic post process. On the COCO benchmark, DETR matched the well-optimized Faster R-CNN baseline, which was an impressive result for a first-generation architecture, though it struggled with small objects and required very long training schedules. Deformable DETR (X. Zhu et al., 2020) later addressed both issues through deformable attention that attends to a sparse set of sampling points rather than the full feature map, improving convergence and small-object performance substantially. While Transformer-based detectors are not directly evaluated in this thesis, a conceptual influence was adopted, as the Hungarian matching procedure used in DETR inspired the instance-level matching framework employed in the Detection→Segmentation pipeline evaluation of Chapter 3.

1.2.2 The YOLO Family

Since its introduction in 2016, the YOLO family has undergone numerous revisions of its main architecture (Figure 2), receiving extensions developed by multiple independent research groups and industry teams. This section traces the key architectural shifts, because they directly shape the model selection and performance patterns observed in Chapter 4.

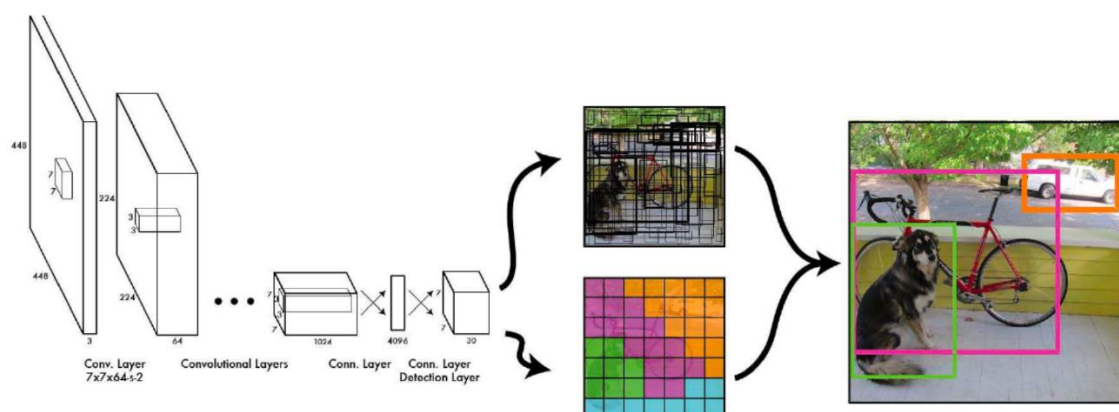


Figure 2: YOLOv1 architecture
(source: analyticsvidhya)

YOLOv1 established the core idea of image division into a grid, predict bounding boxes and class probabilities from each cell, and do it all in one forward pass, as described in section 1.2.1. Performance-wise it achieved 45 FPS with 63.4% mean average Precision (mAP) on PASCAL VOC 2007, which was fast but not especially accurate by modern

standards. YOLOv2 (or YOLO9000) added batch normalization, a dedicated Darknet-19 backbone (a lightweight 19-layer CNN designed for efficient feature extraction), anchor boxes (predefined boxes of various sizes and aspect ratios that models learned to adjust to an object) derived through k-means clustering on training data, and multi-scale training (randomly varying image resolution during training). These relatively straightforward changes resulted in a mAP that reached 78.6% on PASCAL VOC. YOLOv3 (Redmon & Farhadi, 2018) introduced multi-scale predictions via feature pyramid connections and a deeper Darknet-53 backbone, reaching 33% AP on the more demanding COCO benchmark. At this point, YOLO became competitive with two-stage methods on accuracy, while remaining a lot faster on inference.

In 2020, Joseph Redmon stepped away from computer vision research, and the YOLO lineage then fractured into multiple independent branches. YOLOv4 (Bochkovskiy et al., 2020) introduced a systematic framework of “bag-of-freebies” (training-time tricks like Mosaic augmentation and CutMix that cost nothing at inference) and “bag-of-specials” (architectural modules like CSPDarknet53 and PANet that marginally increase cost but substantially improve accuracy). It achieved 43.5% AP on COCO at 65 FPS on an NVIDIA V100. Shortly after, Ultralytics released YOLOv5 (Jocher et al., 2022), the first YOLO implementation written natively in PyTorch rather than Darknet. The switch to PyTorch was arguably as important as any architectural change, since it lowered the barrier to entry enormously and introduced the model-scaling paradigm (nano, small, medium, large, extra-large variants) that became standard in every subsequent release. YOLOv5x reached 50.7% AP on COCO.

The following two years saw a burst of contributions from different teams. YOLOv6 (C. Li et al., 2022), redesigned the backbone (EfficientRep) and neck (Rep-PAN) around hardware efficiency and introduced the anchor-free paradigm with a decoupled head that processes classification and localization through separate branches. YOLOv7 (C.-Y. Wang et al., 2022) focused on gradient-path optimization through E-ELAN (Extended Efficient Layer Aggregation Network), which considered both the number of parameters and the distance gradients must travel during backpropagation. Both versions pushed accuracy further while maintaining or even reducing latency. YOLOv8 (Jocher, et al., 2023), the second major Ultralytics release, consolidated many of these advances. Its C2f module (cross-stage partial bottleneck with two convolutions) replaced the earlier C3 block and the detection head became fully anchor-free with decoupled classification and regression branches. Crucially for this thesis, YOLOv8 was the first Ultralytics model to support oriented bounding boxes (OBB) natively as a first-class task, not just a community add-on. YOLOv9 (C.-Y. Wang et al., 2024) tackled the information bottleneck problem in deep networks through Programmable Gradient Information (PGI) and the GELAN architecture, while YOLOv10 (A. Wang et al., 2024) eliminated NMS at inference time entirely, via dual label assignments during training, a design inspired by DETR’s set-prediction approach. YOLO11 (Ultralytics, 2024), the most recent version evaluated in this thesis, introduced compact C3k2 bottleneck blocks and a C2PSA (Cross Stage Partial with Spatial Attention) module. The emphasis was on

yielding better accuracy-per-FLOP ratios without inflating latency, particularly aiming for edge deployment. OBB detection continued as a native capability. The 34 models evaluated in Chapter 4 span five of these generations (YOLOv8 through YOLO11 plus YOLOv12) in VBB and OBB configurations (wherever available), making the benchmark a notably broad YOLO evaluation conducted on aerial imagery to date. Table 1 provides a condensed timeline of the YOLO family.

Version	Year	Key Innovation	COCO AP
YOLOv1	2016	Grid-based single-pass detection	63.4 mAP ¹
YOLOv2	2017	Anchor boxes, batch norm, Darknet-19	78.6 mAP ²
YOLOv3	2018	Multi-scale predictions, Darknet-53	33.0%
YOLOv4	2020	CSPDarknet, Mosaic aug, BoF/BoS	43.5%
YOLOv5	2020	PyTorch, auto-anchor, model scaling	50.7%
YOLOv6	2022	EfficientRep, anchor-free, decoupled head	52.5%
YOLOv7	2022	E-ELAN, gradient path optimization	51.4%
YOLOv8	2023	C2f module, native OBB support	53.9%
YOLOv9	2024	PGI, GELAN architecture	55.6%
YOLOv10	2024	NMS-free inference, dual assignments	54.4%
YOLO11	2024	C3k2, C2PSA attention, OBB native	54.7%
YOLOv12	2025	Area Attention (A ²), R-ELAN, FlashAttention, attention-centric architecture	55.2%

Table 1: YOLO family evolution summary

1.2.3 Oriented Object Detection in Aerial Imagery

Standard detection represents objects as axis-aligned (or horizontal) rectangles (Vertical Bounding Boxes – VBB), parameterized by center coordinates, width, and height (cx , cy , w , h). This works well enough in ground-level photography, where gravity imposes a natural vertical orientation on most objects. Aerial imagery breaks that assumption entirely, since top views of objects like ships in harbors, vehicles on diagonal roads, bridges crossing rivers at odd angles etc., do not align with the image axes. For a narrow object rotated 45° , the smallest enclosing VBB will contain more than half background pixels. In dense scenes (Figure 3) like ports or parking lots, the overlap between neighboring VBBs becomes severe, confusing both the NMS step and downstream evaluation metrics (G.-S. Xia et al., 2019), (Ding et al., 2018a). Oriented Bounding Boxes (OBB) address this by adding a rotation angle θ to the parameterization: (cx , cy , w , h , θ). The result is a tighter fit around the actual object extent, which brings several practical benefits. IoU calculations become more meaningful, since the overlap metric now reflects genuine spatial agreement rather than an artifact of axis-alignment. Predictions in crowded scenes interfere less with each other and when bounding boxes

¹ AP values on COCO val2017 (x-large variant) unless noted.

² PASCAL VOC mAP.

serve as prompts for a segmentation model (as they do in this thesis) a tighter box means less irrelevant background is fed to the segmentation stage.

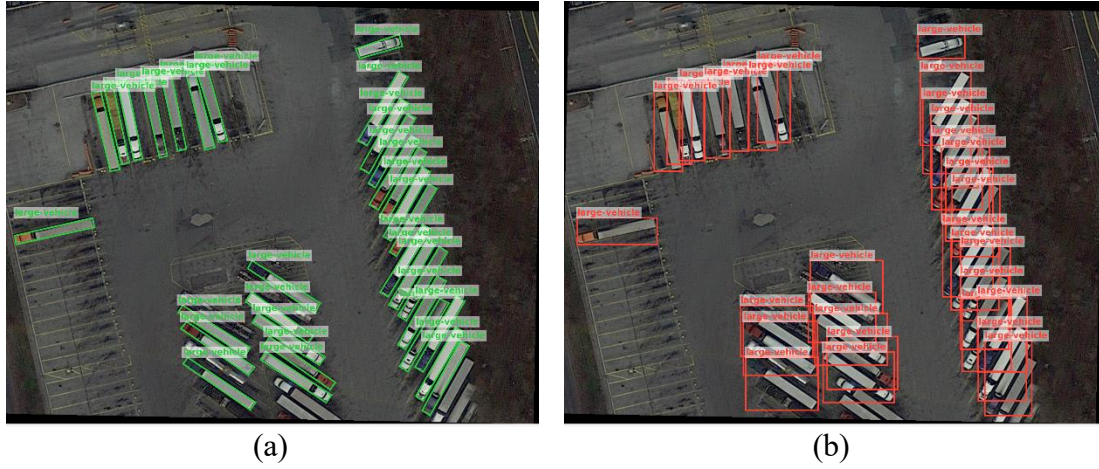


Figure 3: OBB (a) vs VBB (b) in condensed aerial view

(Source: DOTA v.1.0)

The technical challenges are not trivial. Angle periodicity (0° and 180° are identical for symmetric objects) creates discontinuities in standard regression losses, requiring special handling through circular loss functions or angle-classification strategies (Yang & Yan, 2020). Rotated NMS is also more expensive computationally than its axis-aligned counterpart, since computing the intersection area of two rotated rectangles is geometrically more involved. Despite these complications, OBB support has been progressively integrated into mainstream architectures, initially through community extensions (e.g., YOLOv5-OBB) and then as a native feature from YOLOv8-OBB onward. The DOTA dataset, which provides OBB annotations for aerial imagery, but easily enables the derivation of corresponding VBBs from the same data, has become the de facto standard benchmark for oriented object detection. The experimental results of this thesis make a strong case for why OBB matters. In Chapter 4, 24 COCO-pretrained VBB models achieve F1 scores below 0.3% on DOTA, leading to effectively zero useful detection. The 10 DOTA-pretrained OBB models, by contrast, reach up to 62.4% F1, making this gap not subtle, but catastrophic. This is not merely a training-data effect but reflects the fundamental geometric incompatibility of VBBs with arbitrarily oriented aerial objects. In that way, oriented representations are not a refinement, as far as aerial detection concerns, but they are rather a prerequisite.

1.3 Image Segmentation

Image segmentation assigns a label to every pixel in an image, producing dense output maps that capture object shapes far more faithfully than any bounding box can. The field has evolved along two distinct trajectories, the semantic segmentation one, which classifies pixels by category without distinguishing individual objects, and the instance segmentation one, which separates each object as a unique entity (Figure 4). Both have advanced rapidly over the past decade, but the recent emergence of foundation

segmentation models, most notably the Segment Anything Model (SAM), has altered the landscape more profoundly than any single architecture since U-Net. This section traces that progression.

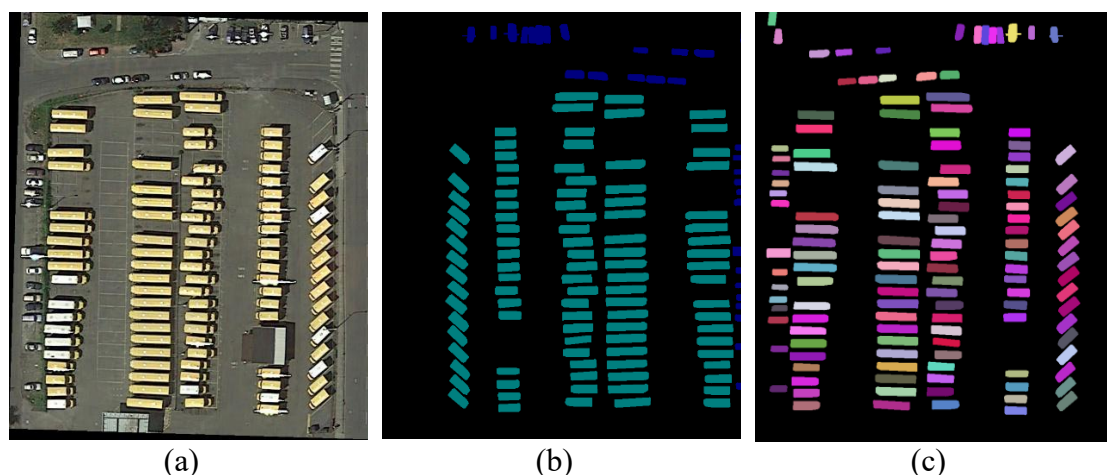


Figure 4: Aerial Image (a), semantic segmentation masks (b) and instance segmentation masks (c)
(Source: iSAID)

1.3.1 Semantic Segmentation

Semantic segmentation took its modern form with Fully Convolutional Networks (FCN), introduced by (J. Long et al., 2015). The key insight, although simple, delivered a powerful outcome. The main idea was to replace the fully connected classification layers of a standard CNN with convolutional layers, allowing the network to output spatial maps rather than a single class label. Skip connections fused coarse, deep features with fine, shallow features, recovering spatial detail that would otherwise be lost through pooling. FCN achieved 62.2% mean IoU on PASCAL VOC 2012, which constituted a substantial jump over prior methods and the first convincing demonstration that end-to-end training could handle dense prediction at scale. U-Net (Ronneberger et al., 2015b), developed initially for biomedical image segmentation, introduced a symmetric encoder-decoder architecture with dense skip connections between corresponding layers. Where FCN connected only a few layers across the depth of the network, U-Net connected every encoding stage to its mirrored decoding stage, preserving spatial detail far more effectively. This made it particularly strong on small datasets and fine-grained boundary recovery, two properties that proved valuable well beyond its original medical imaging context. U-Net has since become one of the most widely adopted architectures in remote sensing, where labeled data is scarce and object boundaries carry real-world significance (e.g., building footprints, road edges, field delineations).

DeepLabv3+ (L.-C. Chen et al., 2018) pushed accuracy further through Atrous Spatial Pyramid Pooling (ASPP), which applies dilated convolutions at multiple rates to capture context at different spatial scales without reducing resolution. Combined with an encoder-decoder structure and a modified Xception backbone, DeepLabv3+ reached

89.0% mIoU on PASCAL VOC 2012, a remarkable figure for its time. More recently, Transformer-based segmentation models like SegFormer (E. Xie et al., 2021) have achieved competitive results using hierarchical attention rather than convolutions, though their advantage over well-tuned CNN models is less pronounced in segmentation than it has been in classification or detection. A point worth making here is that the performance gap between natural-image datasets and aerial-image datasets remains wide. On iSAID dataset (Zamir et al., 2019), which is the largest aerial segmentation benchmark, state-of-the-art semantic segmentation approaches reach around 60–65% mIoU, substantially below the 80–90% figures typical on COCO or PASCAL VOC. The aerial domain introduces complications that standard models were not designed for, such as extreme scale variation within a single image (a small vehicle versus a large-vehicle or a harbor area), dense packing of similar objects, varying GSD (Ground Sampling Distance), and non-standard orientations. These domain-specific challenges motivate the use of a foundation model (SAM) rather than a domain-trained semantic segmentation network in this thesis, as discussed in section 1.3.3.

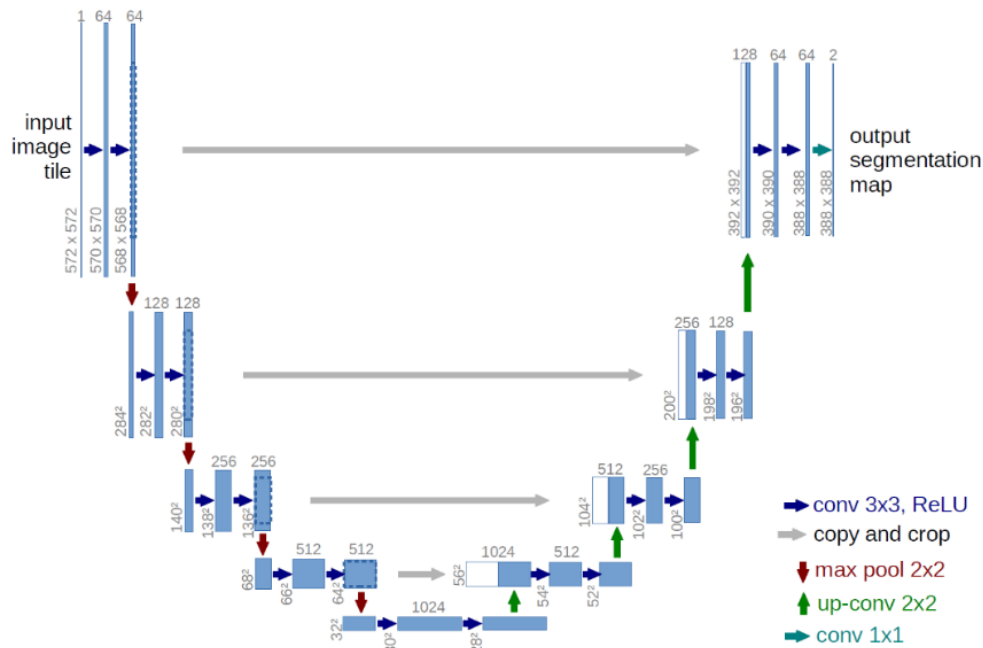


Figure 5: U-NET symmetrical architecture with skip connections between encoder and decoder layers (source: Ronneberger et al., 2015b)

1.3.2 Instance Segmentation

Instance segmentation requires the model to produce a separate mask for each individual object, e.g. not just labeling pixels as “car,” but distinguishing car-1 from car-2 even when they overlap. This is considerably harder than semantic segmentation, and for years, the dominant approach was Mask R-CNN (He et al., 2017). Mask R-CNN extends Faster R-CNN (Ren et al., 2016) by adding a small fully convolutional branch that predicts a binary mask for each Region of Interest (RoI), in parallel with the existing box regression and classification branches. A critical technical contribution was RoIAlign,

which replaces the coarse RoI pooling operation with bilinear interpolation at exact coordinates, eliminating the spatial quantization errors that degraded mask quality in earlier approaches. On the COCO benchmark, Mask R-CNN achieved 35.7% mask AP with a ResNet-101-FPN backbone while running at 5 FPS, modest by today's detection speeds, but acceptable for non-real-time applications. Its real strength was generality, as the same framework could be extended to keypoint detection, panoptic segmentation, and other structured prediction tasks with minimal modification. Single-stage instance segmentation methods followed, seeking to eliminate the two-stage overhead. YOLACT (Bolya et al., 2019) generated a set of prototype masks and then combined them linearly with per-instance coefficients predicted by a detection head, achieving real-time performance at around 30 FPS. SOLOv2 (X. Wang et al., 2020) took a different route, predicting masks by location on the image grid, avoiding the need for explicit box proposals entirely. CondInst (Tian, Shen, & Chen, 2020) used instance-conditional convolution filters, generating unique convolution kernels per instance. These methods generally achieved 30–40% mask AP on COCO, competitive with Mask R-CNN, and much faster.

In the aerial domain, instance segmentation is valuable for tasks like counting individual vehicles, tracking ships across frames, or inventorying building footprints. But there is a practical barrier that outweighs any architectural consideration, and that is the annotation cost. Producing a pixel-perfect mask for a single small vehicle in an aerial image, can take 10–100 times longer than drawing a bounding box around it. For datasets with tens of thousands of instances, which is routine in aerial imagery, the annotation cost, both in terms of time and expense, becomes prohibitive. The iSAID dataset required extensive manual effort to produce its segmentation labels, and even then, covers only 15 object categories. This fundamental cost asymmetry between detection and segmentation annotations is one of the practical problems that this thesis addresses. The main idea is that if a detection model can supply bounding boxes, and a foundation model can convert those boxes into masks without task-specific training, then the expensive annotation step can be significantly reduced, if not eliminated entirely.

1.3.3 Foundation Models: The Segment Anything Model

The Segment Anything Model (SAM) introduced by Meta (Kirillov et al., 2023) at Meta AI Research, takes a different approach to segmentation than the class-supervised models discussed in the previous sections. Instead of training a model to recognize a fixed set of classes, SAM was trained to produce a valid segmentation mask for any object indicated by a user-supplied prompt. The training data for this objective is SA-1B, a dataset of 1.1 billion masks across 11 million natural images, constructed through a model-in-the-loop annotation procedure in which a partially trained SAM was used to propose masks that human annotators then corrected, with the resulting labels used for further training. SA-1B is roughly two orders of magnitude larger than prior segmentation datasets, and this combination of scale and a promptable training objective is what gives SAM its (so called) zero-shot generalization capability, meaning the ability

to operate on visual domains and to segment object categories not explicitly represented in its training data. This is the primary reason why the SAM model is an excellent segmentation candidate for utilization in the aerial view domain, where object representations are fundamentally different from the corresponding ground view ones.

SAM introduced the term “promptable segmentation” (Figure 6), which serves as its main concept. The term refers to the fact that the segmentation output is conditioned on an explicit prompt rather than only on the input image. The model’s architecture supports four prompt types, which include one or more (foreground or background) points, an axis-aligned bounding box, a rough binary mask and free-form text. For each prompt, SAM returns a binary mask together with a confidence score. To handle cases where a prompt is geometrically ambiguous, a single point may legitimately refer to a sub-part, a part, or the whole object. The model produces three candidate masks per prompt at different granularity levels, each accompanied by a predicted IoU score and returns the mask with the highest score as the primary output. This multi-mask scheme is described, in SAM’s paper, as the model’s mechanism for handling ambiguity without requiring user intervention.

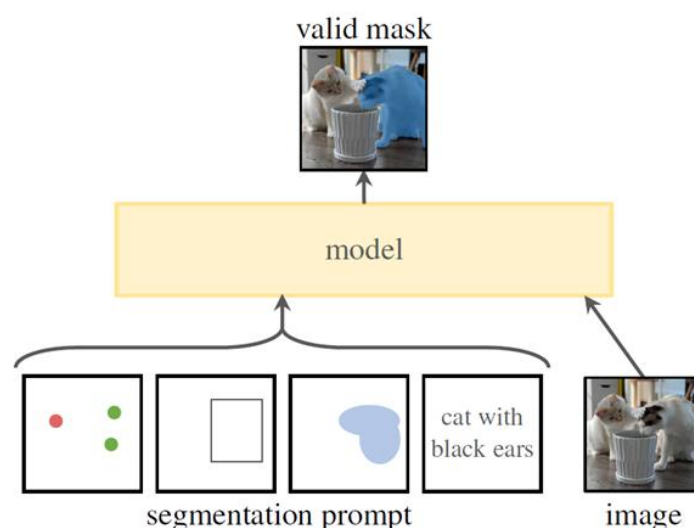


Figure 6: SAM promptable segmentation concept. Four prompt types each produce a valid mask.

(Source: Kirillov et al., 2023)

SAM's architecture is composed of three main components (Figure 7 **Error! Reference source not found.**), namely an Image Encoder, a Prompt Encoder, and a Mask Decoder. The image encoder is run once per image to produce a feature embedding, which can then be reused across multiple prompt queries against the same image. The prompt encoder converts the user's prompt into a set of tokens (or feature representations) and the mask decoder combines both inputs to produce the final mask. Decoupling the image encoding from the prompt encoding is what makes SAM usable in interactive settings, where the image is fixed but the prompt may change many times, thus making the expensive part of the computation (image encoding) reusable across all subsequent queries, improving inference time.

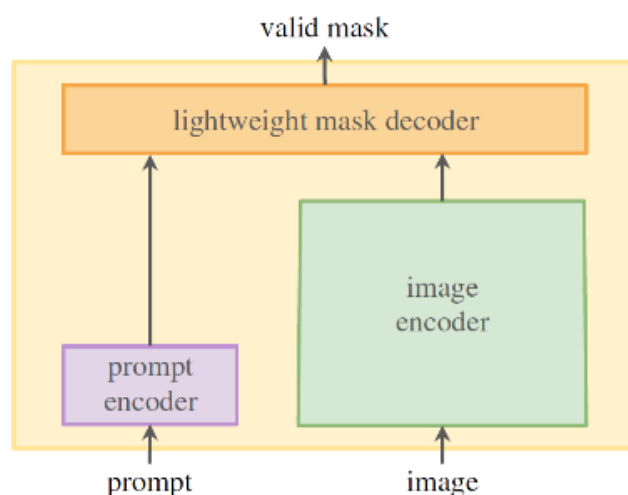


Figure 7: High-level architecture of the Segment Anything Model

(Source: Kirillov et al., 2023)

The **image encoder** is the component responsible for most of SAM's inference cost. It is a Vision Transformer (ViT), initialized from a Masked Autoencoder (MAE) pretraining checkpoint (He et al., 2021) before being trained on SA-1B as part of the full SAM pipeline. The input image is resized and padded to 1024 x 1024 pixels, then partitioned into non-overlapping 16 x 16 patches. Each patch is linearly projected to a token, and the resulting sequence is processed by the Transformer. To make this tractable at high resolution, SAM uses a windowed attention scheme. Most layers restrict self-attention to 14 x 14 token windows, while four (evenly spaced through the network) layers apply global self-attention across the full sequence. After the Transformer, a 1x1 convolution reduces the channel dimension and a 3 x 3 convolution refines it, producing the final image embedding of dimensions 64 x 64 x 256. The encoder's cost is fixed per image and does not depend on the number of prompts subsequently issued.

Three encoder variants are released, differing in depth and width but sharing the same architectural template. ViT-H (Huge) has 632 million parameters, 32 Transformer layers and 16 attention heads per layer. ViT-L (Large) has 308 million parameters across 24 layers with 16 heads. ViT-B (Base) has 91 million parameters across 12 layers with 12 heads. All three produce a 64 x 64 x 256 embedding with identical interfaces, so the prompt encoder and mask decoder are unchanged across variants making the image encoder the only component that varies.

Variant	Parameters	Layers	Attn. Heads	Window Size
ViT-H (Huge)	632M	32	16	14×14
ViT-L (Large)	308M	24	16	14×14
ViT-B (Base)	91M	12	12	14×14

Table 2: SAM image encoder variants

The **prompt encoder** converts user-supplied prompts into vector representations consumed by the mask decoder. SAM distinguishes between sparse prompts (points and boxes) encoded as token sequences and dense prompts (masks) encoded as a spatially aligned feature map that is added to the image embedding before decoding. Each prompt type uses a dedicated encoding mechanism. In point prompting, each point is represented by the sum of two components, namely a Fourier positional encoding of its (x, y) coordinates, capturing spatial location and a learned embedding that indicates whether the point is foreground (positive, on the target object) or background (negative, on a region to exclude). The foreground/background embeddings are learned during training. Multiple points can be combined in a single prompt to refine the segmentation iteratively. In bounding box prompting, a box is encoded as two corner tokens. The top-left and bottom-right corners are used, which are represented by their positional encodings summed with the learned “top-left corner” and “bottom-right corner” embeddings correspondingly. This corner-token scheme encodes both the spatial location of each endpoint and its semantic role, without explicitly representing box dimensions. In text prompting³, a free-form text is encoded by CLIP’s (Radford et al., 2021) text encoder, a pretrained model that maps language and images to a shared embedding space, enabling language-conditioned segmentation. Finally, in mask prompting, a rough binary mask (e.g. from a previous SAM prediction in an iterative workflow) can be provided as a dense prompt. The mask is downsampled by 4x and passed through two 2 x 2 stride-2 convolutions, that progressively increase the channel count to 16, followed by a 1x1 convolution, thus projecting to 256 channels to match the image embedding dimension. GELU activations and layer normalization are applied between convolutions. The resulting feature map is added elementwise to the image embedding. If no mask prompt is provided, a learned “no mask” embedding is broadcast across the spatial domain. This mechanism is one reason SAM supports iterative segmentation refinement, as a previous predicted mask can later be fed back as a mask prompt for improved segmentation. When that iterative refinement is absent, the “no mask” embedding substitutes it.

The **mask decoder** (Figure 8) is a Transformer-based module that fuses the image embedding and the prompt embeddings to predict the segmentation mask. It is small relative to the image encoder, so its per-prompt latency is low, as once the image embedding is cached, additional prompts incur only the decoder cost. The decoder operates on a unified token set comprising the encoded prompt representations and a set of learned output tokens (one per candidate mask, plus one for IoU prediction). Each decoder layer applies four operations in the following sequence:

- Self-attention on all tokens, so prompt and output tokens can interact and exchange information.

³ The text prompting capability is described in (Kirillov et al., 2023) but was not included in the public release of SAM by Meta AI and is therefore not used in this work.

- Cross-attention from tokens to the image embedding (tokens as queries Q, image embedding as keys K and values V), so tokens can pull spatial features relevant to the prompt.
- A point-wise MLP update on each token, applied independently per token.
- Cross-attention from the image embedding to the tokens (image as queries Q, tokens as keys K and values V), so the spatial feature map is updated using the current prompt context.

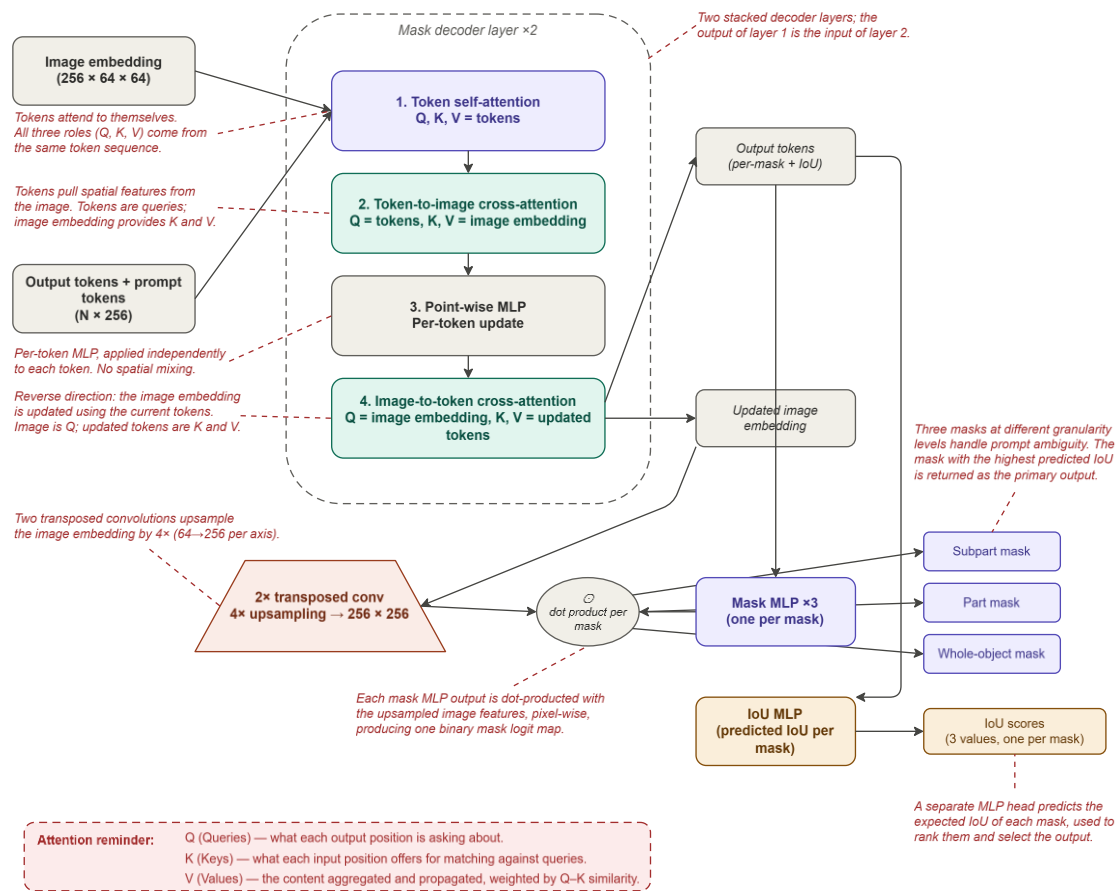


Figure 8: The SAM mask decoder

This bidirectional cross-attention, where tokens attend to image and then image attends to tokens, distinguishes SAM's decoder from a standard Transformer decoder, where information typically flows in one direction. Two decoder layers are applied. The image embedding is then upsampled by $4\times$ using transposed convolutions, each mask output token is projected through a small three-layer MLP and combined with the upsampled features via a dot product, producing a binary mask logit for each candidate mask. A sigmoid function converts logits to probabilities and the masks are upsampled to the original image resolution. The IoU output token is processed by a separate MLP that predicts the expected IoU of each candidate mask. SAM produces three candidate masks per prompt (Figure 9), ranked by predicted IoU and returns the highest-scoring mask.

This is the mechanism that allows the model to handle prompt ambiguity (e.g., a point on a pair of scissors could refer to the left or right handle, or the whole object) by generating all three interpretations and letting the IoU head select among them.

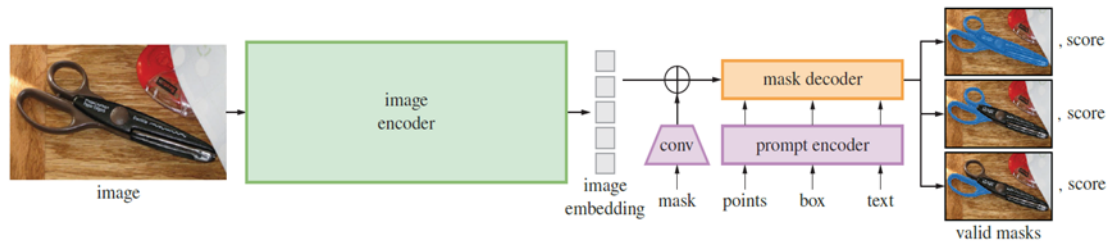


Figure 9: SAM's segmentation candidate masks for a pair of scissors

(Source: Kirillov et al., 2023)

SAM's box-prompt capability is what makes it usable as a segmentation backbone behind an object detector. When a bounding box is supplied as a prompt, SAM treats it as a spatial prior constraining where the target object lies and produces a mask within that region. Any detector, including the YOLO models evaluated in this thesis, can therefore pass its output boxes to SAM, producing pixel-level instance masks without joint training, a shared backbone, or any segmentation-level annotations. Detector and segmentation components remain decoupled and can be updated independently. One constraint, though, is that SAM's box prompt interface accepts only axis-aligned (VBBs) bounding boxes in $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$ format. The YOLO models used in this thesis produce oriented bounding boxes (OBBs), so a coordinate conversion step is required, that takes the minimum and maximum x and y values across the four OBB corner points and uses them to construct the enclosing axis-aligned rectangle. The conversion preserves spatial coverage of the target but, for highly rotated objects, the axis-aligned box encloses additional background pixels that would not be present in the OBB. The downstream effect of this on mask quality is examined in Chapter 4.

Two aspects of SAM's inference cost matter for aerial imagery deployment. First, the image encoder cost is fixed per image, as it processes the full 1024×1024 input in a single forward pass and does not scale with the number of objects present. Second, the released SAM API does not accept batched box prompts, so when a detector produces several detection boxes in one image, each box requires a separate sequential call to the mask decoder. The decoder is fast (a few milliseconds per call), but the lack of batching means that per-image processing time grows linearly with object count. This becomes the dominant cost in crowded aerial scenes containing dozens or hundreds of objects.

Since SAM was released, a number of derivative models have addressed specific limitations of the original architecture. MobileSAM (C. Zhang et al., 2023) replaces the ViT image encoder with a distilled lightweight encoder, reporting comparable mask quality at roughly $5\times$ the throughput. EfficientSAM (Xiong et al., 2023) uses masked image pretraining to produce a smaller encoder that serves as a drop-in replacement. FastSAM (Zhao et al., 2023) departs from the ViT paradigm entirely, using a YOLO-

based convolutional architecture to approximate SAM's outputs at higher throughput, with some loss in mask quality. SAM 2 (Ravi et al., 2024) extends the paradigm to video, by adding a memory mechanism that propagates segmentation across frames, supporting object tracking and temporally consistent masks. In the remote sensing domain, RSPrompter (K. Chen et al., 2023a) learns to generate task-specific prompts for aerial imagery automatically and SAMRS (D. Wang et al., 2023a) uses SAM to generate pseudo-label masks from existing detection annotations, scaling up segmentation dataset construction. PerSAM (R. Zhang et al., 2023a) introduces a one-shot personalization procedure based on a cosine-similarity confidence map between a reference image and the target image, enabling class-specific segmentation from a single annotated example without further training. None of these derivatives are evaluated in the present work, which uses the original SAM architecture, but they are discussed as candidates for future pipeline optimization in Chapter 5.

In this thesis SAM serves as the segmentation backbone of the Detection→Segmentation pipeline, receiving bounding box prompts from a YOLOv11x-OBB detector and producing pixel-level instance masks. The question being asked is not whether SAM achieves state-of-the-art performance on aerial segmentation benchmarks, since domain-trained models typically perform better in absolute terms, but whether SAM's zero-shot mask quality, given detector-derived bounding box prompts, is high enough to support a practically useful Detection→Segmentation pipeline across the 15 object categories of our dataset, without any segmentation-level training data. Chapter 4 reports SAM's mask quality under ground-truth box prompts, isolating segmentation performance from detection errors, and Chapter 5 reports end-to-end pipeline performance using detector-generated prompts. Together they quantify how well the modular Detection→Segmentation approach works in the aerial domain and where its main failure modes lie.

1.4 Aerial and Satellite Imagery Datasets

Progress in aerial computer vision is bounded almost as tightly by the available datasets as it is by the models. This section reviews the datasets most relevant to detection and segmentation in the aerial and satellite domain, ordered by task. The aim is not to enumerate every collection in circulation, as the datasets in community public repositories are hundreds but to characterize the resources that have shaped the state of the art and frame the methodological choices in this thesis. DOTA v1.0 and iSAID, the two primary datasets used in the experiments, are introduced here only briefly, but a thorough exploratory data analysis (EDA) of both is presented in Chapter 2. Finally, this section identifies the structural gaps in the current dataset landscape that this thesis responds to.

1.4.1 Detection Datasets

The detection landscape in aerial imagery has been shaped by a handful of large, multi-class benchmarks released between 2018 and 2022. Earlier datasets such as NWPU VHR-10 (G. Cheng et al., 2014) with 10 categories and around 800 images or RSOD (Y. Long et al., 2017) with 4 classes and 976 images, established the format conventions but were too small to support modern training-from-scratch workflows. In general, one can argue that the contemporary benchmarks fall into three main groups, namely the general-purpose multi-class detection ones, the fine-grained recognition ones and the task-specific collections such as small-object or single-class detection. Some of the most important detection datasets are presented here and a summary of them can be seen in Table 3.

DOTA v1.0 (G.-S. Xia et al., 2019) is the de facto standard for oriented object detection in aerial imagery. It contains 2806 images at a wide range of spatial resolutions (0.12 m to 3.0 m GSD) collected from Google Earth, the Chinese JL-1 satellite and the Gaofen-2 satellite, with 188282 instances annotated as oriented bounding boxes across 15 categories like vehicles, ships, aircraft, sports infrastructure, and urban features. Later versions (v1.5 and v2.0) add some categories and small-object annotations, reaching approximately 1.8 million instances, but v1.0 remains the most widely used release. Detailed analysis of DOTA v1.0 is presented in Chapter 2.

DIOR (K. Li et al., 2020) provides a horizontal bounding box alternative, with broader category coverage. It contains 23463 images at 0.5-30 m resolution with 192518 instances across 20 categories including airports, dams, golf courses, expressway service areas and other complex compound objects that DOTA does not cover. DIOR images are uniformly 800x800 pixels, which simplifies training pipelines but loses the multi-scale challenge that DOTA preserves. The annotation format is axis-aligned only.

FAIR1M (Sun et al., 2022) is the largest fine-grained aerial detection dataset currently released, with over 15000 high-resolution images (0.3-0.8 m GSD) from Gaofen satellites and Google Earth, containing approximately one million instances. Its distinguishing feature is the depth of category breakdown. 5 top-level categories (ships, vehicles, airplanes, courts, roads) decompose into 37 fine-grained sub-categories, such as Boeing 737 versus Airbus A220, or passenger ship versus liquid cargo ship. Annotations are OBBs. FAIR1M targets a different research question from DOTA, regarding fine-grained recognition rather than general detection and the two are complementary rather than competing.

xView (Lam et al., 2018a) was released by the U.S. Defense Innovation Unit, as a large-scale, fine-grained detection benchmark. It contains over one million objects across 60 classes, in approximately 1400 km² of Vantor's (formerly MAXAR) WorldView-3 imagery at 0.3 m GSD. Its category ontology is hierarchical (8 parent classes, 60 leaf classes) and includes object types not present in any other public benchmark, such as

specific construction vehicle subtypes. Annotations are VBBs only and are known to contain some mislabeled and missed instances. The dataset's own construction protocol set a relatively permissive quality bar since batches were accepted at a precision threshold of 0.75 against expert-annotated gold-standard chips at IoU 0.5 (Lam et al., 2018b). (Shermeyer & Van Etten, 2018) in their super-resolution study, explicitly illustrate ground-truth errors in xView including incorrectly sized cars, erroneous boat labels and missing vehicles, noting that these issues inflate false-positive rates and produce spurious misclassifications during evaluation. Moreover, although the WorldView-3 satellite sensor captures eight multispectral bands, the publicly released xView imagery is RGB-only, limiting the dataset's utility for studies that would otherwise exploit valuable near-infrared (NIR) or short-wave infrared (SWIR) signals.

VisDrone (P. Zhu et al., 2018) targets the UAV derived views, rather than satellite imagery. It consists of 10209 images and 261908 video frames captured by drones over 14 Chinese cities, annotated with 2.6 million bounding boxes across 10 categories of vehicles and pedestrians. Spatial resolution is variable but generally much higher than satellite imagery and viewing angles are often oblique rather than strictly nadir, which makes VisDrone closer to traffic surveillance than to remote sensing.

AI-TOD (J. Wang, Xu, et al., 2021) goes further in the small-object direction, using annotations for a mean object size of 12.8 pixels, half that of DOTA, thus becoming the primary benchmark for evaluating tiny-object detection methods.

Dataset	Images	Instances	Classes	Annotation	GSD / Source
DOTA v1.0	2806	188282	15	OBB	0.12–3.0 m / Google Earth, JL-1, GF-2
DIOR	23463	192518	20	VBB	0.5–30 m / Google Earth
FAIR1M	~15000	~1000000	5 / 37 fine-grained	OBB	0.3–0.8 m / Gaofen, Google Earth
xView	~1400 km ²	>1000000	60	VBB	0.3 m / WorldView-3
VisDrone	10209 + video	~2600000	10	VBB	UAV / variable
AI-TOD	28036	700621	8	VBB	mixed / tiny objects (mean 12.8 px)
NWPU VHR-10	800	3651	10	VBB	0.5–2 m / Google Earth, Vaihingen

Table 3: Comparison of detection datasets in the aerial and satellite domain

1.4.2 Segmentation Datasets

Pixel-level segmentation datasets are scarcer than detection datasets, primarily because mask annotation is one to two orders of magnitude more expensive per instance. The aerial segmentation landscape can be divided as the detection equivalent, in general-

purpose multi-class instance and semantic segmentation ones (where the only realistic large-scale option is iSAID) and task-specific single-class or land-cover segmentation ones, where a number of smaller, more focused datasets exist.

iSAID (Zamir et al., 2019) is the segmentation companion to DOTA. It uses the same 2806 images and the same train/validation/test splits as DOTA's v1.0, with pixel-level instance masks added for 655451 instances across the same 15 categories. Each instance is stored as an RGB-coded PNG, where a unique color identifies each class (background is black) and additional metadata distinguishes individual instances of the same class. The iSAID instance count is substantially higher than DOTA's (655451 vs 188282), reflecting a more inclusive annotation policy. iSAID annotators included objects that DOTA annotators omitted, particularly very small vehicles and partially occluded instances. This discrepancy is detected and examined in Chapters 2 and 3 in the context of our AerialFuseCV dataset creation.

SpaceNet (Van Etten et al., 2018) is a long-running competition series that has released building, road and flood detection datasets covering multiple cities worldwide. Imagery is sourced from Vantor's WorldView-2 and WorldView-3 sensors and is delivered as multispectral GeoTIFFs with full georeferencing, a feature that makes SpaceNet structurally different from the Google Earth derived benchmarks. The trade-off is narrower class coverage (most challenges focus on a single object type, buildings or roads) and limited geographic generalization, since each challenge typically covers fewer than ten cities. SpaceNet 7 added a multi-temporal urban development task using PlanetScope's monthly mosaics.

LoveDA (J. Wang, Zheng, et al., 2021) targets domain adaptation for land cover. It contains 5987 aerial images at 0.3 m GSD spanning urban and rural scenes in three Chinese cities, with semantic masks for seven classes (building, road, water, barren, forest, agriculture, background). The deliberate urban and rural split makes it the standard benchmark for cross-domain semantic segmentation, but its RGB-only spectral resolution design and modest size limit its use as a general aerial segmentation resource.

Inria Aerial Image Labeling (Maggiori et al., 2017) provides 810 km² of orthorectified RGB imagery at 0.3 m GSD, across ten cities (Austin, Chicago, Vienna, etc.), with binary (building/not-building) masks. It remains a common building-extraction benchmark and is often used to demonstrate cross-city generalization of segmentation models.

ISPRS 2D Semantic Labeling (the Potsdam and Vaihingen benchmarks) provides true orthophotos at 5 cm and 9 cm GSD respectively, with six-class semantic masks and accompanying digital surface models. Both datasets are small (Potsdam has 38 patches, Vaihingen has 33) but they are notable for being among the few aerial datasets with co-registered elevation data and per-pixel intensity from a calibrated sensor, rather than RGB exported from a consumer platform.

FLAIR (Garioud, Gonthier, et al., 2023) and FLAIR-2 (Garioud, De Wit, et al., 2023) are more recent, large-scale segmentation benchmarks, released by France's national mapping agency (IGNF). FLAIR contains 77412 patches at 0.2 m GSD with 19 land cover classes and FLAIR-2 extends this with co-registered Sentinel-2 time series to enable multi-modal segmentation experiments. It is one of the few aerial datasets to combine very-high-resolution aerial imagery with paired moderate-resolution satellite data in a single benchmark.

Beyond these, the segmentation landscape contains a long tail of specialized datasets, such as SAMRS(D. Wang et al., 2023b), produced by applying SAM to existing detection datasets to generate pseudo-masks, OpenEarthMap (J. Xia et al., 2022) a global high-resolution land cover benchmark, DeepGlobe(Demir et al., 2018) road and land cover at coarser resolutions, as well as many task-specific collections for solar panels, parking lots, agricultural fields and similar narrowly defined targets. None of these provide the combination of multi-class instance annotations, broad category coverage, and paired detection labels that the DOTA+iSAID pairing offers, which is the principal reason this thesis adopts them as its base data sources.

Dataset	Images / Coverage	Classes	Type	GSD / Source / Notes
iSAID	2806 images	15	Instance seg.	Same images as DOTA v1.0, 655451 masks
SpaceNet (various)	Multi-city	1 (per challenge)	Semantic seg.	0.3–0.5 m / WorldView-2/3 / georeferenced GeoTIFF
LoveDA	5987	7	Semantic seg.	0.3 m / aerial / urban–rural domain split
Inria AIL	810 km ²	1 (building)	Binary seg.	0.3 m / aerial / 10 cities
ISPRS Potsdam	38 patches	6	Semantic seg.	5 cm / true orthophoto + DSM
ISPRS Vaihingen	33 patches	6	Semantic seg.	9 cm / true orthophoto + DSM
FLAIR-1	77412 patches	19	Semantic seg.	0.2 m / French aerial / large-scale land cover

Table 4: Comparison of segmentation datasets in the aerial and satellite domain.

1.4.3 The Dataset Gap

A clear gap in the aerial imagery landscape is the absence of ready-made, unified datasets that pair detection annotations with segmentation masks for the same instances, in formats directly consumable by modern detection and segmentation models. DOTA provides OBB labels but no pixel masks. iSAID provides pixel masks but no detection-format labels. The two were created independently, by different annotation teams, using

different tools and conventions. Even though the underlying images are identical, merging the two into a single usable dataset is far from trivial, since filenames differ, and instance-level bboxes' and segmentations masks' correspondence must be verified manually or algorithmically. The absence of paired detection–segmentation data has meant that detection and segmentation models for aerial imagery are almost always evaluated in isolation. In that way, a YOLO paper reports mAP on DOTA, a segmentation paper reports mIoU on iSAID and no one can evaluate how well a detector's outputs feed into a segmentation model. For a Detection→Segmentation pipeline like the one proposed in this thesis, this kind of end-to-end evaluation is essential.

Beyond the absence of paired annotations, the publicly available aerial datasets (e.g. DOTA v1.0, iSAID, xView, DIOR and most of the UAV-derived collections) share a second, less frequently discussed limitation that constrains what models trained on them can ultimately do. The imagery in these datasets is almost exclusively 8-bit, three-channel RGB, much of it exported from consumer-facing platforms such as Google Earth. While this format is convenient and matches the input expectations of off-the-shelf computer vision models, it discards most of the spectral and radiometric information that a modern EO sensor actually captures. A satellite imaging sensor like WorldView-3 records 8 multispectral bands at 16-bit radiometric depth, alongside its 0.3 m panchromatic channel. Sentinel-2 records 13 bands at 12-bit depth and even moderate-resolution platforms typically deliver at least red, green, blue, and near-infrared at 10- or 12-bit precision. Converting any of these to 8-bit RGB collapses the dynamic range from tens of thousands of distinct intensity values down to just 256 (2^8 tones) and discards the spectral bands that distinguish vegetation from artificial surfaces, water from shadow, bare soil from concrete and so on. The information loss is substantial and cannot be recovered downstream. The practical consequence is that models trained on RGB-only aerial datasets learn to recognize objects from their visible-spectrum appearance alone, even when the operational deployment scenario could supply much richer input. A YOLO model trained on DOTA cannot, by construction, exploit near-infrared reflectance to disambiguate a painted parking lot from a green field, or use a thermal band to distinguish a recently active vehicle from a parked one. Foundation models like SAM inherit the same limitation, since their training corpus (SA-1B with 11 million RGB images) contains no multispectral or hyperspectral examples at all. Whether the disregarded spectral information would, in fact, improve detection or segmentation for the object categories considered here remains an open question, that the current datasets cannot answer, as there is no equivalent of DOTA or iSAID built on full-radiometric, multispectral source imagery. The methodology of this thesis, therefore, necessarily operates within the RGB-only domain, but the limitation should be understood as a property of the available data, not an inherent ceiling on what aerial computer vision pipelines could achieve with appropriately constructed datasets.

A related, but equally consequential omission, is the absence of georeferencing information in the public aerial benchmarks. Each DOTA image is a JPEG or PNG file

with no associated geographic metadata, such as coordinate reference system, affine transformation (linking pixel coordinates to map coordinates), or at least an acquisition footprint. The same is true for iSAID, xView and for the great majority of comparable datasets. This is again a consequence of their construction from imagery sources like Google Earth and of the fact that the majority of the computer vision communities address the issue mainly in computer science terms, rather than with downstream real geospatial applications in mind, such as Geospatial Engineers do. The result is that any object a model detects or segments on such an image, exists only in image-pixel space and cannot be projected back into a real-world coordinate frame without an additional, dataset-external step. For applications that consume detection or segmentation outputs as inputs to a GIS workflow (building footprint extraction, road network updating, vehicle traffic monitoring, change detection, damage assessment etc.) the absence of georeferencing turns a successful inference into an unusable artefact. A 95% IoU mask of a building is of limited value if its location on the Earth's surface is unknown. In operational remote-sensing settings, every native sensor product (a GeoTIFF from a satellite tasking, an orthophoto from a UAV flight, a Sentinel-2 tile from the Copernicus archive) is delivered with full geospatial metadata as a matter of course. The disconnect between how public aerial datasets are distributed and how real remote-sensing data is structured introduces friction during deployment, since a model trained and evaluated on datasets like DOTA produces pixel-space outputs that must be re-projected, often through a custom integration layer, before they can be consumed by a downstream geospatial system. This friction is rarely acknowledged in the published evaluation literature, but it directly affects the engineering effort required to put aerial computer vision pipelines into production.

Taken together, these three properties of the current aerial dataset landscape (the absence of paired detection–segmentation labels, the spectral and radiometric downgrade of the source imagery and the loss of georeferencing information) define a gap that is wider than is usually acknowledged. The first of these gaps, is what the AerialFuseCV dataset, presented in this thesis, directly addresses by reconciling DOTA bounding boxes with iSAID masks at the instance level for end-to-end pipeline evaluation. Its construction methodology is described in Chapter 2. The other two are structural properties of the available benchmarks that this thesis cannot resolve within its scope.

1.5 Related Work

The individual components of the Detection→Segmentation pipeline, namely the YOLO detector and the SAM segmenter, have each been studied in the aerial domain, but almost always in isolation. This section surveys the relevant work regarding YOLO-based detection on aerial benchmarks, SAM applied to remote sensing imagery, and prior attempts at detection-to-segmentation pipeline architectures.

1.5.1 YOLO in Aerial Imagery

YOLO has been applied to aerial object detection since YOLOv3, which achieved 40–50% mAP on DOTA, but struggled visibly with small objects and dense scenes (e.g., harbors filled with dozens of closely packed ships). The primary limitations were the relatively coarse feature resolution of Darknet-53 and the lack of orientation-aware anchors, which made the model blind to the rotational diversity of aerial objects. These early results established that YOLO could operate in the aerial domain but left significant room for improvement. YOLOv5 through YOLOv7 brought substantial progress. Community-developed OBB extensions and demonstrated that orientation-aware detection was feasible within the YOLO framework, though these extensions required non-trivial modifications to the loss function, anchor design, and NMS procedure. Published results on DOTA using these versions typically reported 50–60% mAP for oriented detection, competitive with purpose-built oriented detectors like RoI Transformer (Ding et al., 2018b) and Oriented R-CNN (X. Xie et al., 2021). The Ultralytics ecosystem played a major role in democratizing the task, as pretrained weights, standardized training scripts and model-scaling support made YOLO accessible to researchers who lacked the engineering resources (or background) to implement custom detectors from scratch.

With YOLOv8 and YOLO11, OBB detection became a native, first-class task rather than a community patch. This integration simplified training pipelines and improved stability. However, the published literature on these newer versions applied to DOTA remains surprisingly sparse. Most DOTA-focused papers still benchmark against older architectures (Faster R-CNN, RetinaNet, Rotated R-CNN) and the handful that evaluate YOLOv8+ often use DOTA v1.5 or v2.0 rather than v1.0, making direct comparison difficult. There is, to the author’s knowledge, no published work that systematically benchmarks the full range of YOLOv8 through YOLO11 variants, in both VBB and OBB configurations, on DOTA v1.0. This thesis fills that gap with a 34-model benchmark spanning five generations and two bounding box variants, as documented in Chapter 4.

1.5.2 SAM in Remote Sensing

Since SAM’s release in April 2023, the remote sensing community has explored its zero-shot capabilities with considerable enthusiasm. (Osco et al., 2023) provided one of the earliest systematic evaluations, testing SAM across UAV, airborne and satellite imagery with point, box and text prompts. Their findings were mixed, as SAM performed well on large, well-defined objects like buildings and roads (60-70% IoU with box prompts) but struggled with small or densely packed objects like vehicles (20-30% IoU with point prompts). (Ren et al., 2023) reach similar conclusions in a systematic study, reporting that SAM generalizes reasonably well to overhead imagery for many common target types but exhibits characteristic failure modes on objects with low contrast against the background, ambiguous boundaries, or of unusual scale. A separate line of work has

focused on adapting SAM to the remote sensing domain through fine-tuning. RSPrompter (K. Chen et al., 2023b) learns to generate prompts automatically for aerial imagery, while SAMRS (Du et al., 2023) uses SAM itself to scale up remote sensing segmentation datasets by generating pseudo-labels from existing detection annotations (presenting a concept closely related to the pipeline approach of this thesis), though SAMRS focuses on dataset construction rather than real-time pipeline deployment. Combinations of Grounding DINO (S. Liu et al., 2023) and SAM have emerged as a practitioner-driven workflow for semi-automated annotation, with empirical evaluations in specialized domains, demonstrating both the promise and the limitations of the approach (Mumuni & Mumuni, 2024). Adaptation to the aerial domain specifically remains underexplored in the peer-reviewed literature.

What is notably absent from this body of work is a systematic evaluation of SAM on DOTA-class aerial objects using detection-derived bounding box prompts across all 15 categories, with both box and point prompt strategies and across all three model variants. Most existing studies either use a subset of categories, test on a different dataset, or evaluate only a single SAM variant. The SAM baseline evaluation in this thesis (Chapter 4) addresses this gap directly, providing per-class IoU results for six SAM configurations (3 variants $\times$ 2 prompt types) on the full AerialFuseCV validation set of 28032 instances.

1.5.3 Detection-to-Segmentation Pipelines

The idea of using detection outputs to guide segmentation is not new. Mask R-CNN itself can be viewed as an integrated detection-to-segmentation pipeline, though it requires pixel-level mask supervision during training. Weakly supervised approaches have attempted to derive masks from bounding box annotations alone, like BoxInst (Tian, Shen, Wang, et al., 2020), that generates pseudo-masks from box labels using color-based affinity, eliminating the need for pixel-level annotation during training. These methods produce coarser masks than their fully supervised counterparts and are typically evaluated on natural-image benchmarks such as COCO, with their performance on aerial imagery remaining underexplored.

The emergence of SAM opened a genuinely new possibility: a pipeline where the detector requires only bounding box labels for training and the segmentation model requires no task-specific training at all. The detector learns from detection annotations, produces boxes at inference and SAM converts those boxes to masks zero-shot. The total annotation requirement is bounded by the detection dataset alone. Several groups have explored this idea conceptually (e.g., the SAMRS dataset pipeline), but published implementations that evaluate the full pipeline end-to-end, measuring detection accuracy cascading into segmentation quality, with per-class analysis and prompt-strategy comparison, are rare (if not totally absent) in the aerial/satellite imagery domain.

This thesis implements and evaluates exactly such a Detection→Segmentation pipeline, coupling a YOLO detector with SAM in a strictly modular architecture. The detector

produces bounding boxes, the boxes are converted to SAM-compatible prompts, and SAM generates instance masks. No components share weights, no joint fine-tuning is performed and no training regime with segmentation annotations is being performed. The end-to-end evaluation measures how detection errors propagate into segmentation quality, providing a realistic assessment of pipeline performance under real-world conditions where ground-truth boxes are unavailable.

1.6 Research Gaps and Thesis Positioning

The literature review in the preceding sections reveals several gaps that this thesis addresses. These can be summarized as follows:

- **Gap 1: No systematic SAM evaluation on DOTA-class aerial objects.** Existing SAM evaluations in remote sensing either use different datasets (SpaceNet, LoveDA), test a subset of categories, or evaluate only one model variant and one prompt strategy. No published study benchmarks all three SAM variants (ViT-H, ViT-L, ViT-B) with both box and point prompts across 15 DOTA categories on a dataset with verified instance-level ground-truth.

- **Gap 2: No comprehensive YOLO benchmark for modern versions on DOTA v1.0.** While individual papers have evaluated one or two YOLO versions on DOTA, there is no single study that systematically benchmarks YOLOv8 through YOLO12 in both VBB and OBB configurations on the same dataset, under identical training conditions. The 34-model benchmark in Chapter 4 is, to the authors' knowledge, one of the most comprehensive YOLO evaluations conducted on aerial imagery to date.

- **Gap 3: No end-to-end evaluation of a modular YOLO+SAM pipeline for aerial imagery.** The YOLO-to-SAM pipeline concept has been discussed conceptually and partially implemented for dataset construction (SAMRS), but no published work evaluates the complete pipeline end-to-end on a multi-class aerial benchmark, measuring how detection errors propagate into segmentation quality with per-class breakdowns and prompt-strategy analysis.

- **Gap 4: No unified detection-segmentation dataset for aerial imagery.** DOTA and iSAID share the same images but were never designed to be used together. No publicly available dataset pairs oriented bounding boxes with pixel-level segmentation masks at the instance level, in formats compatible with both YOLO and SAM. The AerialFuseCV dataset, created through a systematic fusion and refinement process documented in Chapter 2, fills this gap.

- **Gap 5: Limited analysis of prompt-strategy effects on aerial segmentation quality.** While the choice between box and point prompts is known to affect SAM

performance, no study has quantified this effect across 15 aerial object classes with varying sizes, shapes, and densities.

This thesis is positioned at the intersection of these gaps. It does not introduce a new detection or a new segmentation architecture. Instead, it asks a practical question, whether given the best available off-the-shelf models (modern YOLO for detection and SAM for segmentation) how well does a modular, training-free pipeline perform on aerial imagery and what are the conditions under which it succeeds or fails? The answer involves creating the dataset (AerialFuseCV), running the experiments (34 YOLO models, 6 SAM configurations, 1 end-to-end pipeline) and analyzing the results in an in-depth and systematic way, which is not often performed in the photogrammetry and remote sensing domain.

Chapter 2 Relevant Public Datasets

Aerial object detection and segmentation impose requirements that are not adequately addressed by generic computer vision datasets, particularly with respect to annotation formats and spatial characteristics. Chapter 2 introduces the external datasets used in this study and analyzes their properties, annotation schemes, and inherent limitations. Specifically, it describes the DOTA v1.0 dataset, which provides oriented bounding box annotations for aerial object detection, and the iSAID dataset, which offers pixel-level semantic segmentation masks for aerial imagery. These datasets constitute the raw data sources on which all subsequent experiments are based. By examining their class definitions, annotation conventions, spatial characteristics, and known inconsistencies, this chapter establishes the constraints under which the proposed methodology operates. The insights presented here motivate the methodological choices described in Chapter 3, where these datasets are integrated into a unified evaluation framework.

2.1 DOTA v1.0 Dataset Exploratory Data Analysis (EDA)

The Dataset for Object detection in Aerial images (DOTA) v1.0 (G.-S. Xia et al., 2019) was selected as the primary detection benchmark, as DOTA is among the most widely used and comprehensive multi-class aerial detection datasets available at the time of this research, providing sufficient data diversity for a robust evaluation. Moreover, DOTA provides native oriented bounding box (OBB) annotations, which are essential for the research questions regarding the necessity of rotation-aware detection in aerial imagery and, finally, DOTA has become the de facto benchmark in the aerial computer vision community, enabling direct comparison with state-of-the-art methods.

DOTA v1.0 consists of 2806 high-resolution aerial images, collected from various platforms and sensors, including Google Earth, the Chinese satellite JL-1, and the Gaofen-2 (GF-2) satellite. The images exhibit significant diversity in spatial resolution, ranging from 0.12 meters to 3.0 meters ground sample distance (GSD), geographic coverage spanning multiple countries and environments, and sensing conditions including varying seasons, illumination, and atmospheric conditions. Image dimensions vary from 800×800 pixels to 4000×4000 pixels, with most images approximately 2848×2848 pixels. The dataset contains 188282 object instances across 15 categories relevant to EO applications. Each object is annotated with an oriented bounding box specified by four corner points ($x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4$), which preserve rotation information. The 15 object categories include: plane, ship, storage-tank, baseball-diamond, tennis-court, basketball-court, ground-track-field, harbor, bridge, large-vehicle, small-vehicle, helicopter, roundabout, soccer-ball-field, and swimming-pool. Three official splits are provided:

- Training split contains 1411 images with 137923 annotated instances, used for model training.

- Validation split contains 458 images with 28853 instances, used for hyperparameter tuning and monitoring during training.
- Test split contains 937 images with 21506 instances, where ground-truth labels are withheld and evaluation is conducted through the official DOTA competition server with limited submission attempts.

The test set ground-truth labels are not publicly available, limiting local evaluation and iterative experimentation. The official evaluation server imposes submission limits incompatible with extensive ablation studies. The validation split is widely used for ablation and reproducible local evaluation, while leaderboard comparisons are typically reported on the test split via the official server.

Figure 10 presents the detailed class distribution across training and validation splits. The dataset exhibits significant class imbalance, a characteristic inherent to real-world aerial imagery. Ships constitute the largest class with 37028 instances (28.96% of the dataset), followed by small-vehicles with 31564 instances (24.69%) and large-vehicles with 21356 instances (16.70%). These three classes alone account for approximately 70% of all instances. Conversely, several classes are significantly underrepresented. Baseball-diamond (0.49%), ground-track-field (0.37%), roundabout (0.45%), and soccer-ball-field (0.37%) each constitute less than 0.5% of the dataset. This extreme imbalance poses challenges for model training and evaluation, particularly for rare classes where insufficient examples limit the model's ability to learn discriminative features. Additionally, classes such as vehicles and helicopters are characterized by small object sizes (typically 10-20 pixels at common image resolutions), making them particularly challenging for detection algorithms.

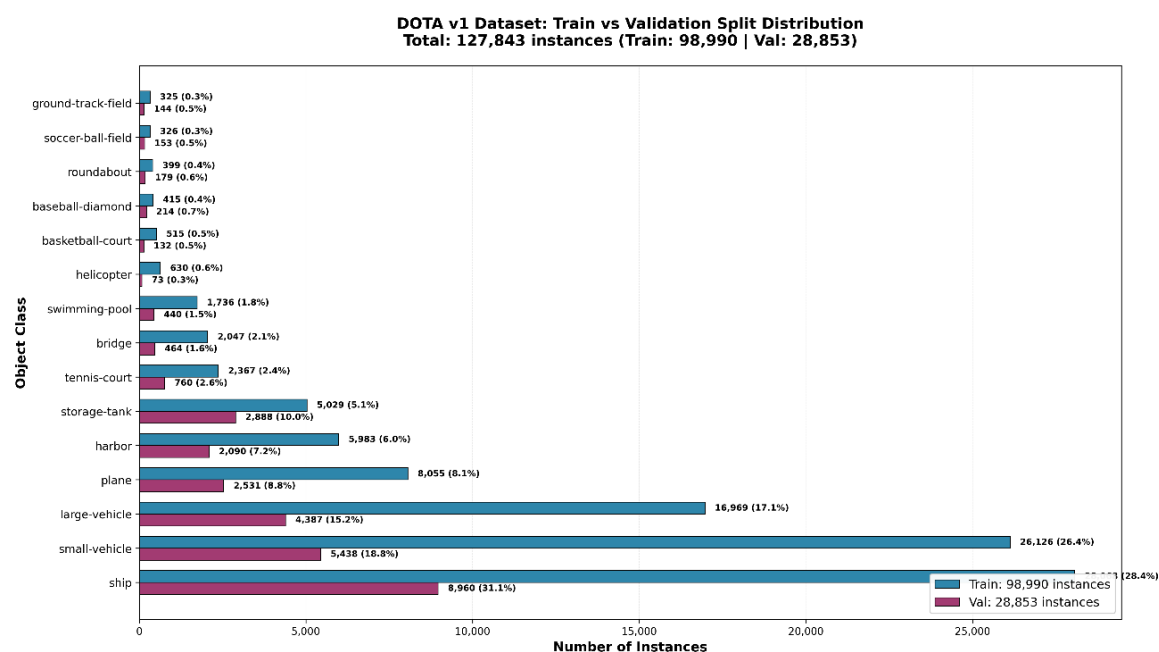


Figure 10: DOTA v1.0 class distribution and statistics for training and validation splits

This class distribution reflects real-world aerial imagery characteristics where certain object types (vehicles, ships) are inherently more common in surveillance scenarios, while others (sports facilities, helicopters) occur more rarely. This imbalance needs careful consideration during evaluation, as overall metrics may be dominated by performance on the majority classes, potentially obscuring poor performance of rare (but potentially important) classes for specific applications.

DOTA provides oriented bounding box annotations in a text-based format, with one annotation file per image. Each line in the annotation file specifies a single object instance with the format: $x_1 y_1 x_2 y_2 x_3 y_3 x_4 y_4$ class name difficulty, where (x_1, y_1) , (x_2, y_2) , (x_3, y_3) , (x_4, y_4) represent the four corner points of the oriented rectangle in clockwise order, class name indicates the object category, and difficulty is a binary flag (0 for normal, 1 for difficult instances). For the YOLO baseline experiments, these OBB annotations were converted into two formats:

- The oriented format (OBB): which preserves rotation information (class id $x_1 y_1 x_2 y_2 x_3 y_3 x_4 y_4$ with normalized coordinates in $[0, 1]$), used for YOLO versions that are OBB enabled.
- The axis-aligned/vertical format (VBB): which discards rotation (class id $cx cy w h$, where cx, cy represent the box center and w, h represent width and height), used for YOLO versions, pretrained on COCO dataset, using VBB.

This dual-format approach (see Figure 11) enables a direct comparison of oriented versus axis-aligned detection performance, addressing the research question regarding the necessity of rotation-aware methods for aerial imagery.

The DOTA annotations exhibit generally high quality, with most objects precisely annotated. However, some systematic challenges exist. For objects with ambiguous boundaries (e.g., harbors, bridges), annotator consistency varies. The "difficulty" flag is applied inconsistently across the dataset, with some clearly challenging instances unmarked and some relatively easy instances flagged. For extremely small objects (swimming pools, helicopters), annotation precision is limited by image resolution. These quality considerations are important for interpreting evaluation results, particularly for classes where annotation ambiguity may affect metric reliability.



Figure 11: DOTA v1.0 example image with overlaid vertical bounding boxes – VBB (left) and oriented bounding boxes – OBB (right)

2.2 iSAID Dataset Exploratory Data Analysis (EDA)

The instance Segmentation in Aerial Images Dataset (iSAID) (Zamir et al., 2019), was selected as the segmentation ground-truth source, mainly due to the fact that it is built using the same source images as DOTA v1.0, providing a direct spatial correspondence between detection and segmentation annotations. This provides a unique advantage among other available aerial datasets, essential for the integrated pipeline evaluation. iSAID also provides pixel-level segmentation masks with class alignment to DOTA's 15 categories, enabling direct comparison between detection and segmentation tasks. The dataset represents one of the largest and highest-quality segmentation datasets for aerial imagery.

iSAID includes annotations (see Figure 12) for images sourced from DOTA v1.0. It provides both semantic segmentation masks, in which all objects of the same class are RGB color-coded in the same way (see Table 5) and instance segmentation masks, where each instance is annotated as a separate entity, through different RGB color-coding. This separation serves different computer vision applications, as defined in Section 1.1. Three official dataset splits are provided:

- Training split contains 1411 semantic/instance segmentation masks covering approximately 99000 annotated instances across 15 object categories.
- Validation split contains 458 images with semantic/instance masks covering approximately 28900 instances, used for segmentation model evaluation.
- Test split contains 937 images where ground-truth segmentation masks are withheld and evaluation is conducted through the official iSAID competition server with limited submission attempts.



Figure 12: iSAID instance segmentation and semantic segmentation masks example

Class ID	Class Name	Color	RGB Coords	Hex Code
0	plane		(0, 127, 255)	#007FFF
1	ship		(0, 0, 63)	#00003F
2	storage-tank		(0, 63, 63)	#003F3F
3	baseball-diamond		(0, 63, 0)	#003F00
4	tennis-court		(0, 63, 127)	#003F7F
5	basketball-court		(0, 63, 191)	#003FBF
6	ground-track-field		(0, 63, 255)	#003FFF
7	harbor		(0, 100, 155)	#00649B
8	bridge		(0, 127, 63)	#007F3F
9	large-vehicle		(0, 127, 127)	#007F7F
10	small-vehicle		(0, 0, 127)	#00007F
11	helicopter		(0, 0, 191)	#0000BF
12	roundabout		(0, 191, 127)	#00BF7F
13	soccer-ball-field		(0, 127, 191)	#007FBF
14	swimming-pool		(0, 0, 255)	#0000FF
—	background		(0, 0, 0)	#000000

Table 5: iSAID semantic segmentation masks' RGB color encoding for 15 classes

In this thesis, focus was placed exclusively on the iSAID semantic segmentation annotations, since the aim is pixel-level object mapping rather than instance differentiation. The dataset consists of approximately 127566 instance annotations (train: ~98711, validation: ~28855) with precise pixel-level boundaries, corresponding to DOTA v1.0 images, that span across the same 15 object categories. Figure 13 presents the class distribution across training and validation splits.

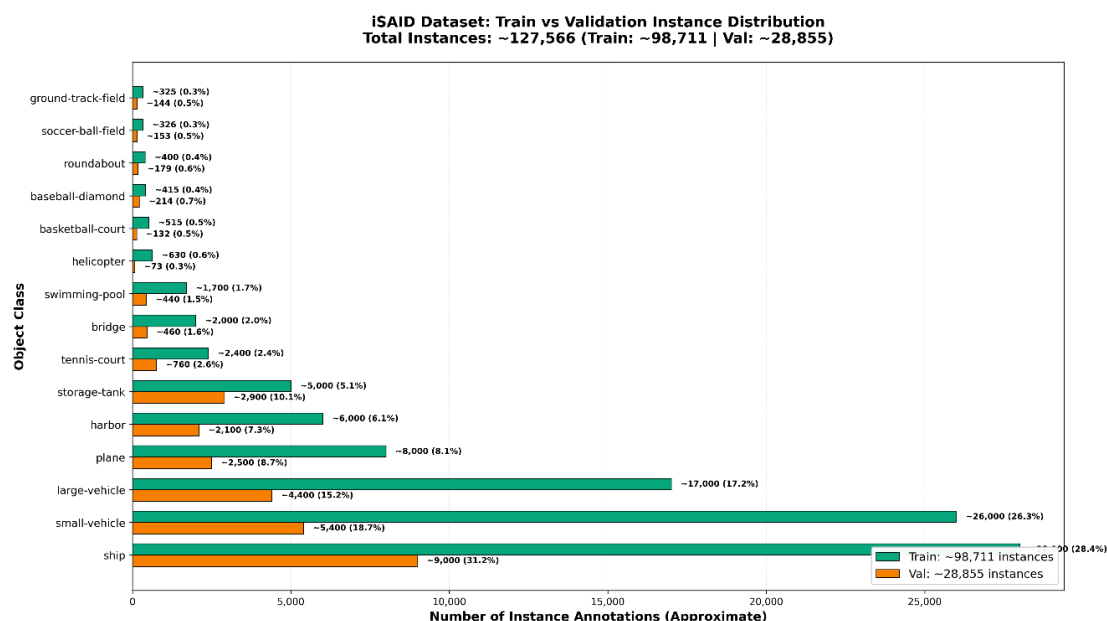


Figure 13: iSAID class distribution and statistics per split

The dataset exhibits extreme class imbalance, with ships constituting the largest class with approximately 37000 instances (29.0% of the dataset), followed by small-vehicles with approximately 31400 instances (24.6%) and large-vehicles with approximately 21400 instances (16.8%). These three dominant classes alone account for approximately 70% of all instance annotations. In addition, several classes are significantly underrepresented. Ground-track-field (0.3%), soccer-ball-field (0.4%), roundabout (0.4%), basketball-court (0.5%), helicopter (0.6%), and baseball-diamond (0.4%) each constitute less than 0.6% of the dataset. This extreme imbalance poses challenges for the training and evaluation of segmentation models, particularly for rare classes where small sample sizes limit the statistical reliability of per-class metrics. The class distribution reflects both real-world prevalence and annotation feasibility. Vehicle classes are ubiquitous in aerial imagery and possess relatively clear, well-defined boundaries amenable to pixel-level annotation. Infrastructure classes (bridges, harbors) and sports facilities occur more rarely and often feature ambiguous boundaries or complex geometries that complicate precise segmentation. Small objects such as swimming-pools and helicopters present additional challenges due to their limited pixel footprint and irregular shapes.

The class alignment between DOTA and iSAID enables direct comparison between detection and segmentation performance. Analysis can be conducted on whether classes that are easy to detect are also easy to segment or pose other challenges. This cross-task analysis could provide insights into the fundamental visual properties that make certain object types more or less amenable to automated analysis, informing future dataset design and model development strategies. However, out of 2806 images in DOTA v1.0, iSAID provides segmentation annotations for only a subset and, in addition, other spatial alignment issues (analyzed in the following section) have been detected in the effort to combine the two datasets.

Chapter 3 Methodology

3.1 AerialFuseCV Dataset Creation and Analysis

After extensive research, no available datasets were found to offer perfectly aligned, paired detection and segmentation data, in a unified format, suitable for an integrated multiclass object Detection→Segmentation pipeline evaluation. Moreover, DOTA and iSAID, despite using the same source images, were created independently with different annotation tools, formats, and coordinate systems. As a result, the two datasets use incompatible file structures and naming conventions, making automatic pairing difficult. DOTA stores annotations (bounding box coordinates) in text files with specific naming patterns, while iSAID uses PNG files (segmentation masks) with different naming conventions. Merging requires careful filename matching and verification. More importantly, not all DOTA images have iSAID annotations and vice versa, so identification of the overlapping subset, suitable for integrated pipeline evaluation, is necessary. Also, even for images that exist in both datasets, the instance-level correspondence must be verified, in order to ensure that detection bounding boxes and segmentation masks refer to the same physical objects.

To address these challenges and enable Detection→Segmentation pipeline evaluation, the AerialFuseCV dataset was created by systematically fusing DOTA v1.0 and iSAID into a unified dataset with paired bounding boxes and segmentation masks. This fusion process required careful data engineering to maintain annotation quality while creating a format compatible with both YOLO detection and SAM segmentation models. While the images between DOTA v1.0 and iSAID have a 1:1 correspondence, initial testing of the AerialFuseCV dataset (during preliminary pipeline experiments) revealed critical annotation discrepancies, resulting in poor performance and reducing the validity of the metrics-based quantitative evaluation. Specifically, not all detection bounding boxes had corresponding segmentation mask pixels, and conversely, not all segmentation mask regions had corresponding detection bounding boxes. These discrepancies are not errors per se, but rather reflect the independent annotation processes used to create DOTA and iSAID. DOTA and iSAID were annotated by different teams, at different times, using separate annotation protocols. DOTA annotators focused on object-level detection suitable for surveillance applications, while iSAID annotators emphasized precise instance boundaries suitable for GIS integration and area measurement. Figure 14 illustrates representative examples of these discrepancy patterns discovered during initial evaluation.



Figure 14: Excluded images due to annotation inconsistencies from AerialFuseCV refinement process.
Segmentation masks with no corresponding detection BBoxes

The complete AerialFuseCV dataset creation process (Figure 15) was organized into five systematic stages, designed to identify compatible images, convert annotations to unified formats, add and verify the segmentation ground-truth masks, refine the dataset through instance-level matching to ensure annotation quality and organize it in appropriate splits, with relevant documentation.

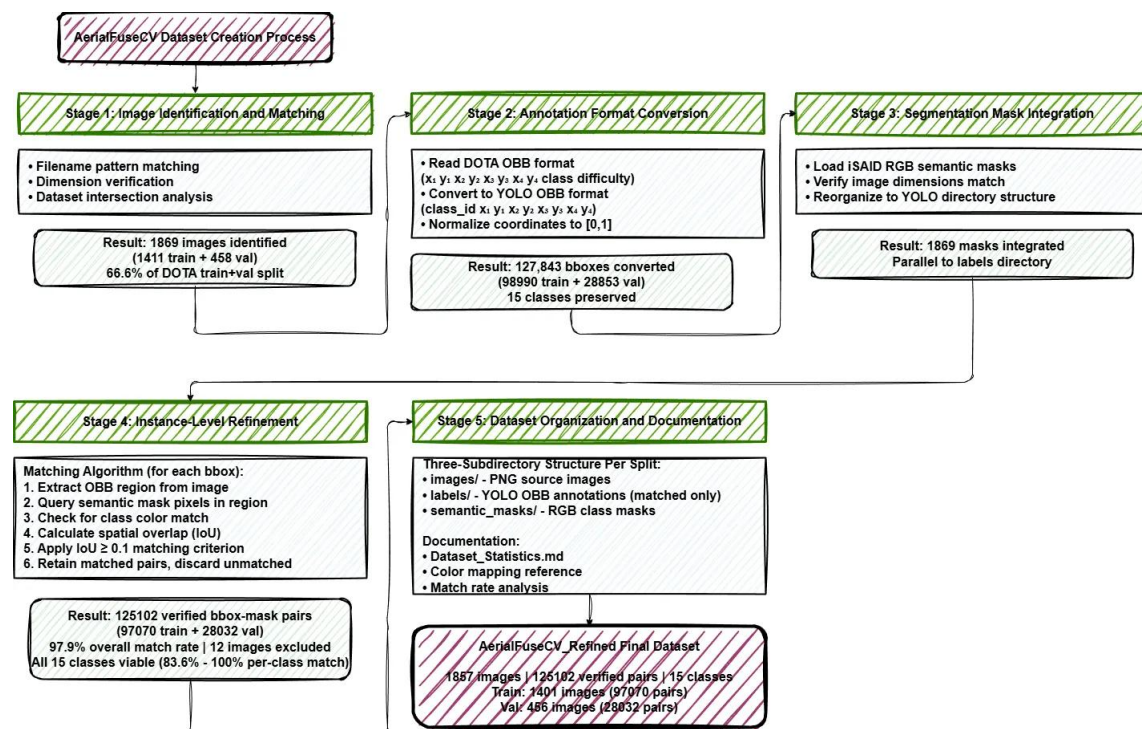


Figure 15: AerialFuseCV creation methodology

Stage 1 - Image Identification and Matching: The intersection between DOTA v1.0 and iSAID image sets was identified through filename pattern matching and dimension verification. Of the 2806 images in DOTA v1.0, a total of 1869 images (1411 in training and 458 in validation splits) were found to have corresponding segmentation masks in iSAID, representing 66.6% of the combined DOTA train and validation splits. The 937 test split images were excluded as iSAID does not provide public segmentation masks for this subset. The original DOTA split structure was preserved, with an initial total of 1411 training images and 458 validation images.

Stage 2 - Annotation Format Conversion: Detection annotations were converted from the original DOTA text format (x_1 y_1 x_2 y_2 x_3 y_3 x_4 y_4 class-name difficulty) to YOLO OBB format with normalized coordinates in the $[0,1]$ range. This conversion preserved the rotation information (critical for aerial objects), while enabling compatibility with YOLO OBB-capable model versions. A total of 127843 bounding box instances (98990 training and 28853 validation) across 15 classes were converted and verified for spatial correspondence to their source images.

Stage 3 - Segmentation Mask Integration: The segmentation masks from iSAID were maintained in their original PNG format but reorganized to match the YOLO directory structure. Each semantic mask file was verified to have dimensions identical to its corresponding image, ensuring pixel-level spatial alignment. The RGB class color encoding specified in Table 5 was preserved without modification, maintaining compatibility with iSAID evaluation protocols.

Stage 4 - Instance-Level Refinement. A systematic refinement algorithm was developed, aiming to match detection boxes to segmentation masks through spatial overlap analysis. The process used, as baseline, the DOTA bounding boxes (and not the iSAID segmentation masks) existence. For each bounding box, the algorithm:

- Extracts the image region defined by the OBB coordinates.
- Queries the semantic mask to identify all pixels within that region.
- Checks if the mask contains pixels matching the bounding box color.
- Calculates spatial overlap using connected components analysis.
- Applies the $\text{IoU} \geq 0.1$ matching criterion between bounding box region and corresponding class mask pixels.
 - Retains only bounding box-mask pairs meeting the matching criterion, discarding unmatched bounding boxes and orphaned segmentation regions.

The IoU threshold of 0.1 was selected as a conservative criterion requiring only minimal spatial overlap, ensuring that boxes and masks refer to the same general object while tolerating the annotation inconsistencies inherent in independent annotation processes. Higher thresholds (e.g., 0.3) would exclude valid pairs where DOTA boxes extend beyond iSAID masks or vice versa. Lower thresholds (e.g., 0.05) would risk accepting

false matches. The threshold was determined through a trial-and-error process and proved sufficient in balancing Precision (avoiding false matches) against Recall (retaining valid pairs with imperfect alignment). The refinement process analyzed all 127843 bounding boxes, across all 1869 originally retained images, yielding the results of Table 6. A count of 12 images (10 in train and 2 in val splits) was discarded due to total mismatching, leaving the AerialFuseCV dataset with a final number of 1857 validated images, across all splits. The overall match rate of 97.9% indicates that approximately 98 out of 100 bounding boxes have corresponding segmentation masks with sufficient spatial overlap.

Split	Original Bboxes	Matched Pairs	Match Rate	Retained Images
Train	98990	97070	98.1%	1401
Val	28853	28032	97.2%	456
Total	127843	125102	97.9%	1857

Table 6: AerialFuseCV refinement process overall results

Per-class analysis (Table 7) revealed excellent match rates across all 15 classes following the refinement procedure. All classes demonstrated match rates ranging from 83.6% to 100%, with most classes (12 of 15) achieving >95% match rates. Analyzing results for the three lowest-performing classes, it is obvious that helicopter achieved 83.6% validation match rate, possibly due to small object size (typically 10-20 pixels), creating ambiguous box-mask boundaries, harbor achieved 89.6% training match rate due to complex irregular shapes (docks, piers) with different annotation standards between DOTA and iSAID and basketball-court and large-vehicle achieved 93.2% validation match rates, likely due to occlusion and density in urban scenes. Despite these challenges, all 15 classes achieved sufficient match rates for reliable evaluation.

Class	Train Split				Validation Split			
	Bboxes	Masks	Matched	Match Rate	Bboxes	Masks	Matched	Match Rate
plane	8055	8171	7689	95.5%	2531	2572	2478	97.9%
ship	28068	29205	27969	99.6%	8960	10166	8934	99.7%
storage-tank	5029	4699	4960	98.6%	2888	2466	2822	97.7%
baseball-diamond	415	464	413	99.5%	214	220	214	100.0%
tennis-court	2367	2481	2355	99.5%	760	788	748	98.4%
basketball-court	515	580	494	95.9%	132	145	123	93.2%
ground-track-field	325	391	313	96.3%	144	215	138	95.8%
harbor	5983	7739	5360	89.6%	2090	2716	1954	93.5%
bridge	2047	2042	2004	97.9%	464	487	446	96.1%
large-vehicle	16969	30386	16709	98.5%	4387	7720	4088	93.2%
small-vehicle	26126	160844	25798	98.7%	5438	50985	5275	97.0%
helicopter	630	632	609	96.7%	73	94	61	83.6%
roundabout	399	428	386	96.7%	179	186	176	98.3%
soccer-ball-field	326	501	312	95.7%	153	185	141	92.2%
swimming-pool	1736	2445	1699	97.9%	440	740	434	98.6%

Table 7: AerialFuseCV refinement process per class results

Stage 5 - Dataset Organization and Documentation. The AerialFuseCV dataset was organized in a three-subdirectory structure per split. This enables seamless integration into the experimental testing workflows, as detection models can access images and labels (bounding boxes) directories, using standard YOLO protocols, while segmentation models access the parallel semantic masks directory to obtain pixel-level ground-truth. The integrated detection-segmentation pipeline uses all three directories simultaneously, matching bounding box predictions with semantic mask ground-truth for comprehensive evaluation.

```

AerialFuseCV
├── train
│   ├── images          (1401 PNG files)
│   ├── labels          (1401 TXT files, matched instances only)
│   └── semantic_masks  (1401 PNG files, RGB color-coded)
├── val
│   ├── images          (456 PNG files)
│   ├── labels          (456 TXT files, matched instances only)
│   └── semantic_masks  (456 PNG files, RGB color-coded)
└── Dataset_Statistics.md (comprehensive statistics report)

```

The AerialFuseCV dataset exhibits extreme class imbalance characteristic of real-world aerial imagery (Figure 16), in the same way its source datasets do (DOTA v1.0 and iSAID). The ship class constitutes the largest class with 36903 matched pairs (29.5% of the dataset), followed by small-vehicle with 31073 pairs (24.8%) and large-vehicle with 20797 pairs (16.6%). These three dominant vehicle and maritime classes alone account for 71.0% of all verified bbox-mask pairs, reflecting both the prevalence of transportation infrastructure in aerial surveillance imagery and the relative ease of annotating objects with clear, well-defined boundaries. Seven moderately represented classes exist, which are plane (8.1%), storage-tank (6.2%), harbor (5.8%), tennis-court (2.5%), swimming-pool (1.7%), and bridge (2.0%), together contributing 26.3% of the dataset. Conversely, five classes are severely underrepresented, each constituting less than 0.6% of the dataset, namely the helicopter (0.5%), basketball-court (0.5%), baseball-diamond (0.5%), roundabout (0.4%), soccer-ball-field (0.4%), and ground-track-field (0.4%). This disparity between the most represented (ship) and the least represented (ground-track-field) classes poses methodological challenges for both training and evaluation. Aggregate metrics such as overall mean IoU may be dominated by the performance on vehicle classes, potentially obscuring poor performance on rare but operationally important classes such as infrastructure or sports facilities. Despite this imbalance, all 15 classes demonstrate sufficient absolute instance counts to enable statistically meaningful per-class evaluation, with the refinement process successfully retaining 97.9% of spatially corresponding bbox-mask pairs across the complete class spectrum. The distribution pattern closely mirrors that of the source DOTA and iSAID datasets, confirming that the instance-level matching process did not introduce systematic class-specific bias.

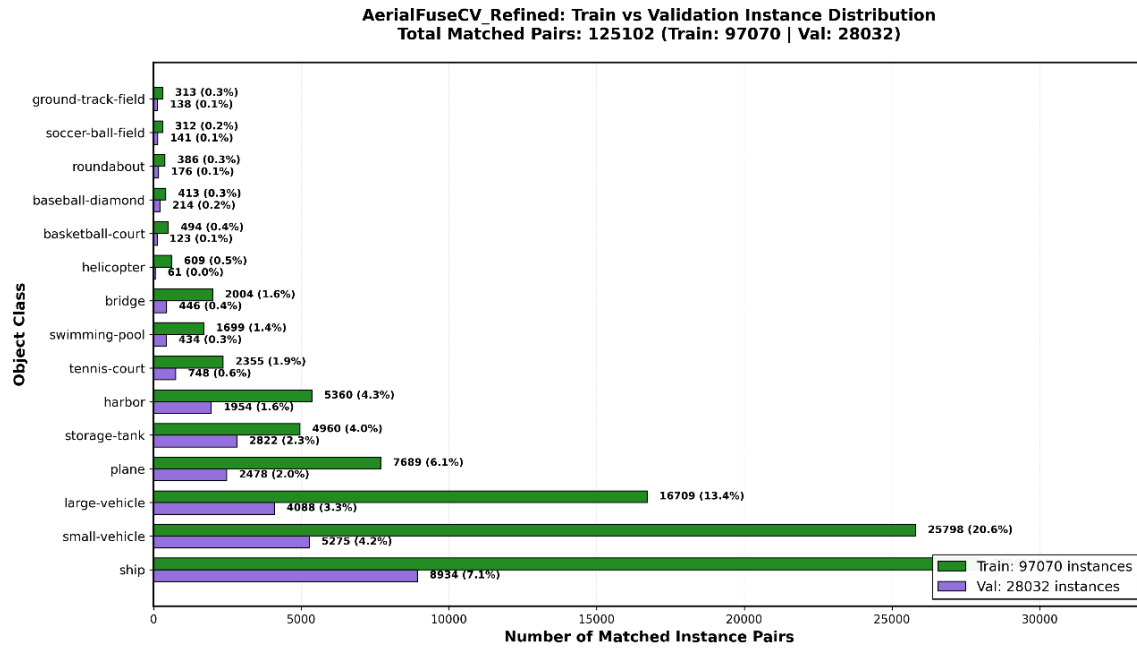


Figure 16: AerialFuseCV class distribution and statistics per split

The final AerialFuseCV dataset represents a validated multi-task resource for aerial object detection and segmentation research. With 1857 high-resolution images, 125102 verified bbox-mask instance pairs across 15 object categories, and 97.9% overall match quality, the dataset enables rigorous evaluation of integrated detection-segmentation pipelines. The systematic quality control process, including empirical validation and complete re-processing, ensures data integrity and reproducibility for all subsequent experimental evaluations. With this dataset in place, a summary of the datasets used in the following experiments is presented in Table 8.

Dataset	Evaluation Task	Annotation Type	Splits Used	Remarks
DOTA v1.0	Object detection stage	Oriented bounding boxes (OBB)	Val	YOLO baseline evaluation (OBB) and detection benchmark
		Vertical bounding boxes (VBB)		YOLO baseline evaluation (VBB) and detection benchmark
iSAID	Semantic segmentation stage performance	Pixel-level semantic masks	Val	SAM baseline evaluation and segmentation benchmark
AerialFuseCV	Detection-segmentation pipeline performance	Paired YOLO OBB labels + iSAID semantic masks	Val	Unified dataset enabling end-to-end pipeline evaluation and error propagation analysis

Table 8: Datasets used and role in evaluation experiments

3.2 YOLO Baseline Evaluation

The YOLO baseline evaluation establishes the detection performance foundation for the integrated Detection→Segmentation pipeline. This section describes the experimental design, model selection rationale, and evaluation protocol used to assess YOLO object detection capabilities on aerial imagery before integration with SAM segmentation. The baseline evaluation was designed to address the following research questions:

- Whether transfer learning from ground imagery (COCO dataset) pretrained YOLO models is feasible, when the imagery domain shifts to aerial vertical views.
- Which of the two possible detection annotation types [rotation-aware oriented bounding boxes (OBB) or axis-aligned (VBB) annotations] are best suited for aerial object detection.
- Which YOLO architecture provides optimal performance for subsequent pipeline integration, balancing Precision and inference time.

To answer these questions, a two-experiment evaluation strategy was implemented (Table 9), comparing YOLO models trained with oriented versus vertical bounding box formats.

Experiment	Models Evaluated	Pretraining Dataset	Annotation Format	Evaluation Dataset
VBB (Vertical BBoxes)	24 models: YOLOv8 (n,s,m,l,x), YOLOv9 (t,s,m,c,e), YOLOv10 (n,s,m,b,l,x), YOLOv11 (n,s,m,l,x), YOLOv12 (n,s,m,l,x)	COCO (118000 images, 860000 instances, ground-level natural images)	Axis-aligned bounding boxes (cx, cy, w, h)	DOTA v1.0 validation split (458 images, 28853 instances, 15 classes) - annotations converted to VBB format
OBB (Oriented BBoxes)	10 models: YOLOv8-OBB (n,s,m,l,x), YOLOv11-OBB (n,s,m,l,x)	DOTA v1.0 train split (1411 images, 98990 instances, aerial imagery)	Oriented bounding boxes ($x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4$)	DOTA v1.0 validation split (458 images, 28853 instances, 15 classes) - native OBB format

Table 9: Full YOLO baseline evaluation experimental setup

Vertical (or Axis aligned) Bounding Box (VBB) experiment: The VBB experiment evaluates whether standard YOLO models pretrained on ground-level imagery can transfer to aerial object detection without rotation-aware representations. This experiment serves as an initial baseline in order to quantify the domain gap between ground level natural images (COCO) and overhead aerial imagery (DOTA), to evaluate whether rotation information is beneficial for aerial object detection and to establish whether vanilla YOLO models can be deployed for aerial applications without dataset-specific training. 24 models were selected from YOLOv8 through YOLOv12, including

all size variants (n, s, m, l, x) and specialized variants (v9t, v9c, v9e). These models were pretrained on the COCO dataset (118000 images, 860000 instances) using standard axis-aligned bounding boxes in the format (cx, cy, w, h) where cx and cy represent the box center and w and h represent width and height. A critical constraint of the VBB experiment is that COCO-pretrained models were not trained on DOTA-specific object categories. Of the 15 classes in DOTA, only 3 have approximate semantic correspondence to COCO classes, limiting the VBB evaluation to these overlapping categories (Table 10). The class correspondence was established based on semantic similarity, and so COCO's 'car' class maps to DOTA's 'small-vehicle' (both representing passenger vehicles), COCO's 'boat' class maps to DOTA's 'ship' (both representing sea vessels), and COCO's 'truck' and 'bus' classes map to DOTA's 'large-vehicle' (both representing commercial and public transport vehicles). These 3 classes include 18785 instances, a percentage of 65.1% of the DOTA validation split, since they are the most represented ones in the dataset. In that way, they provide sufficient data for transfer learning assessment, while acknowledging that 12 DOTA classes (plane, storage-tank, baseball-diamond, tennis-court, basketball-court, ground-track-field, harbor, bridge, helicopter, roundabout, soccer-ball-field, swimming-pool) have no COCO equivalents and therefore cannot be evaluated in the VBB experiment. In that way, the VBB experiment evaluates cross-domain transfer for only the overlapping categories and should not be interpreted as a full 15-class detector benchmark.

COCO Class (Pretraining)	DOTA Class (Evaluation)	DOTA Validation Instances	Correspondence Rationale
car	small-vehicle	5438	Both represent passenger vehicles and compact cars viewed from above
boat	ship	8960	Marine vessels including both small watercraft (COCO) and larger ships (DOTA)
truck, bus	large-vehicle	4387	Commercial and public transport vehicles including trucks, buses, and large vans

Table 10: COCO to DOTA Class Correspondence, for YOLO VBB experimental evaluation

Oriented Bounding Box (OBB) experiment: The OBB experiment evaluates YOLO models specifically designed for rotated object detection in aerial imagery. Ten pretrained models were assessed, YOLOv8-OBB (n, s, m, l, x variants) and YOLOv11-OBB (n, s, m, l, x variants). These models were pretrained by Ultralytics on the DOTA v1.0 training split using the native oriented annotation format where each object is represented by four corner points ($x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4$) in normalized coordinates. This representation preserves object rotation information, which is critical for aerial objects that appear at arbitrary orientations due to camera viewpoint and object alignment. The selection of YOLOv8 and YOLOv11 OBB variants was driven by the fact that these models represent the current state-of-the-art in oriented object detection for aerial imagery, with proven performance on DOTA benchmarks. Ultralytics provides publicly available pretrained model weights trained on DOTA using consistent hyperparameters and training protocols, ensuring fair comparison across model sizes and

generations. Also, using all five size variants (nano, small, medium, large, x-large), spanning from 3 million to 68 million parameters, enables analysis of the performance-efficiency trade-off across different computational budgets. Since basic training of these YOLO versions was performed on DOTA dataset, all DOTA classes can be evaluated, during the experiment phase.

All YOLO baseline experiments were conducted on the DOTA v1.0 validation split. The validation split was selected due to the fact that it maintains complete independence from the training data used to pretrain the OBB models, ensuring that performance metrics reflect true generalization rather than memorization. Moreover, the validation split includes all 15 object categories with sufficient representation for per-class analysis. The DOTA test split was not used because ground-truth annotations are withheld for official competition evaluation, making local assessment impossible. While this means the reported metrics differ from published DOTA leaderboard results (which use the test split), the validation split provides a reliable proxy for detection performance that can be consistently measured and reproduced. All annotations had to be converted according to model needs. For the VBB experiment, annotations were converted to YOLO standard format (class_id cx cy w h) by computing the axis-aligned bounding box that encloses each oriented box. The conversion process was validated by visual inspection of 50 randomly sampled images to ensure spatial correspondence between converted annotations and source images. For the OBB experiment, DOTA annotations were converted from the original text format (x_1 y_1 x_2 y_2 x_3 y_3 x_4 y_4 class_name difficulty) to YOLO OBB format (class_id x_1 y_1 x_2 y_2 x_3 y_3 x_4 y_4) with normalized coordinates in the [0,1] range. This conversion preserved rotation information while enabling compatibility with YOLO OBB models.

Detection performance was quantified using five complementary metrics, which were computed using a matching strategy where each predicted box is matched to the ground-truth box with highest IoU, subject to a minimum IoU threshold of 0.1 for a valid match. Metrics used were Bounding Box Precision, Bounding Box Recall, Bounding Box IoU and Bounding Box DICE coefficient for the evaluated classes (three classes for VBB and all fifteen for OBB). Moreover, for a unified and overall assessment the averages of the above metrics (meanPrecision, meanRecall, meanF1, meanIoU and meanDICE) were calculated.

Additionally, inference time was measured for each model, in order to assess its computational efficiency and deployment viability. Timing measurements were conducted on identical hardware with the same size of 16, recording the average time per image across the entire validation split. The inference time was separated from any other algorithmic actions (e.g. writing results and visualizations, calculating metrics etc.) so a clear understanding of the per image inference speed may be formed. This timing analysis enables identification of the performance-efficiency trade-off across model variants, revealing whether larger models justify their increased computational cost

through substantial accuracy gains or if smaller models provide sufficient performance at lower latency.

The final pipeline model will be selected based on detection performance on the DOTA v1.0 validation set, measured by mean F1 score across all classes and inference speed to enable practical deployment and integration compatibility with SAM segmentation. While the OBB experiment includes pretrained weights optimized for aerial imagery, the VBB track provides essential context for understanding the value of domain-specific training and rotation-aware representations. This evaluation enables informed architectural decisions for the integrated pipeline rather than defaulting to the largest or most recent model.

3.3 SAM Baseline Evaluation

The SAM baseline evaluation establishes the segmentation performance foundation for the integrated Detection→Segmentation pipeline. This section describes the experimental design, model variant selection, and evaluation protocol used to assess SAM's zero-shot segmentation capabilities on aerial imagery, when prompted with ground-truth bounding boxes, isolating segmentation quality from detection errors. The baseline evaluation was designed to address the following research questions:

- Whether SAM's zero-shot segmentation, pretrained on natural ground-level images (SA-1B dataset), can transfer effectively to overhead aerial imagery without domain-specific fine-tuning.
- Which of the two prompt strategies (box prompts versus point prompts) provides superior segmentation quality for aerial objects with diverse shapes, sizes, and aspect ratios.
- Which SAM model variant (ViT-H, ViT-L, ViT-B) provides the optimal balance between segmentation quality and inference speed for integration into the real-time Detection→Segmentation pipeline.

To answer these questions, a comprehensive evaluation strategy was implemented, assessing three SAM model variants across two prompt strategies, for a total of six experimental configurations, as shown in Table 11.

The Segment Anything Model (SAM) is pretrained on SA-1B, a dataset comprising 1 billion segmentation masks across 11 million diverse natural images. Three SAM model variants were evaluated, differing in image encoder capacity while maintaining identical prompt encoder and mask decoder architectures. SAM ViT-H (Huge) uses a Vision Transformer with 630 million parameters, representing the highest capacity variant, with maximum segmentation quality. SAM ViT-L (Large) uses 308 million parameters, providing a balanced trade-off between quality and computational efficiency. SAM ViT-

B (Base) uses 91 million parameters, representing the most lightweight variant optimized for inference speed. The selection of all three variants enables systematic analysis of the model size versus performance trade-off, identifying the minimum capacity sufficient for high-quality aerial segmentation.

Two prompting strategies were evaluated (for each model variant) to determine the optimal method for guiding SAM segmentation. Box prompts provide SAM with axis-aligned (vertical) bounding boxes ($x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$) in pixel coordinates, defining a rectangular region containing the target object. For this evaluation, ground-truth oriented bounding boxes from AerialFuseCV were converted to axis-aligned format by computing the enclosing rectangle, ensuring spatial coverage while maintaining compatibility with SAM's native box prompt format. This strategy provides strong spatial priors, effectively constraining the segmentation region and reducing ambiguity for objects with complex boundaries. Point prompts provide SAM with a single point (cx , cy) in pixel coordinates, computed as the center of the ground-truth bounding box and labeled as positive (foreground). This strategy provides minimal spatial information, requiring SAM to infer object boundaries from the single interior point, through its learned understanding of object coherence and typical shapes. The point prompt evaluation assesses SAM's ability to segment objects with limited guidance, simulating scenarios where precise bounding boxes are unavailable.

Model Variant	Architecture	Pretraining Dataset	Prompt Strategies	Evaluation Dataset
SAM ViT-H (Huge)	Vision Transformer Huge (ViT-H/16), 630M parameters, image encoder + prompt encoder + mask decoder	SA-1B	Box prompts (ground-truth OBB converted to axis-aligned)	AerialFuseCV (validation split)
			Point prompts (bbox centers)	
SAM ViT-L (Large)	Vision Transformer Large (ViT-L/16), 308M parameters, same architecture as ViT-H but smaller capacity	SA-1B	Box prompts (ground-truth OBB converted to axis-aligned)	AerialFuseCV (validation split)
			Point prompts (bbox centers)	
SAM ViT-B (Base)	Vision Transformer Base (ViT-B/16), 91M parameters, most lightweight variant	SA-1B	Box prompts (ground-truth OBB converted to axis-aligned)	AerialFuseCV (validation split)
			Point prompts (bbox centers)	

Table 11: SAM baseline evaluation experimental setup

All SAM baseline experiments were conducted on the validation split of the AerialFuseCV dataset, consisting of 456 high-resolution aerial images with 28032 verified bbox-mask pairs across 15 object categories. At this separate evaluation stage, the use of ground-truth bounding boxes as prompts (rather than YOLO detections) isolates SAM's segmentation capabilities from upstream detection errors, enabling pure

assessment of segmentation quality, independent of detection performance. This experimental design allows quantification of SAM's upper bound performance when provided with perfect object localization, establishing a baseline against which the integrated pipeline (using YOLO-predicted prompts) can be compared. The validation split from AerialFuseCV ensures consistency with the YOLO baseline evaluation, enabling direct comparison of detection versus segmentation performance on identical images and object instances. Ground-truth segmentation masks were obtained from iSAID semantic masks integrated into AerialFuseCV, providing human-annotated pixel-level masks for all 15 object categories. Each predicted mask was compared against its corresponding ground-truth mask using Mask IoU to measure the overlap between predicted and ground-truth masks and Mask DICE coefficient, providing a more balanced metric that is less sensitive to mask size variations than IoU. Per-class IoU and DICE enable identification of category-specific segmentation challenges, while mean IoU and mean DICE across all classes quantify overall performance. Inference time per image was measured to assess computational efficiency, excluding data loading and other computation actions (e.g. metric calculation, results writing etc.) to isolate model processing time.

The SAM variant and prompt strategy for the integrated pipeline would be selected based on mean IoU across all classes as the primary segmentation quality metric, inference time to ensure the pipeline meets deployment constraints and robustness across diverse object categories, particularly for challenging classes with irregular shapes or small sizes. While larger models (ViT-H, ViT-L) are expected to provide higher segmentation quality, the integrated pipeline's total inference time must remain practical for real-world applications. Given that YOLO is considered as one of the fastest performing detection models, the SAM component of the integrated pipeline should not introduce excessive computational overhead. The zero-shot evaluation protocol assesses SAM's ability to generalize to aerial imagery, despite pretraining exclusively on natural ground-level images, quantifying the domain gap and identifying any systematic failures on aerial object categories.

3.4 Integrated Detection→Segmentation Pipeline Evaluation

This section describes the experimental design for evaluating the integrated Detection→Segmentation pipeline, which combines YOLO object detection with SAM semantic segmentation to achieve end-to-end instance-level aerial object delineation. While baseline evaluations (sections 3.2 and 3.3) assess detection and segmentation components independently using ground-truth annotations, the integrated pipeline evaluation quantifies real-world performance where detection outputs directly feed segmentation inputs. This enables measurement of error propagation, component interaction, and system-level capabilities that cannot be assessed through component-level assessments alone.

The integrated pipeline implements a sequential three-stage architecture (Figure 17). For each input aerial image, YOLO first predicts oriented bounding boxes with class labels and confidence scores. Each detection is then converted to an axis-aligned bounding box (as SAM does not natively support rotated box prompts) or the box center point is calculated, depending on the chosen strategy to prompt SAM. A flexible design was implemented, in which the prompting strategy can be differentiated for each class, through a predetermined config file. This strategy allows for mixed prompting (boxes for some classes and points for others) within a single pipeline run, enabling extensive testing and class-specific optimization, where prompt type can be tailored to object characteristics. Finally, following proper prompting, SAM generates a corresponding segmentation mask, per detected object in the image, in a sequential manner, as it doesn't integrate a batch processing capability. The instance segmentation masks are accumulated across all detections to produce the final instance segmentation result that corresponds to the input image.

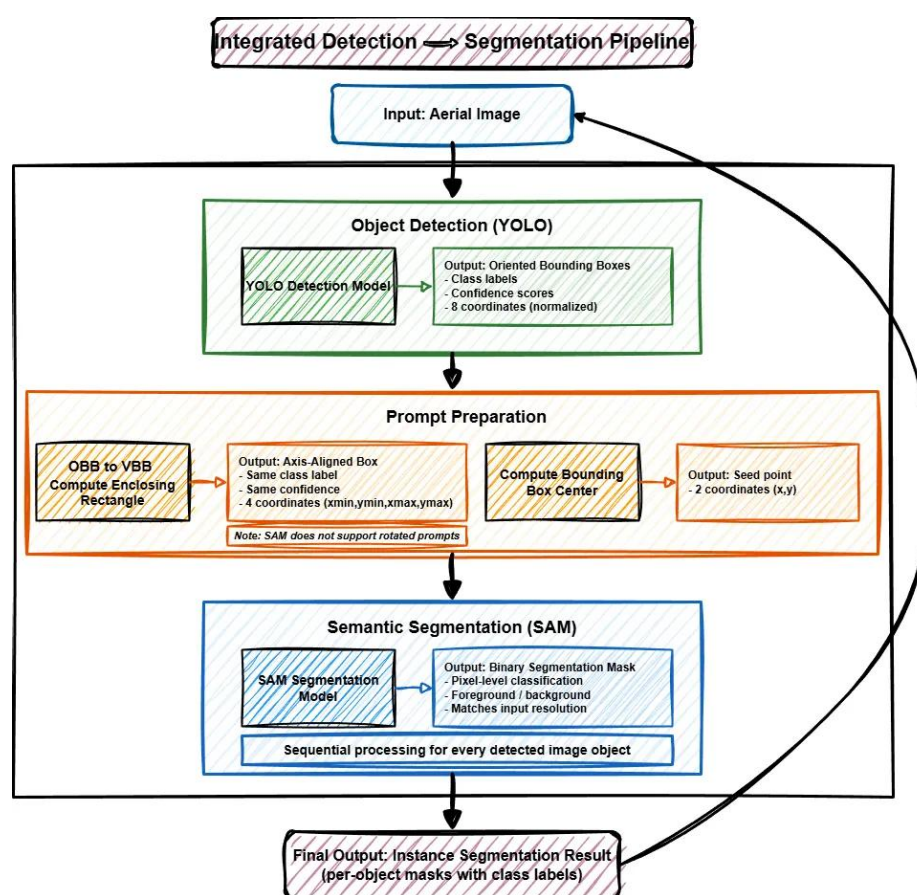


Figure 17: Integrated Detection-Segmentation Pipeline Architecture

This integrated pipeline evaluation aims to answer the following research questions, as a first complete end-to-end approach, towards a real-time operational system:

- Does the integrated pipeline achieve end-to-end segmentation quality comparable to SAM with ground-truth prompts?

- How do detection errors (false positives, false negatives, localization inaccuracies) propagate through the pipeline and affect final segmentation results?
- Which object categories benefit most from the integrated approach, and which exhibit failures requiring targeted improvements?
- What is the computational efficiency of the integrated pipeline, and does it meet practical deployment constraints for real-time aerial imagery processing?

These questions investigate the detection-segmentation interaction, quantify error propagation through the pipeline, identify class-specific failure modes, and assess practical deployment viability. Unlike baseline evaluations that use ground-truth annotations to isolate component capabilities, pipeline evaluation uses predicted detections as segmentation prompts, enabling measurement of real-world performance where upstream errors affect downstream results.

All integrated pipeline experiments are conducted on the AerialFuseCV validation split (456 images, 28032 ground-truth bbox-mask pairs, 15 classes) to ensure consistency with baseline experimental evaluations. This unified dataset enables direct comparison both with both the YOLO baseline experiment on DOTA validation set and the SAM baseline experiment with ground-truth prompts on AerialFuseCV validation set. The integrated pipeline evaluates end-to-end performance with predicted prompts on the same AerialFuseCV validation images, which eliminates inconsistencies coming from different test sets, ensuring all metrics reflect performance on the same object instances.

Evaluation metrics combine detection and segmentation assessment. Detection metrics (Precision, Recall, F1) quantify object localization and classification at the instance level using true positive, false positive, and false negative counts derived from the Hungarian matching algorithm (Kuhn, 1955) between predictions and ground-truth instances. The Hungarian algorithm is used as a flexible and fast approach to solve the assignment problem, by finding the optimal one-to-one correspondence between predicted and ground-truth instances that maximizes total IoU across all pairs. In that way it prevents possible double-counting scenarios where multiple predictions overlap the same ground-truth object. Moreover, segmentation metrics (mask IoU, mask DICE) quantify boundary delineation quality, using the same definitions as the SAM baseline experiment (section 3.3). These segmentation metrics are computed only on matched prediction-ground-truth pairs (with $\text{IoU} \geq 0.1$ and again via Hungarian matching) to isolate SAM performance from detection errors, enabling fair comparison against the SAM baseline where ground-truth prompts were used. This matching threshold of 0.1 was selected in a more lenient manner, to ensure that partial overlaps between detection box and ground-truth instance are counted in the segmentation quality assessment.

Based on these selected metrics, the research questions may be elaborated further. The first one investigates whether predicted detection prompts can approach the quality of ground-truth prompts. The SAM baseline experiment of section 3.3 represents an upper bound on achievable segmentation performance when prompts perfectly identify object locations. The performance difference between SAM's operation in the integrated pipeline (receiving imperfect YOLO detections) and in the dedicated SAM baseline experiment (with perfect ground-truth detection prompts) may be quantified as:

$$\Delta_{IoU} = IoU_{SAM \text{ baseline}} - IoU_{Integrated \text{ pipeline}}$$

This metric quantifies the impact of detection errors on final mask quality. The integrated system's performance is affected by three main error sources, which are:

- The detection Recall errors, where undetected ground-truth objects (false negatives) receive no segmentation prompts and contribute zero segmentation quality, reducing overall coverage.
- The detection Precision errors, where false positive detections generate erroneous segmentation masks that inflate computational cost without corresponding to actual objects.
- The localization quality errors, where imperfect bounding box alignment provides suboptimal spatial guidance to SAM, potentially degrading mask boundaries even when objects are correctly detected.

The difference Δ_{IoU} reveals whether any performance limitations stem primarily from detection errors, or from SAM's zero-shot capabilities on aerial imagery. When Δ_{IoU} is large, it indicates that the pipeline performs much worse than SAM baseline due to predicted prompts being much worse than GT prompts, identifying that the detection stage is the performance bottleneck. When Δ_{IoU} is small, it indicates that the pipeline performs close to SAM baseline, giving accurate detections and consequently segmentation prompts and any performance downgrade comes either from SAM's zero-shot segmentation limitations, or from object related limitations (size, unclear boundaries, etc.).

Per-class performance decomposition identifies category-specific patterns using both detection metrics (Recall, Precision, F1 score) and segmentation quality on detected objects (mean mask IoU and DICE). This dual assessment reveals whether class-specific failures originate from detection limitations (low detection metric scores), where the detector fails to localize objects, or from segmentation challenges (low mask IoU/DICE despite correct detection), where SAM struggles with particular object geometries or visual characteristics. For example, objects with high detection Recall but moderate segmentation IoU indicate challenging boundary delineation, while classes with low

Recall but high segmentation IoU suggest that the detector (and not the segmenter) is the limiting factor.

Finally, computational efficiency assessment measures each component's inference times separately, under the same hardware configuration and environment as the baseline experiments. Inference time is recorded separately, as the pipeline evaluation algorithm performs additional activities (e.g., metrics calculation and aggregation, visualization of examples per class, writing log files etc.) which add computational overhead, but they are used only during evaluation and not in a real-world pipeline application. In that way, the recorded times express the pure computational cost of the integrated system's detection-segmentation stages. Moreover, the recorded timing performance for each stage (detection or segmentation) provides the ability to identify computational bottlenecks and quantify each stage's contribution to total pipeline throughput. This analysis establishes whether total per-image processing time is better suited to real-time aerial surveillance applications, or for batch processing scenarios where multi-second latency is acceptable. The timing breakdown also guides any future optimization priorities by revealing which component dominates computational cost.

3.5 Evaluation Framework – Performance Metrics

This section defines the comprehensive evaluation framework used to assess detection and segmentation performance across all baseline and integrated pipeline experiments. Two complementary metric categories are used, which aim to evaluate the detection stage (YOLO model) for object localization and classification quality, as well as to evaluate the segmentation stage (SAM model) for semantic mask generation quality. Each metric is selected to reveal specific aspects of model performance while acknowledging the limitations inherent to single-metric evaluation. Implementation of multiple metrics enables a holistic assessment of model capabilities and failure scenarios.

3.5.1 Detection Stage Assessment Metrics

Metrics for the detection stage assessment quantify the quality of bounding box predictions produced by YOLO models. These metrics are organized into three categories (Table 12), which are classification metrics that measure correctness of object detection and class assignment, localization metrics that measure spatial accuracy of bounding boxes, and aggregation metrics that combine the multiple class-level metrics into a unified benchmark, for final model ranking.

Metric	Type	Definition	Range	Interpretation
Precision	Classification	$Precision = \frac{TP}{TP+FP}$ Proportion of positive predictions that are correct	[0, 1]	High Precision = few false alarms. 1= all detections correct, 0= all false positives.

Metric	Type	Definition	Range	Interpretation
Recall	Classification	$Recall = \frac{TP}{TP+FN}$ Proportion of actual positives that are detected	[0, 1]	High Recall=few missed objects. 1.0=all objects detected, 0=all objects missed.
F1 Score	Classification	$F1 = \frac{2 \times (Precision \times Recall)}{(Precision + Recall)}$ Harmonic mean of Precision and Recall	[0, 1]	Balanced measure. Penalizes extreme imbalance between P and R.
BBox IoU	Localization	$IoU = \frac{Pred \cap GT}{Pred \cup GT}$ Overlap ratio of predicted and ground-truth boxes	[0, 1]	Spatial accuracy >0.5=detected, >0.7=good, >0.9=excellent. Penalizes oversized or misaligned boxes
BBox DICE	Localization	$DICE = 2 \frac{Pred \cap GT}{(Pred + GT)}$ Similarity coefficient emphasizing overlap	[0, 1]	Less sensitive to box size differences than IoU. Better for small objects
Mean Precision	Aggregate	$meanP = \frac{1}{c} \sum_{c=1}^{15} Precision_c$	[0, 1]	Combine per class Precision, Recall, and F1, IoU and DICE in a single evaluation metric, for total performance.
Mean Recall	Aggregate	$meanR = \frac{1}{c} \sum_{c=1}^{15} Recall_c$	[0, 1]	
Mean F1	Aggregate	$meanF1 = \frac{1}{c} \sum_{c=1}^{15} F1_c$	[0, 1]	
Mean IoU	Aggregate	$meanP = \frac{1}{c} \sum_{c=1}^{15} IoU_c$	[0, 1]	
Mean DICE	Aggregate	$meanP = \frac{1}{c} \sum_{c=1}^{15} DICE_c$	[0, 1]	

Table 12: Detection Assessment Metrics - Definitions and Interpretations

Classification metrics include Precision, Recall, and F1 Score and their computation is based on the categorization of the model's predictions as:

- **True Positives (TP)**, which represent correct detection predictions matched to ground-truth (object exists and was detected).
- **False Positives (FP)**, which represent detection predictions with no valid ground-truth match (object does not exist but it was detected).
- **False Negatives (FN)**, which represent ground-truth objects with no valid detection prediction match (object exists and it was not detected).

The mathematical definition of the classification metrics, along with their physical meaning, is as follows:

- **Precision** measures the proportion of predicted detections that correctly correspond to real objects, computed as:

$$Precision = \frac{TP}{TP + FP} \in [0,1]$$

High Precision indicates few false alarms, with perfect Precision ($p = 1$) meaning all predictions are correct and zero Precision ($p = 0$) meaning all predictions are spurious. Precision is critical for applications where false detections impose costs, such as automated surveillance systems where excessive false alarms reduce operator trust. However, Precision alone is insufficient and sometimes misleading because a model can achieve 100% Precision by making only a single highly confident prediction, while failing to detect all other objects present in an image. In general, Precision tends to favor conservative models that predict only when extremely confident, potentially missing many true objects.

- **Recall** measures the proportion of real objects that are successfully detected, computed as:

$$Recall = \frac{TP}{TP + FN} \in [0,1]$$

High Recall indicates comprehensive object coverage, with perfect Recall ($R=1$) meaning all objects are detected and zero Recall ($R=0$) meaning all objects are missed. Recall is critical for applications requiring complete situational awareness, such as aerial reconnaissance where missing critical objects (e.g. enemy vehicles, weapons' positions, infrastructure compromisation etc.) could have severe consequences. Again, Recall alone is not a sufficient detection performance measure, as a model achieving 100% Recall by predicting objects everywhere in an image, would produce massive numbers of false positives. In general, Recall favors aggressive models that predict liberally to avoid missing objects, potentially flooding results with false detections.

- **F1 score** is the harmonic mean of Precision and Recall, that addresses the trade-off between them. It is computed as:

$$F1 = \frac{2 \times (Precision \times Recall)}{(Precision + Recall)} \in [0,1]$$

F1 score penalizes extreme imbalance between Precision and Recall (borderline model behaviors), as high F1 scores require both metrics to be high simultaneously. For example, a model with 90% Precision and 10% Recall achieves only 18% F1, while balanced 50% Precision and 50% Recall yields 50% F1. This property makes F1 particularly valuable for imbalanced datasets where a naive model could achieve high Precision by predicting only the majority class or high Recall by predicting everything. F1 reflects the overall detection quality in a single metric, making it suitable for model selection and ranking. However, F1 weights Precision and Recall equally, which may not align with application priorities. Depending on the intended use of the pipeline Recall may be prioritized in scenarios where all critical objects must be detected whatever the cost (e.g. detect all threats even with false alarms), while Precision may be prioritized in user-facing applications (e.g. avoid annoying users with false positives). In such cases,

the $F_\beta = (1 + \beta^2) \frac{Precision \times Recall}{\beta^2 Precision + Recall} \in [0,1]$ metric may be used, where if $\beta > 1$ then Recall is weighted more, but if $\beta < 1$ then Precision is weighted more (Van Rijsbergen, 1979).

Localization metrics include Bounding Box IoU and Bounding Box DICE coefficient, their mathematical definitions and physical meanings of which, are as follows:

- **Bounding Box IoU (Intersection over Union)** measures spatial localization accuracy by computing the overlap ratio between predicted and ground-truth bounding boxes. IoU is calculated as:

$$IoU = \frac{Intersection\ Area}{Union\ Area} = \frac{Pred \cap GT}{Pred \cup GT} \in [0,1]$$

Detected objects' bounding boxes with IoU=0 mean that there is no overlap with ground-truth data, while IoU=1 means perfect alignment. The IoU metric penalizes both under-sized boxes (missing object regions) and over-sized boxes (including excess background), rewarding tight, accurate localization. Conventions for IoU quality assessment include $IoU \geq 0.5$ to be commonly used as the minimum threshold for object detection correctness (Everingham et al., 2010a), $IoU \geq 0.7$ to be considered as good localization suitable for most applications (Hosang et al., 2016) and $IoU \geq 0.9$ to serve as excellent localization, approaching total alignment and generally achievable only in high-Precision segmentation scenarios (Lin et al., 2015b). IoU is sensitive to both box position and box size, making it a strict measure of spatial accuracy. For small objects, IoU is a particularly demanding (and possibly unfair) measure, since even minor pixel-level misalignments translate into large relative area differences. For example, an average passenger car (of 4.5 x 2 m dimensions) depicted in a very high resolution (VHR) satellite image of 0.5m spatial resolution, will correspond to a ground-truth bounding box of around 9 x 4 pixels. If the predicted bounding box is offset or expanded by just one pixel on each side (yielding an 11 x 6 pixel box), then the resulting IoU drops to approximately 55%, despite the prediction differing from the ground-truth by only a single pixel.

- **Bounding Box DICE** coefficient provides an alternative localization measure, computed as:

$$DICE = 2 \times \frac{IoU}{1 + IoU} = 2 \times \frac{Pred \cap GT}{(|Pred| + |GT|)} \in [0,1]$$

DICE is always higher than IoU for the same detection (e.g., IoU=0.5 corresponds to DICE=0.667). This relationship makes DICE more forgiving of small localization errors, particularly beneficial for small objects, where IoU can be overly punitive. DICE emphasizes overlap relative to the average box size rather than total covered area,

making it less sensitive to size discrepancies than IoU. However, this same property means DICE can be misleadingly high for poor localizations on small objects. Both IoU and DICE are reported to provide complementary perspectives on localization quality, with IoU serving as the stricter standard and DICE providing a more optimistic view suitable for small object assessment.

Finally, as aggregation metrics the average scores of F1 (**meanF1**), Precision (**meanPrecision**) and (**meanRecall**), are used across all classes, in order to obtain an overall detection quality measure. In the same manner, averages of IoU (**meanIoU**) and DICE (**meanDICE**), are used across all classes as measures of total localization accuracy.

All detection metrics rely on a matching strategy where an IoU threshold of 0.1 is chosen, for matching detection and ground-truth bounding boxes across all experimental phases. This low threshold can be considered as too lenient, but it is motivated by empirical observations during dataset preparation. It has been found that many DOTA ground-truth bounding boxes frequently exhibit systematic over-annotation (too wide bounding boxes around objects), encompassing substantial background regions beyond actual object boundaries. On the other hand, detectors tend to produce tighter bounding boxes that more accurately delineate object extents than the ground-truth annotations themselves. Under standard matching thresholds (e.g., $\text{IoU} \geq 0.5$ or even 0.3), these tighter, more accurate predictions can be incorrectly classified as false positives due to imperfect ground-truth quality rather than actual detection failures. By adopting a more lenient threshold of $\text{IoU} \geq 0.1$, it is possible to offset this annotation variance while still filtering spurious detections. This threshold is applied uniformly across all baseline evaluations, ensuring methodological consistency.

3.5.2 Segmentation Stage Assessment Metrics

Segmentation metrics (Table 13) quantify mask quality at the pixel level. Mask IoU and Mask DICE apply the same mathematical definitions as their bounding box counterparts (section 3.5.1) but operate on binary foreground-background masks rather than rectangular regions. This shift from box-level to pixel-level evaluation makes segmentation assessment inherently more demanding, as even minor boundary errors significantly impact scores.

Metric	Type	Mathematical Definition	Range	Interpretation
Mask IoU	Pixel-level	$\text{IoU} = \frac{\text{Pred} \cap \text{GT}}{\text{Pred} \cup \text{GT}}$ Overlap ratio of predicted and ground-truth boxes	[0, 1]	≥ 0.5 acceptable, ≥ 0.7 good, ≥ 0.85 excellent
Mask DICE	Pixel-level	$\text{DICE} = 2 \frac{\text{Pred} \cap \text{GT}}{(\text{Pred} + \text{GT})}$ Similarity coefficient emphasizing overlap	[0, 1]	More forgiving than IoU.

Table 13: Segmentation Assessment Metrics

Detection metrics evaluate approximate localization using four coordinates, while segmentation metrics evaluate thousands of pixels requiring near-perfect boundary delineation. For example, a 5-pixel boundary error on a 1000-pixel object (0.5% area) minimally affects IoU, while the same absolute error on a 50-pixel object (10% area) causes substantial degradation. This sensitivity makes Mask IoU particularly demanding for thin structures, where correctly identifying 95% of pixels but missing 5% along narrow boundaries yields only 90% IoU despite high overall accuracy.

Both Mask IoU and Mask DICE are reported for complementary perspectives, with the same quality thresholds as detection metrics (IoU ≥ 0.5 acceptable, ≥ 0.7 good, ≥ 0.85 excellent). DICE consistently produces higher scores than IoU due to its mathematical relationship, providing better score separation in low-performance regimes common for small objects. IoU serves as the primary metric for consistency with computer vision benchmarks, while DICE provides secondary assessment for challenging cases.

3.5.3 Integrated Detection→Segmentation Pipeline Assessment Metrics

The integrated pipeline combines detection and segmentation into a single end-to-end system, requiring an adapted evaluation approach that accounts for the interaction between the two stages. While the baseline evaluations assess each component independently against ground-truth annotations, the pipeline evaluation must handle additional complexities since the detection errors propagate into the segmentation stage. As a result, a single prediction may fail at detection, localization, or segmentation steps. Moreover, class frequency imbalances affect the interpretation of aggregate scores differently when the pipeline’s components interact. A total summary of the metrics used to evaluate the integrated pipeline, is presented in Table 14.

Metric	Type	Definition	Range	Interpretation
Micro Precision	Detection	$Prec_{micro} = \frac{\sum_{c=1}^{C=n} TP_c}{\sum_{c=1}^{C=n} TP_c + \sum_{c=1}^{C=n} FP_c}$ pooled across all classes	[0, 1]	Fraction of generated prompts that correspond to real objects
Micro Recall	Detection and Pipeline	$Rec_{micro} = \frac{\sum_{c=1}^{C=n} TP_c}{\sum_{c=1}^{C=n} TP_c + \sum_{c=1}^{C=n} FN_c}$ pooled across all classes	[0, 1]	Fraction of GT objects that receive a segmentation prompt (= Coverage)
MicroF1	Detection	Harmonic mean of microPrecision and microRecall	[0, 1]	Balanced detection quality, penalizes imbalance between microPrec. and microRec.
Mask IoU	Segmentation	Instance-weighted mean of per-pair IoU (matched pairs)	[0, 1]	Pixel-level mask quality, isolating segmentation from detection failures
Mask DICE	Segmentation	Instance-weighted mean of per-pair DICE (matched pairs)	[0, 1]	Complementary to IoU, more forgiving for small objects

Table 14: Integrated Pipeline Assessment Metrics Summary

A key methodological distinction between the baseline and pipeline detection evaluations concerns the aggregation strategy. The YOLO baseline evaluations of section 3.5.1 employ macro-averaging, where per-class Precision, Recall, and F1 are computed independently and then averaged with equal weight across all 15 classes. This treats each class as equally important regardless of its instance count, giving less represented classes (e.g., swimming pools with 435 instances) the same influence on the final score as more represented ones (e.g., ships with 7966 instances). Macro-averaging is appropriate for component-level evaluation because it reveals whether a model performs consistently across the full class spectrum, without letting high-frequency classes mask failures on rarer ones. The integrated pipeline instead employs micro-averaging for its detection metrics, where all TP, FP, and FN counts are pooled across classes before computing Precision, Recall, and F1. Formally, micro-averaged Precision and Recall are computed as:

$$Precision_{micro} = \frac{\sum_{c=1}^{c=n} TP_c}{\sum_{c=1}^{c=n} TP_c + \sum_{c=1}^{c=n} FP_c} \quad Recall_{micro} = \frac{\sum_{c=1}^{c=n} TP_c}{\sum_{c=1}^{c=n} TP_c + \sum_{c=1}^{c=n} FN_c}$$

where the summation runs over all $c = 1, \dots, 15$ classes. The microF1 score is the harmonic mean of microPrecision and microRecall, following the same definition as section 3.5.1. The choice of micro-averaging for pipeline detection metrics is motivated by the system’s evaluation purpose. The pipeline is assessed as an operational system, where the goal is to understand how it performs on the actual distribution of objects it encounters during deployment. In this context, correctly detecting 100 ships matters more than correctly detecting 1 helicopter, because ships occur far more frequently in the evaluation area. Micro-averaging reflects this operational reality by weighting each object instance equally, producing metrics that directly correspond to the expected per-object detection success rate during real-world use. By contrast, macro-averaging would overweight the contribution of rare classes, distorting the overall score relative to the system’s actual throughput on the majority of objects it processes.

In addition, microRecall has a second, distinct interpretation, when examining the overall integrated pipeline performance. It expresses the total Coverage, as it directly measures the fraction of objects in the scene that receive any segmentation at all. An object missed by the detector receives no mask, making undetected objects the most impactful failure mode of the integrated system. This Coverage measure, therefore, serves as a single-number summary of the pipeline’s end-to-end completeness.

Property	Macro-Averaging	Micro-Averaging	Practical Impact Examples
Unit of weight	Class	Instance	Micro: ship (7966 inst.) counts 69x more than basketball-court (115 inst.)
Rare class influence	Equal to frequent	Proportional to count	Macro: swimming-pool (0% F1) pulls average down as much as ship

Property	Macro-Averaging	Micro-Averaging	Practical Impact Examples
Evaluation purpose	Component capability	System performance	Macro for diagnostics, micro for deployment assessment
Applied to	YOLO baseline, SAM baseline	Integrated pipeline detection metrics	Both reported per-class

Table 15: Macro vs Micro Averaging Comparison

The segmentation assessment uses the same pixel-level Mask IoU and Mask DICE definitions established in section 3.5.2, but applies them in a fundamentally different evaluation context. In the SAM baseline, every ground-truth object receives a prompt and produces a mask, so segmentation quality is measured across all available instances. In the integrated pipeline, only objects that YOLO successfully detects receive a segmentation prompt. Segmentation metrics are therefore computed exclusively on matched prediction-ground-truth pairs, where matching is performed via Hungarian algorithm assignment with a matching threshold of $\text{IoU} \geq 0.1$. This restriction is essential for isolating SAM’s mask generation quality from YOLO’s detection failures, enabling fair comparison against the SAM baseline where ground-truth prompts guaranteed perfect detection. The overall segmentation IoU and DICE are reported as instance-weighted averages. Each matched pair contributes its individual IoU and DICE score to the aggregate, weighted equally regardless of class. This differs from the class-level macro-averaging of baseline experiments, where a class mean is computed first and then averaged. Instance-level weighting means that a class with 5000 matched pairs (e.g., ships) has proportionally greater influence on the overall segmentation score than a class with 50 matched pairs (e.g., basketball courts), reflecting the actual composition of segmented output the system produces.

This dual-metric framework enables the pipeline evaluation to answer two distinct questions simultaneously. Detection metrics (microPrecision, microRecall, microF1) answer whether the system finds the right objects, while segmentation metrics (instance-weighted Mask IoU, Mask DICE) answer whether it delineates them accurately once found. Per-class results for all metrics are reported alongside the aggregated scores to provide full transparency and enable class-level diagnostics, even though the aggregate pipeline scores use micro and instance-level strategies rather than macro-averaging.

Chapter 4 Experimental Results

Following the experimental setup, according to the plan presented in Chapter 3, a complete run of all experiments was performed. This chapter presents the experimental results for the two baseline evaluations of YOLO detection (using VBB and OBB) and SAM segmentation. Also, the evaluation results of the integrated Detection→Segmentation pipeline are presented. Each section addresses the corresponding research questions established in Chapter 2 and provides detailed performance analysis.

All experiments were conducted on a dedicated workstation configured specifically for deep learning inference and evaluation tasks. The system features an AMD Ryzen 7 5700G processor with 8 cores and 16 threads operating at 4.6GHz base clock, providing sufficient computational capacity for data preprocessing, metric computation, and evaluation orchestration. System memory consists of 64GB DDR4 RAM (2×32GB modules) running at 1064MHz, enabling efficient handling of large datasets and batch processing operations without memory bottlenecks. GPU acceleration is provided by a ZOTAC RTX 3090 Ti graphics card equipped with 10752 CUDA cores and 24GB GDDR6X memory operating at 1.56GHz, delivering the computational throughput necessary for neural network inference across thousands of validation images. This hardware configuration ensures consistent and reproducible inference timing measurements, as all models are evaluated under identical thermal and power conditions on the same GPU architecture, eliminating hardware variability as a confounding factor in performance comparisons. The substantial 24GB GPU memory capacity proves critical for loading large model variants and processing high-resolution aerial imagery without requiring image downsampling or batch size reduction, while the high CUDA core count enables parallel processing of multiple detection and segmentation operations simultaneously.

4.1 YOLO Detection Baseline Results

The YOLO detection baseline evaluation tested 34 models across two annotation formats, including 24 COCO-pretrained models with Vertical Bounding Boxes (VBB) and 10 DOTA-pretrained models with Oriented Bounding Boxes (OBB). This section presents the YOLO detection baseline results, addressing the three research questions of whether transfer learning from ground imagery to aerial imagery is feasible without domain-specific training, whether rotation-aware oriented bounding box (OBB) representations outperform axis-aligned vertical bounding boxes (VBB) for aerial object detection, and which YOLO architecture provides optimal performance for pipeline integration.

Across all evaluated models, a consistent performance gap emerged between the two annotation formats. Models trained or evaluated using OBB annotations substantially

outperformed those using VBB annotations in terms of F1 score, Recall, and bounding box localization metrics (IoU and DICE). The detailed results of the VBB and OBB experiments are presented in Sections 4.1.1 and 4.1.2, respectively. These subsections analyze overall performance trends, failure modes, and per-class behavior in greater depth, leading to the final selection of a detection model for the integrated Detection→Segmentation pipeline.

4.1.1 Vertical Bounding Box (VBB) Experiment

The VBB experiment evaluates transfer learning from ground-level imagery to aerial imagery by testing the main YOLO model variants. These models span YOLOv8 through YOLOv12 architectures with size variants (nano, small, medium, large, x-large) and specialized variants (e.g., YOLOv9t, YOLOv9c, YOLOv9e). All models are pretrained on COCO's ground-level natural objects using standard axis-aligned bounding boxes in (cx, cy, w, h) format.

A critical constraint of the VBB experiment is the semantic mismatch between COCO and DOTA taxonomies. Of DOTA's 15 object categories, only 3 classes have approximate semantic correspondence to COCO classes. These 3 overlapping classes comprise 18785 instances (65.1% of DOTA validation split), providing sufficient data for transfer learning assessment while acknowledging that 12 DOTA classes have no COCO equivalents and cannot be evaluated in this experiment. Table 16 presents overall performance for the top-performing VBB models. All VBB models demonstrate catastrophic failure with F1 scores below 0.3%.

Model	Mean F1	Mean Recall	Mean Precision	Mean BBox IoU	Mean BBox DICE	Inf. Time (ms)
YOLOv12l	0.1%	0.1%	30.2%	1.0%	1.2%	27.9
YOLOv11x	0.1%	0.0%	22.7%	0.7%	0.9%	25.7
YOLOv10x	0.3%	0.2%	15.6%	1.3%	1.5%	21.5
YOLOv8x	0.2%	0.1%	13.1%	1.1%	1.3%	25.4
YOLOv8l	0.2%	0.1%	16.1%	1.4%	1.6%	20.9

Table 16: YOLO-VBB Overall Performance (Top 5 Models, matching IoU>0.1)

The near-zero F1 scores across all 24 VBB models reveal a fundamental incompatibility between ground-level and aerial imagery domains rather than a minor performance gap. YOLO models pretrained on COCO learn visual representations that are inherently tied to horizontal, near-ground viewpoints where vehicles are recognized through features such as wheels, doors, windows, shadows, and street-level context. In overhead imagery these cues disappear almost entirely, replaced by roof textures, different lighting, and a radically different geometric appearance. At the same time, aerial imagery introduces extreme multiscale variation that COCO-pretrained models are not equipped to handle. In DOTA, objects of the same class appear at different pixel sizes within and across images, due to changes in ground sample distance, altitude, and sensor characteristics,

ranging from large ships occupying thousands of pixels to vehicles compressed into only a few dozen. COCO pretraining largely learns scale distributions typical of street-level photography, where object sizes are relatively consistent and depth variation is limited, whereas aerial imagery demands robust recognition across orders of magnitude of scale variation. This mismatch in learned scale priors further weakens transfer, as the model struggles to simultaneously detect very small and very large instances in vertical views. As a result, the internal visual and geometric primitives learned from COCO do not align with what is required for vertical view interpretation, producing systematic detection failures.

This domain mismatch is further amplified by semantic drift within the three nominally overlapping classes, as the per class analysis (Table 17) shows, for the best F1 score model (YOLOv12l) within the scope of this experiment. Complete per-class results for all 24 VBB models are provided in Appendix A.

Class	F1	Recall	Precision	BBox IoU	BBox DICE	GT Instances
ship	0.1%	0.1%	17.6%	0.5%	0.6%	8960
small-vehicle	0.1%	0.0%	28.6%	1.0%	1.2%	5438
large-vehicle	0.0%	0.2%	21.9%	0.6%	0.7%	4387

Table 17: YOLOv12l-VBB Per-Class Performance (3 Overlapping Classes, matching IoU>0.1)

Even when labels appear comparable, the meaning of a COCO “car” differs materially from a DOTA “Small-vehicle”. Ground imagery depicts cars embedded in rich contextual scenes with road markings, pedestrians, and traffic flow, whereas aerial imagery presents them as isolated, small, often rectangular shapes in parking lots or highways with minimal surrounding context. The model’s learned concept of a vehicle therefore diverges from what it must recognize in aerial views, deepening the transfer gap and causing unstable detection behavior, even in similar testing images. An indicative example is given in Figure 18, where YOLOv12x model performs some detections of vehicles in one image, but completely fails to repeat it in another image of similar characteristics. Even more, some of the few detection it produces, mislabel “Large-vehicles” for “Small-vehicles” or fail to separate clamped objects, producing one enclosing box for more than one vehicle.

In addition, the use of axis-aligned bounding boxes in the VBB experiment removes rotation information that is essential in aerial imagery. Objects such as ships, planes, and vehicles appear at arbitrary orientations, yet VBB forces them into horizontal rectangles that include large amounts of background and neighboring objects, degrading IoU and confusing the learning signal by blurring the distinction between object extent and enclosing box. These geometric distortions interact with the multiscale problem, because small, rotated objects become even more poorly represented when forced into oversized axis-aligned boxes.



Figure 18: Class “Small-vehicle” and “Large-vehicle” unstable detection performance from YOLOv12x model, over similar imagery

Surprisingly, some successful detections of good quality do come out occasionally, but this seems more of a random behavior, rather than some systematic case, where factors that cause success can be quantified and reproduced. Even in that case, the successful detection is not stable across all model variants as shown on Figure 19.

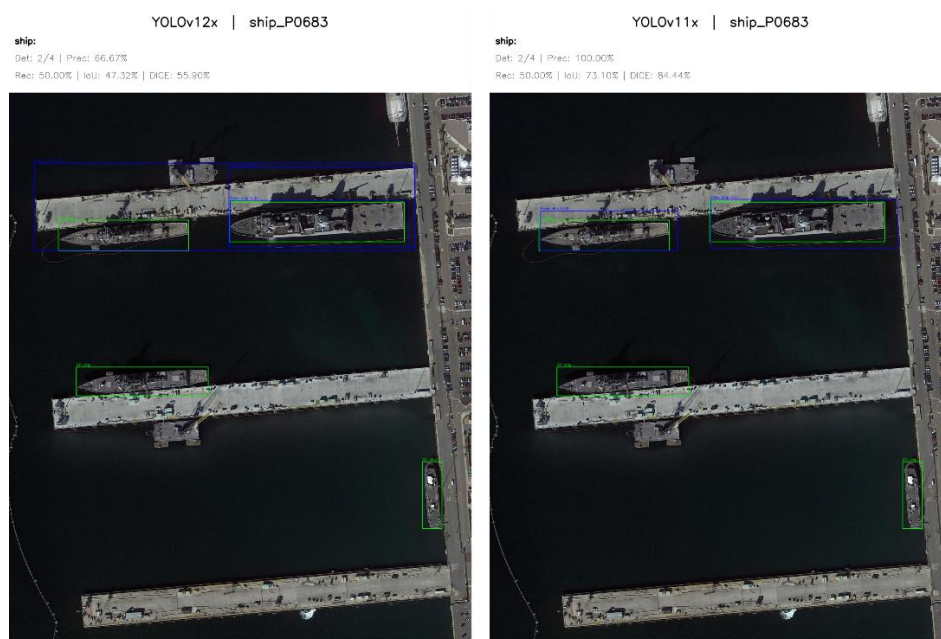


Figure 19: Class “Ship” partly successful detections, with unstable results from YOLOv12x (left) and YOLOv11x (right) models

Inference timing analysis reveals that VBB models process images rapidly, with the top-performing models achieving inference times between 21ms and 28ms per image on NVIDIA RTX 3090 Ti hardware. YOLOv10x demonstrates the fastest inference at 21.5ms, while YOLOv12l and YOLOv11x require 27.9ms and 25.7ms respectively, representing only minor variations across different architectures and size variants. This computational efficiency stems from the architectural simplicity of axis-aligned

bounding box detection, which requires only four coordinate parameters (center x, center y, width, height) compared to more complex rotated representations. The detection head can process these axis-aligned boxes through simpler geometric computations without requiring rotation angle estimation or corner point calculation, enabling faster inference throughput. However, this speed advantage proves meaningless in practical deployment contexts given the detection failure, as processing images quickly while detecting virtually no objects provides no operational value.

These factors act together rather than independently. The models do generate predictions, as shown by normal inference times and moderate Precision values, but Recall remains almost zero, meaning that virtually none of the real objects are actually detected at even a lenient IoU threshold of 0.1. This behavior is effectively more similar to random box placement, rather than meaningful recognition. Taken together, the VBB experiment provides a clear answer, that transfer learning from ground view pretrained detectors to aerial imagery is not feasible without domain-specific training or finetuning.

4.1.2 Oriented Bounding Box (OBB) Experiment

The OBB experiment evaluates YOLO models (Table 18) specifically designed and trained for rotated object detection in aerial imagery. Ten models spanning YOLOv8-OBB and YOLOv11-OBB architectures, including all size variants (nano, small, medium, large, x-large), were pretrained by Ultralytics on the DOTA training split, using native oriented bounding box annotations, that preserve rotation information through four-corner-point image coordinates. Unlike the VBB experiment, which was constrained to 3 overlapping classes (due to semantic misalignment between COCO and DOTA taxonomies), the OBB experiment evaluates all 15 DOTA categories across the validation split. This comprehensive evaluation enables assessment of aerial detection performance across diverse object types, sizes and geometric complexities, from large rectangular structures such as harbors and tennis courts to small vehicles and helicopters occupying only a few pixels in the image.

Model	Params	Mean F1	Mean Recall	Mean Precision	Mean BBox IoU	Mean BBox DICE	Inf. Time (ms)
YOLOv8n-OBB	3.2M	55.4%	45.9%	77.3%	57.1%	64.5%	44.2
YOLOv8s-OBB	11.2M	57.8%	48.3%	77.4%	60.3%	67.7%	41.7
YOLOv8m-OBB	25.9M	61.4%	52.3%	79.1%	61.3%	68.6%	46.2
YOLOv8l-OBB	43.7M	62.8%	54.3%	77.4%	61.3%	68.3%	50.2
YOLOv8x-OBB	68.2M	62.6%	54.0%	77.9%	61.8%	69.0%	61.7
YOLOv11n-OBB	2.6M	54.9%	44.5%	79.4%	58.9%	66.3%	40.4
YOLOv11s-OBB	9.4M	59.7%	49.8%	80.3%	61.0%	68.3%	39.4
YOLOv11m-OBB	20.1M	60.5%	50.5%	80.7%	62.8%	70.0%	43.4
YOLOv11l-OBB	25.3M	62.4%	52.5%	81.1%	63.9%	71.1%	47.8
YOLOv11x-OBB	56.9M	62.4%	52.3%	82.8%	64.6%	71.8%	59.8

Table 18: Complete YOLO - OBB Model Progression (matching IoU>0.1)

The OBB model progression reveals systematic performance scaling patterns that are fundamentally different from the VBB equivalent ones, which presented a detection failure. All OBB models achieve F1 scores between 54.9% and 62.4%, demonstrating functional aerial object detection capability across all 15 DOTA classes. Within each architectural generation, performance improves rapidly from nano to medium variants, with YOLOv8-OBB progressing from 55.4% F1 (nano) to 61.4% F1 (medium) and YOLOv11-OBB advancing from 54.9% to 60.5% over the same size range. Beyond medium size, however, diminishing returns emerge, as YOLOv8m-OBB at 61.4% F1 with 25.9M parameters performs nearly identically to YOLOv8x-OBB at 62.6% F1 with 68.2M parameters, despite the latter requiring 2.6x more parameters and 33% longer inference time (from 46.2 ms to 61.7 ms). This plateau indicates that medium-sized models capture most of the representational capacity necessary for aerial detection, with larger models providing only marginal gains at substantial computational cost. Across different model generations, YOLOv11-OBB outperforms YOLOv8-OBB at comparable parameter counts, possibly due to version 11 architectural refinements in the backbone and neck, that enhance feature extraction and computational efficiency (Ultralytics, n.d.). YOLOv11m-OBB achieves 60.5% F1 with 20.1M parameters, matching YOLOv8m-OBB performance (61.4% F1, 25.9M parameters) while being 23% smaller, demonstrating superior parameter efficiency.

A critical pattern across the OBB models' progression is the systematic Precision advantage of YOLOv11 over YOLOv8, with YOLOv11 variants achieving 2-5% higher Precision at all size categories. YOLOv11x-OBB reaches 82.8% Precision compared to YOLOv8x-OBB's 77.9%, a 4.9% improvement that directly impacts pipeline viability. In the integrated Detection→Segmentation architecture, each false positive detection triggers unnecessary segmentation, wasting extra inference time for these spurious boxes. The Precision-Recall trade-off also reveals design priorities, as YOLOv11x-OBB sacrifices 2% of Recall compared to YOLOv8l-OBB (52.3% versus 54.3%) to gain 5.4% in Precision (82.8% versus 77.4%), reflecting an optimization toward detection quality over coverage. This trade-off proves advantageous for pipeline integration, where high-quality prompts on detected objects are more valuable than high-volume but lower-quality prompts that overload the segmentation stage with false positives. In any case, the best overall performing model is YOLOv11x-OBB, which achieves the highest Precision and bounding box localization quality metrics while maintaining competitive F1 score and Recall.

Comparing OBB and VBB YOLO models in terms of detection accuracy metrics is not possible, due to fundamental differences in training domains and evaluation scope (VBB models were pretrained on COCO and evaluated on only 3 overlapping classes). The performance gap between VBB versions (0.1-0.3% F1) and OBB versions (54.9-62.4% F1) reflects this domain incompatibility rather than a fair architectural comparison, making direct metric-to-metric evaluation meaningless. However, inference timing provides a valid comparison point, as computational efficiency depends solely on architectural design and not on training data. VBB models demonstrate faster inference

times (21–28 ms per image) compared to OBB models (40–62 ms), reflecting an approximately $2\times$ speed advantage for axis-aligned detection. This efficiency arises not only from the simpler four-parameter VBB representation (cx, cy, w, h), but more critically from the lower computational cost of axis-aligned IoU calculations during Non-Maximum Suppression (NMS). In contrast, OBB detection requires eight-coordinate corner regression and polygon-based IoU computation for rotated NMS, which is substantially more expensive.

A Per-class performance analysis for the dominant YOLO11x-OBB model (Table 19) reveals a significant variation across the 15 DOTA categories, ranging from near-perfect detection for tennis courts (93.7% F1) to complete failure for swimming pools (0.4% F1), a performance difference that exposes class-specific detection challenges. Tennis courts achieve exceptional performance through a combination of distinctive rectangular geometry with consistent aspect ratios, uniform internal texture contrasting sharply with surroundings, and standard dimensions that reduce scale variability. The 98.1% Precision indicates the model rarely misclassifies other structures as tennis courts, while 89.6% Recall confirms reliable detection across varying imaging conditions, missing only $\sim 10\%$ of the 750 ground-truth instances. Harbor detection (89.5% F1) succeeds despite complex irregular shapes because distinctive waterfront infrastructure patterns, such as docks, piers, and breakwaters, provide strong visual signatures that the model learns to recognize across diverse harbor configurations from marinas to shipping ports. Large vehicles (85.1% F1) benefit from clear object boundaries, consistent shape templates for trucks and buses, and abundant training data (3737 instances), achieving 91.1% Precision that demonstrates effective discrimination from background elements and smaller vehicles (Figure 20). Complete per-class results for all OBB models are provided in Appendix B.

Class	F1	Recall	Precision	BBox IoU	BBox DICE	GT Instances
tennis-court	93.7%	89.6%	98.1%	92.8%	96.2%	750
harbor	89.5%	86.8%	92.5%	70.8%	81.8%	2636
large-vehicle	85.1%	79.8%	91.1%	81.6%	89.6%	3737
plane	78.4%	66.9%	94.6%	80.8%	89.1%	2591
baseball-diamond	75.1%	66.4%	86.6%	73.7%	84.3%	214
ship	74.2%	62.0%	92.3%	75.6%	85.4%	7966
small-vehicle	62.2%	52.9%	75.6%	76.3%	86.2%	4489
basketball-court	61.2%	48.5%	83.1%	90.8%	95.1%	115
ground-track-field	60.3%	48.6%	79.5%	84.6%	91.4%	141
storage-tank	58.1%	41.6%	96.5%	75.4%	85.3%	2343
helicopter	54.9%	38.4%	96.6%	74.4%	85.1%	67
roundabout	53.4%	37.4%	93.1%	70.9%	81.4%	181
soccer-ball-field	48.8%	38.6%	66.3%	87.2%	92.5%	141
bridge	40.6%	27.6%	76.6%	63.9%	76.6%	449
swimming-pool	0.4%	0.2%	20.0%	15.1%	26.3%	435

Table 19: YOLOv11x-OBB Per-Class Detection Performance (matching IoU>0.1)

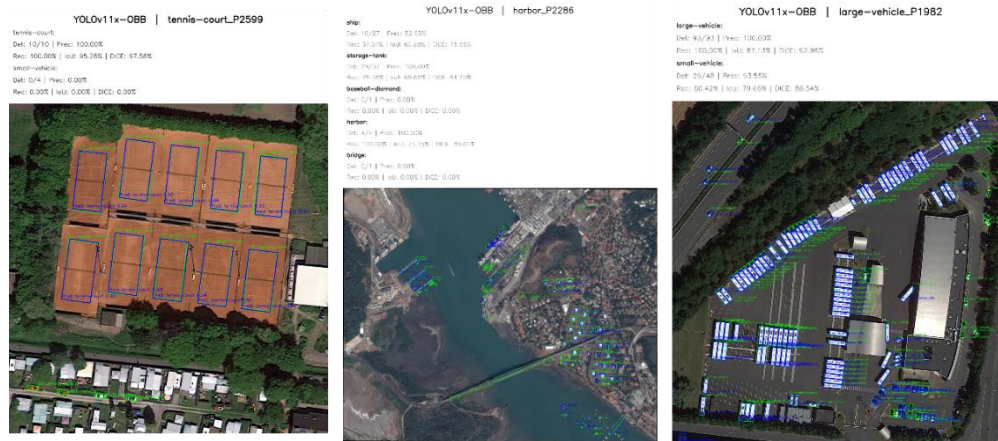


Figure 20: YOLOv11x-OBB best detection performance classes examples

Medium-performing classes (Figure 10Figure 21) exhibit functional but imperfect detection that reveals specific failure modes. Small vehicles achieve only 62.2% F1 despite abundant training data (4489 instances), with low Recall (52.9%) reflecting the fundamental challenge of detecting objects of a few pixels' size in dense urban scenes. These scenes present a high degree of difficulty, as many vehicles in crowded parking lots or multi-lane highways are missed due to occlusion by trees or buildings, ambiguous boundaries when tightly packed, or insufficient resolution. The 25.4% loss in Precision due to false positives indicate some confusion between vehicles and similar-sized objects. Storage tanks demonstrate the opposite pattern with 58.1% F1 driven by 41.6% Recall but 96.5% Precision, indicating conservative detection where the model only predicts storage tanks when very confident, producing only a few false positives but missing almost half of the valid instances. Ground track fields on the other hand can be quite ambiguous since they refer to the part of the court that is meant for running and usually surround other sport courts in a unified facility, which is a not well-defined individual object. In that way, detection may be difficult, depending on the total surrounding environment's complexity.

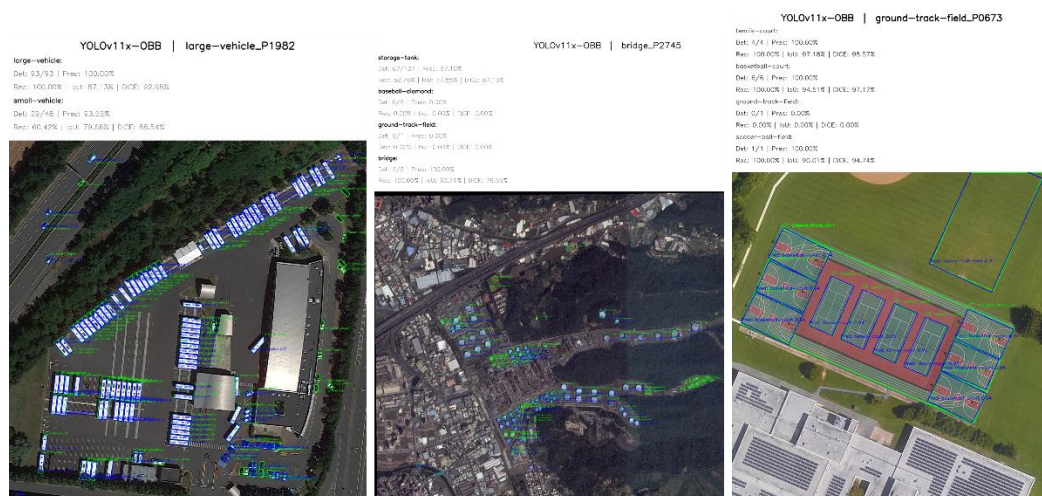


Figure 21: YOLOv11x-OBB medium detection performance classes examples

The three lowest-performing classes (Figure 22) reveal failure cases, possibly connected to complicated object geometries. Bridge detection (40.6% F1, 27.6% Recall) struggles with ambiguous boundaries where bridges transition gradually into roads or water without clear visual discontinuities, soccer ball fields (48.8% F1, 38.6% Recall) suffer confusion with similar green rectangular structures despite distinctive geometry, and swimming pools detection fails completely (0.4% F1, 0.2% Recall) due to including extreme class imbalance (435 instances versus 4489 small-vehicles), very small object size (5-15 pixels) that amplifies localization sensitivity and visual similarity to other urban surrounding objects, making detection difficult.

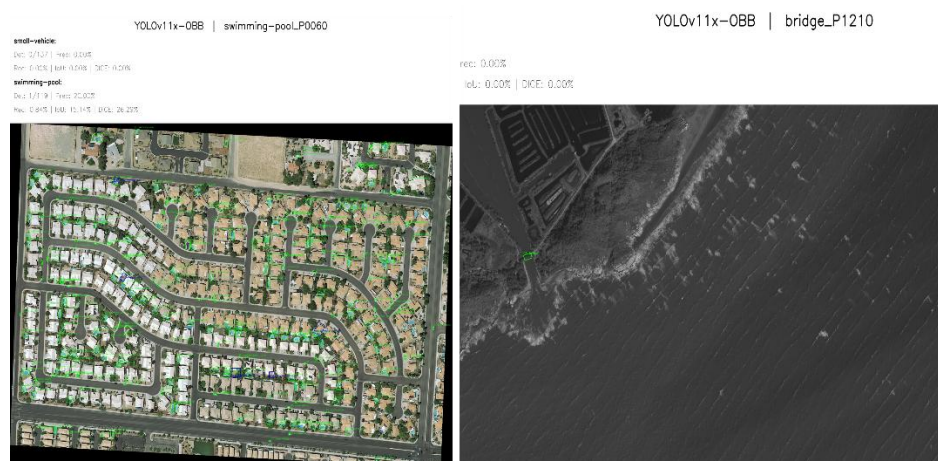


Figure 22: YOLOv11x-OBb worst detection performance classes examples

Comparing swimming pool failure (435 instances, 0.4% F1) with baseball diamond success (214 instances, 75.1% F1) reveals that visual distinctiveness is crucial for detection performance (Figure 23). Baseball diamonds possess unique geometric signatures including diamond-shaped infields with right-angle baselines plus semi-circular outfield arcs that can compensate for limited training data, while swimming pools lack distinctive features and appear as generic rectangular water bodies. This finding suggests that targeted data augmentation for rare classes should prioritize increasing visual diversity through varying imaging angles, lighting conditions, and occlusion patterns rather than simply collecting more examples of identical appearance.

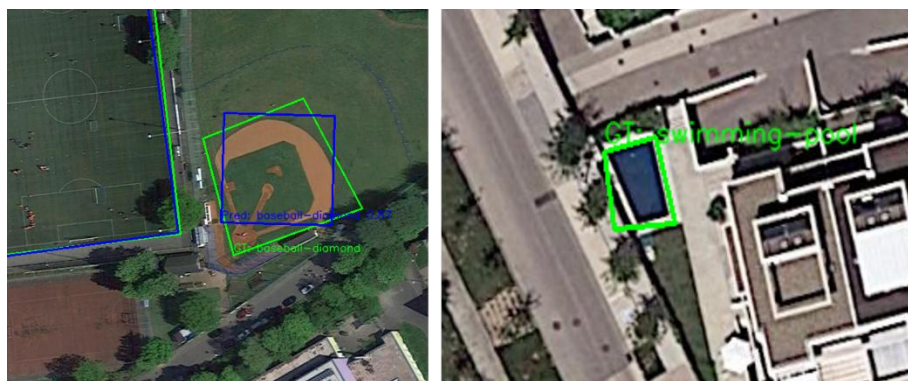


Figure 23: “Baseball diamond” successful vs “swimming pool” failed detection by YOLOv11x-OBb

Moreover, the experimental results revealed cases of annotation discrepancies within the DOTA v1.0 dataset (Figure 24) that cannot be resolved without extensive manual review and correction. Object categories, such as roundabouts, are often annotated loosely or inconsistently, leading to systematically lower IoU and DICE scores. In these cases, predictions that are geometrically tighter and more accurate than the ground-truth are penalized, because improved localization results in poorer alignment with imprecise annotations. Additional instances of mislabeled objects were also identified, introducing noise into the evaluation and degrading metric reliability in ways that do not reflect actual model performance. These annotation issues likely affected model training and evaluation, meaning that reported performance for several classes is probably underestimated. However, quantifying the exact impact of these discrepancies would require a thorough audit of the entire training split, which lies beyond the scope of the present study.



Figure 24: “Roundabout” loose annotation and tight model detection (left) and a “basketball-court” labeled as “soccer-ball-field” (right)

4.2 SAM Segmentation Baseline Results

The SAM segmentation baseline evaluation assesses the Segment Anything Model's zero-shot segmentation capability on aerial imagery. It utilizes ground-truth bounding boxes as prompts, eliminating detection errors, in order to isolate pure segmentation performance. Six SAM configurations were evaluated, spanning three model sizes (ViT-H with 630M parameters, ViT-L with 307M parameters, ViT-B with 91M parameters) and two prompt strategies (box prompts using full bounding box coordinates, point prompts using single centroid coordinates). All configurations were evaluated on the AerialFuseCV validation split, which provides both oriented bounding boxes from DOTA v1.0 annotations and pixel-perfect segmentation masks from iSAID annotations for matched instances across 15 object classes. This dual-annotation structure enables direct assessment of SAM's mask generation quality when provided with ideal spatial guidance, representing an upper bound on segmentation performance achievable in the integrated pipeline, where prompt quality depends on upstream detector accuracy.

The evaluation addresses three research questions regarding SAM's applicability to aerial imagery segmentation. It is aimed to define of whether SAM's zero-shot segmentation capability, trained primarily on natural ground-level images, transfers effectively to overhead aerial perspectives without domain-specific fine-tuning, how prompt strategy (box versus point) affects segmentation quality and computational efficiency in the aerial domain and finally which SAM architecture variant provides optimal performance for pipeline integration, balancing segmentation accuracy against inference speed and memory requirements. These questions directly inform the design of the integrated Detection→Segmentation pipeline, as the chosen SAM configuration must process thousands of detection-derived prompts efficiently while maintaining mask quality sufficient for geospatial applications that require precise object boundaries.

4.2.1 Box-Prompted Segmentation

Box-prompted segmentation evaluates SAM's ability to generate accurate masks when provided with complete spatial context through object bounding box specification. Each ground-truth bounding box from AerialFuseCV annotations, represented as four corner points in oriented bounding box (OBB) format, must be converted to an axis-aligned (VBB) enclosing rectangle before being passed to SAM's box encoder, as SAM's architecture does not natively support rotated box prompts and expects rectangular regions defined by top-left and bottom-right coordinates in axis-aligned format. This conversion computes the minimum and maximum x and y coordinates across the four OBB corner points to construct an axis-aligned bounding box, that fully encloses the oriented object, inevitably including additional background pixels for rotated objects and neighboring objects for clamped object cases, but maintaining complete spatial coverage of the target region. The OBB to VBB conversion introduces a structural limitation, where SAM must segment rotated objects from non-ideal prompts, potentially degrading mask quality for highly elongated or rotated instances. The VBB box is then passed to SAM's box encoder, which processes the spatial extent as a dense geometric prior for mask generation. This prompt strategy provides maximal spatial information, defining the approximate object location, scale, and extent, enabling SAM to focus mask prediction within the bounded region rather than searching the entire image.

Assessment results (Table 20) show that box-prompted SAM configurations achieve robust segmentation performance across all three model sizes, with mean mask IoU ranging from 66.9% to 67.7% and DICE coefficients from 78.5% to 79.0%, demonstrating that SAM's zero-shot segmentation capability transfers effectively to aerial imagery when provided with accurate spatial prompts. The minimal performance variation across model sizes, spanning only 0.8% in IoU and 0.5% in DICE, indicates that segmentation quality is primarily determined by prompt accuracy rather than model capacity in this domain. SAM-ViT-H achieves marginally highest IoU at 67.7% with 943ms inference time, while SAM-ViT-L and SAM-ViT-B produce nearly identical results at 67.3% and 66.9% IoU respectively, with substantially faster inference at 800ms and 669ms. The match rate (representing the percentage of ground-truth instances for

which SAM successfully generated masks) exceeds 99.5% across all configurations, confirming that SAM reliably produces segmentation outputs when provided with valid box prompts regardless of object class, size, or orientation.

SAM Variant	Params (M)	meanMask IoU (%)	meanMask DICE (%)	Inference (ms)	Match Rate (%)
SAM-ViT-H-BOX	636	67.7	79.0	943.2	99.5
SAM-ViT-L-BOX	308	67.3	78.7	800.4	99.5
SAM-ViT-B-BOX	91	66.9	78.5	669.4	99.9

Table 20: SAM Box-Prompted Overall Performance

Per-class segmentation performance reveals substantial variation across object categories, with mask IoU ranging from 40.5% for helicopters to 83.8% for tennis courts, indicating that segmentation difficulty depends on intrinsic object characteristics rather than model limitations. High-performing classes (IoU above 75%) include tennis courts (83.8%), soccer ball fields (81.2%), basketball courts (75.4%), and large vehicles (74.9%), all sharing geometric regularity or consistent shape templates that enable accurate mask prediction even when objects appear at arbitrary orientations. Medium-performing classes (IoU 60-75%) include small vehicles (71.2%), swimming pools (69.7%), ships (67.5%), baseball diamonds (67.3%), storage tanks (64.8%), and harbors (61.6%), demonstrating functional segmentation quality suitable for most geospatial applications despite moderate boundary ambiguity or shape variation within classes. Low-performing classes (IoU below 60%) include ground track fields (56.7%), bridges (54.7%), planes (50.0%), and helicopters (40.5%), reflecting fundamental segmentation challenges from elongated thin structures, complex articulated shapes, and severe occlusion patterns that confound SAM's boundary prediction even with accurate box prompts.

Class	Mask IoU (%)	Mask DICE (%)	GT Instances
tennis-court	83.8	90.5	750
soccer-ball-field	81.2	88.6	141
basketball-court	75.4	84.5	115
large-vehicle	74.9	84.6	3737
small-vehicle	71.2	82.6	4489
swimming-pool	69.7	80.4	435
ship	67.5	79.5	7966
baseball-diamond	67.3	79.5	214
storage-tank	64.8	77.2	2343
harbor	61.6	75.2	2636
roundabout	58.5	72.1	181
ground-track-field	56.7	70.2	141
bridge	54.7	69.1	449
plane	50.0	65.0	2591
helicopter	40.5	55.0	67

Table 21: SAM-ViT-B-BOX Per-Class Segmentation Performance

In general, SAM presents an excellent segmentation capability for well defined and distinguishable objects, that are enclosed in a tight detection polygon (Figure 25Figure 25: SAM segmentation of class “helicopter” instances). It seems to perform well even under the limitation of processing only VBB boxes that overlap each other in crowded environments (Figure 26). Prompting box quality appears to be the main factor that drives success, as it enables SAM to segment objects even in high altitude (small scale) imagery (Figure 27), that inherently present an increased difficulty level due to small size and surrounding environment occlusions.

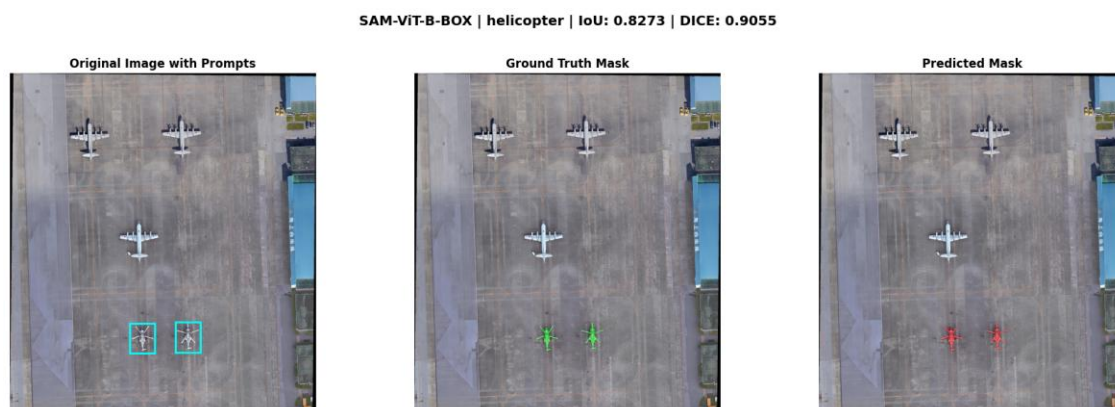


Figure 25: SAM segmentation of class “helicopter” instances

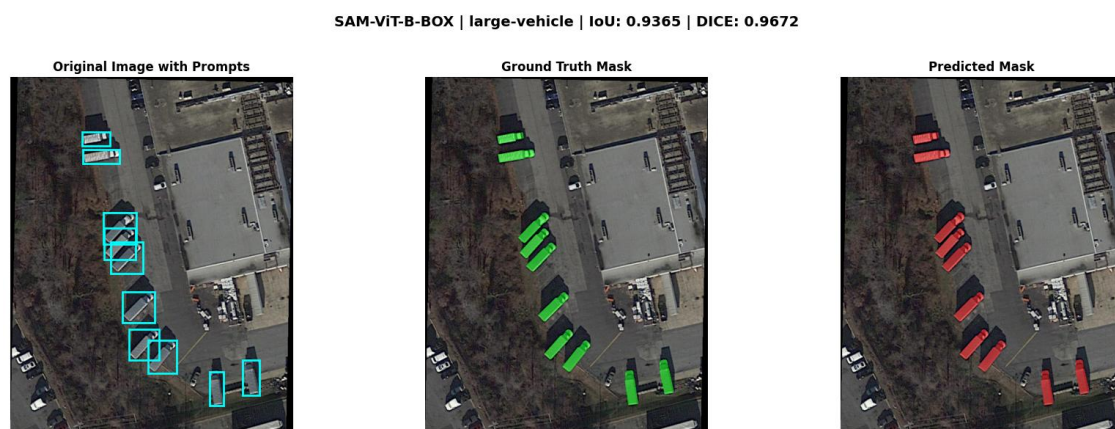


Figure 26: SAM segmentation of class “large-vehicle” instances

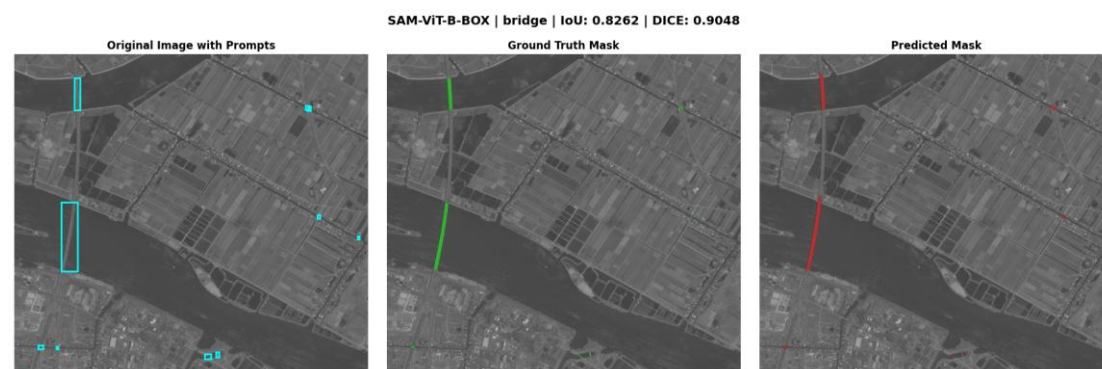


Figure 27: SAM segmentation of class “bridge” in small scale imagery

Segmentation failures do occur, even in cases of large scale imagery with big distinguishable objects, as the background inclusion in the VBB, especially in highly rotated objects, can mislead the model. An example is given in Figure 28, where the aircraft carrier (of the class “ship”) failed to be segmented, as its enclosing box sits loosely around the object, including a lot of sea and dock surface. In contrast, the small ship was segmented perfectly, despite its smaller size and the fact that it lies between the carrier and the dock, which partly occludes the object boundaries. This example verifies the box quality importance for the segmentation rate of success.

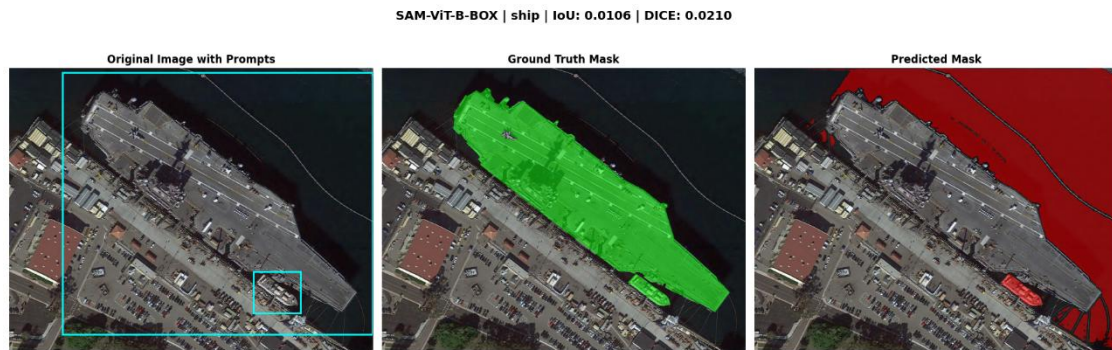


Figure 28: Failed segmentation of the aircraft carrier (class "ship") due to VBB background inclusion

AerialFuseCV dataset’s box annotations come from DOTA v1.0, and therefore they inherit its annotation discrepancies, as described in section 4.1.2. Since loose enclosing polygons affect the segmentation quality these discrepancies lead to a degradation of total computed metrics, with their total effect not possible to be quantified, without extensive dataset review at an image-by-image level. Figure 29 presents an example of different segmentation results, for the same class and for a comparable sized class-object, due to bbox annotation inconsistency.

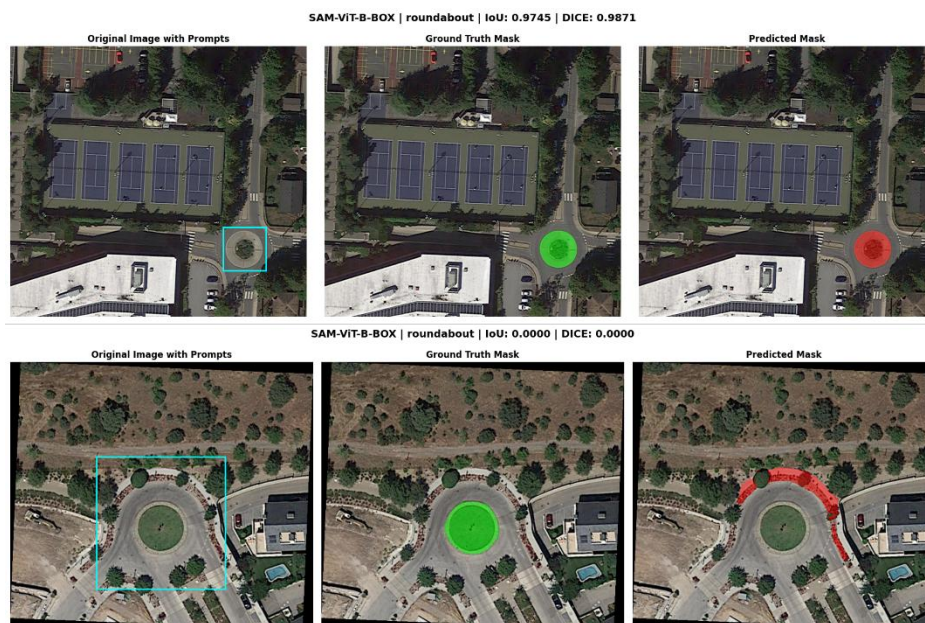


Figure 29: Successful (up) and failed (down) segmentation of objects of class “roundabout”

Focusing on the inference timing analysis, it becomes clear that a speed-accuracy trade-off across model sizes is present, with smaller variants achieving disproportionate efficiency gains while experiencing a minimal quality loss. SAM-ViT-B processes masks 29% faster than ViT-H (669ms vs 943ms) while sacrificing only 0.8% in IoU, representing a highly favorable efficiency-to-quality ratio. This efficiency advantage compounds when the whole dataset is being processed, as SAM-ViT-B saves ~ 2.5 mins over SAM-ViT-L and ~ 5 mins over SAM-ViT-H, which translates to meaningful savings in operational deployment scenarios involving thousands of daily images. The computational cost of larger SAM variants stems primarily from the vision transformer encoder, which processes the entire image at full resolution, before applying the prompt-conditioned mask decoder, making inference time roughly proportional to encoder parameter count rather than prompt complexity or object count. Segmentation times could be higher with batch processing (inference with multiple detection boxes at a time), but SAM's segmentation prompt interface processes each bounding box in a separate inference operation, as the released API does not natively accept multiple box prompts in a single batch. When multiple boxes are provided, segmentation must be done by iterating over each box and making sequential calls to SAM's predictor, effectively requiring serial processing of box prompts rather than true batched prompting. This is a significant limitation at the moment, as YOLO predicts multiple objects and multiple object instances in the same image and a serial segmentation processing presents an important performance bottleneck, when aiming towards a real or near real time system.

4.2.2 Point-Prompted Segmentation

Point-prompted segmentation evaluates SAM's ability to generate accurate masks when provided with minimal spatial context through a single seed point coordinates. It represents the most challenging prompting scenario, where SAM must infer object extent, scale, and boundaries from a single interior point without explicit spatial guidance. The seed point can be manually placed on what is considered to be the best location over the object's surface, in interactive segmentation workflows, but in an automated system that is not feasible. For that reason, the prompting point is automatically calculated as the corresponding bbox's centroid. It is computed as the geometric mean of the four corner points and passed to SAM's point encoder as a positive prompt. This prompt strategy is evaluated, as it could be proved useful in operational scenarios where objects are highly rotated and their corresponding VBB includes significant background area, that can degrade segmentation quality or in cases of object instances that are densely located in small areas (e.g. vehicles in parking lots). One important drawback of this approach is the possibility that the automatically calculated seed point will not lie on the object itself, especially in irregular shaped and rotated object instances. The point-prompted evaluation experiment establishes a lower bound on achievable segmentation quality under minimal spatial guidance, contrasting with the box-prompted upper bound to quantify the value of richer prompt information.

The experimental results analysis for all SAM variants (Table 22), shows comparable performance despite model size, since 7x model capacity from SAM-ViT-B to SAM-ViT-H, results in an insignificant 0.7% performance gain. Mean mask IoU ranges from 46.6% to 47.5% and DICE coefficients from 56.8% to 57.5%, which constitutes a ~20% performance reduction from box prompted cases, while proving that spatial context provided by bounding boxes is critical for accurate segmentation in aerial imagery. Paradoxically, SAM-ViT-L-POINT achieves marginally highest performance (47.5% IoU, 57.5% DICE) despite having ~2x fewer parameters than SAM-ViT-H and ~3x more than SAM-ViT-B, possibly suggesting that medium size encoder provides optimal balance between feature extraction capability and data overfitting.

SAM Variant	Params (M)	meanMask IoU (%)	meanMask DICE (%)	Inference (ms)	Match Rate (%)
SAM-ViT-H-POINT	632	47.3	57.2	1015.3	100.0
SAM-ViT-L-POINT	308	47.5	57.5	852.6	100.0
SAM-ViT-B-POINT	91	46.6	56.8	724.5	100.0

Table 22: SAM Point-Prompted Overall Performance

Point prompted per-class analysis (Table 23) exhibits substantially different rank ordering compared to box-prompted results, with uniformly degraded absolute values. This reveals that prompt strategy affects object categories in a non-uniform way, that is based on their geometric characteristics. Tennis courts remain the highest-performing class at 74.6% IoU, demonstrating that large rectangular structures with high internal contrast and consistent geometry maintain robust segmentation even under minimal spatial guidance. Large vehicles (65.2% IoU) and small vehicles (62.7% IoU) achieve surprisingly strong performance with point prompts, indicating that compact objects with relatively uniform circular or rectangular shapes enable SAM to infer boundaries effectively from centroid locations alone. Also, this reveals that the practice of calculating the centroid as the prompting seed point, is effective for those specific classes. Swimming pools (59.3% IoU) demonstrate moderate point-prompted performance despite the detection failure of the YOLO baseline experiments, which confirms that segmentation quality decouples from detection difficulty when correct spatial prompts are provided. In other words, an object may be difficult to be detected but can be segmented very well if it successfully does. Medium-performing classes include ships (47.9% IoU), soccer ball fields (47.8%), baseball diamonds (47.2%), and planes (44.0%), experiencing moderate degradations that reflect increased boundary ambiguity when objects exhibit elongated shapes, irregular boundaries, or variable orientations without explicit bounding box constraints. Low-performing classes reveal severe point-prompt failures, with bridges achieving only 12.8% IoU, helicopters at 29.4%, and storage tanks at 32.3%. Bridge performance collapse stems from geometric ambiguity where linear infrastructure transitions gradually into surrounding terrain, making centroid-based segmentation impossible without explicit spatial extent definition. Storage tank degradation reflects scale ambiguity, as circular objects of

varying sizes cannot be reliably segmented from point prompts alone without radius information encoded in bounding boxes.

Class	Mask IoU (%)	Mask DICE (%)	GT Instances
tennis-court	74.6	83.5	750
large-vehicle	65.2	75.4	3737
small-vehicle	62.7	74.5	4489
swimming-pool	59.3	69.5	435
ship	47.9	59.4	7966
soccer-ball-field	47.8	57.3	141
baseball-diamond	47.2	58.5	214
plane	44.0	55.7	2591
basketball-court	38.4	46.6	115
roundabout	36.7	43.9	181
ground-track-field	35.2	44.4	141
harbor	34.4	45.4	2636
storage-tank	32.3	43.3	2343
helicopter	29.4	37.7	67
bridge	12.8	18.3	449

Table 23: SAM-ViT-L-POINT Per-Class Segmentation Performance

Examples from the experimental results show cases where the centroid point calculation for SAM prompting may be beneficial for some classes, while totally inappropriate for others. Cases of strong segmentation results may be seen on Figure 30, where good seed point placement leads to excellent masks, even with linear objects (such as harbors) and objects that are densely put together in a small area (such as parked vehicles).

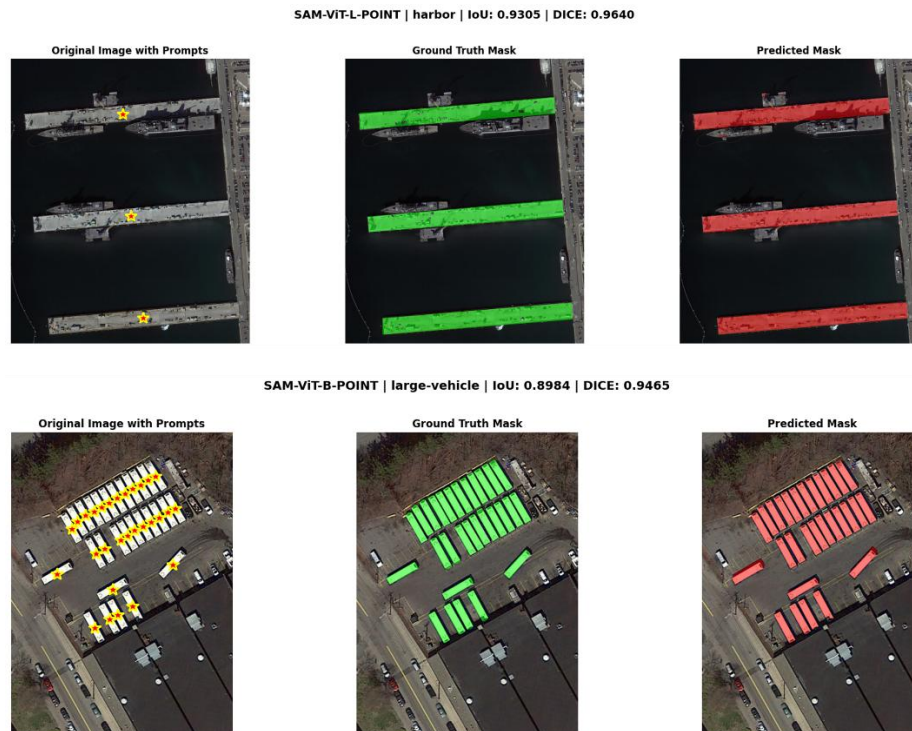


Figure 30: High quality segmentation cases with bbox centroid point prompting

In Figure 32 objects of class “roundabout” are segmented perfectly, as the centroid of the bbox lies well on the main structure’s surface, effectively eliminating the loose bounding box ambiguity that presented in the box prompting experiment. Of course, overall class “roundabout” performance, shows that this is not always the case, as it is ~26% lower than the box prompting one, for SAM-ViT-B-POINT.



Figure 31: Case of class “roundabout” that benefits from centroid point SAM prompting

On the other hand, segmentation results of other classes objects are, in many cases, wrong, due to the bbox centroid point placement. Examples of such cases are presented in Figure 32 for classes “ground-track-field” and “tennis-court”. For the field, the calculated centroid seed point physically lies inside the field, enclosed by the track, leading to a segmentation that doesn’t correspond to the intended object. Also, for the tennis courts, the centroid point resulted to the segmentation not of the actual field, but rather than the central nets on top of which it happened to lie. That reveals an inherent limitation of the seed point placement, that may fail depending on objects’ shapes, but also an “agnostic” type of segmentation procedure followed by SAM model, that can be performed almost in every defined object (zero-shot). This is a very powerful capability, but its success lies on the quality of object localization.

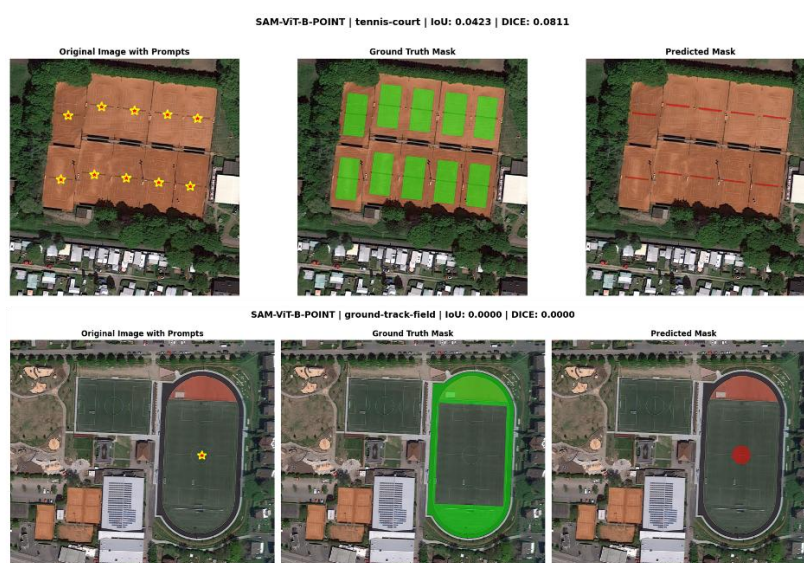


Figure 32: Case of class “roundabout” that benefits (up) and case of class “ground-track-field” that does not benefit from centroid point SAM prompting

The inference timing analysis across point-prompted configurations shows that SAM-ViT-B-POINT processes images at 724.5ms, SAM-ViT-L-POINT at 852.6ms, and SAM-ViT-H-POINT at 1015.3ms per image. SAM-ViT-B is 29% faster than ViT-H while sacrificing only 0.7% in IoU, following the same favorable efficiency-to-quality ratio observed in the box-prompted experiment. Processing the entire validation split (1060 images, 28032 prompts) takes around 13 minutes for ViT-B and 18 minutes for ViT-H, a ~5 minute difference that compounds at operational scale. Given the negligible quality variation across model sizes (0.9% IoU span), SAM-ViT-B-POINT would again represent the most efficient configuration if point prompts were to be adopted

4.2.3 Box vs Point Prompted Segmentations' Comparison

The performance gap between prompting strategies is consistent across all three model sizes, indicating that prompt quality matters more than model capacity for aerial object segmentation. Mean IoU degradation when switching from box to point prompts amounts to 20.4% for ViT-H, 19.8% for ViT-L, and 20.3% for ViT-B, which constitutes a stable ~20% penalty, regardless of encoder size. The same pattern holds for DICE coefficients, with gaps of 21.8%, 21.2%, and 21.7% respectively. This consistency suggests that the information lost by replacing a bounding box with a single centroid point is the dominant factor and that additional model parameters cannot compensate for the missing spatial context. Interestingly, model size variation within each prompting strategy is minimal. Box-prompted IoU spans only 0.8% across the three variants (66.9–67.7%) and point-prompted IoU spans 0.9% (46.6–47.5%). In both cases, the quality difference between the smallest and the largest model is an order of magnitude smaller than the difference introduced by switching prompt type. This reinforces the conclusion that, for aerial imagery segmentation with SAM, investing in better prompts yields far greater returns than investing in larger encoders.

Cross-prompt, per-class comparison using the SAM-ViT-B variant (Table 24) reveals that no object class benefits from point prompts over box prompts, with IoU degradation ranging from 8.4 to 43.6%. The smallest gaps occur for planes (8.4%), tennis courts (8.7%), and swimming pools (9.4%). These classes share compact and geometrically regular shapes, where the bbox centroid reliably falls on the object's surface, allowing SAM to infer reasonable boundaries even without explicit extent information. The largest gaps occur for bridges (43.6%), basketball courts (37.1%), and storage tanks (34.0%). Bridges suffer from extreme geometric ambiguity, as their elongated, thin structure makes a single centroid point insufficient to define object extent. Complete per-class results for all SAM models and prompt types are provided in Appendix C.

Class	Box IoU (%)	Point IoU (%)	IoU Gap (%)	Box DICE (%)	Point DICE (%)	DICE Gap (%)
bridge	54.7	11.1	43.6	69.1	16.6	52.5
basketball-court	75.4	38.2	37.1	84.5	46.8	37.7
storage-tank	64.8	30.8	34.0	77.2	41.3	35.9

Class	Box IoU (%)	Point IoU (%)	IoU Gap (%)	Box DICE (%)	Point DICE (%)	DICE Gap (%)
soccer-ball-field	81.2	52.7	28.4	88.6	62.8	25.8
roundabout	58.5	32.2	26.3	72.1	38.6	33.5
harbor	61.6	35.9	25.7	75.2	48.1	27.0
ground-track-field	56.7	34.5	22.3	70.2	44.3	25.9
baseball-diamond	67.3	45.7	21.6	79.5	57.0	22.5
ship	67.5	47.1	20.3	79.5	58.4	21.1
helicopter	40.5	27.9	12.6	55.0	36.4	18.5
large-vehicle	74.9	63.1	11.8	84.6	73.7	10.9
small-vehicle	71.2	59.7	11.4	82.6	72.1	10.5
swimming-pool	69.7	60.3	9.4	80.4	70.3	10.1
tennis-court	83.8	75.1	8.7	90.5	83.5	7.0
plane	50.0	41.6	8.4	65.0	53.6	11.4

Table 24: Box vs Point Prompt Performance Gap for SAM-ViT-B variant

Inference time comparison between the two prompting strategies (Table 25) shows that point prompts are only marginally slower than box prompts. Across all three model sizes, the point-to-box time ratio ranges from 1.07x to 1.08x, corresponding to absolute differences of 52 to 72 milliseconds per image. At dataset scale, this translates to roughly one additional minute of processing for the entire validation split (1060 images), an operationally negligible overhead. The near-parity in inference cost is expected, as both strategies share the same image encoder, which handles the image in a single forward pass and dominates total computation time. The small time overhead, observed for point prompts, is possibly due to SAM’s ambiguity-aware design. When prompted with a single point, SAM generates three candidate masks at different scales, which are scored based on confidence scores and the one with the highest score is selected as output mask. On the other hand, box prompts reduce ambiguity, by sufficiently defining the object spatial boundaries, so SAM knows exactly what region to segment, producing a deterministic mask. This additional scoring and selection step, when SAM is point-prompted, accounts for the measured 52-72ms difference.

SAM Variant	Box (ms)	Point (ms)	Difference (ms)
SAM-ViT-H	943.2	1015.3	72.1
SAM-ViT-L	800.4	852.6	52.2
SAM-ViT-B	669.4	724.5	55.1

Table 25: Box vs Point Inference Time Comparison per SAM Variant

The combined performance and timing analysis leads to a clear conclusion regarding prompting strategy selection for the integrated Detection→Segmentation pipeline. Point prompts produce ~20% lower IoU than box prompts, consistently across all model sizes and all object classes, while offering no meaningful computational advantage (7-8% slower). This means that switching from box to point prompts would degrade segmentation quality substantially, without any speed benefit to justify the trade-off. Even in the best-case class scenario (tennis courts), point-prompted IoU at 75.1%

remains 8.7% below the box-prompted 83.8%, a gap that is unacceptable for geospatial applications requiring precise object delineation. For the integrated pipeline, where YOLO detections provide bounding boxes as a natural output, the box-prompted strategy is the only viable option. The detector already produces spatial extent information at no additional cost and passing it directly to SAM as a box prompt exploits the full spatial context available. Downgrading to point prompts, by extracting only the box centroid, would discard useful localization information and result in worse masks at essentially the same processing time. The experimental evidence therefore validates box-prompted segmentation as the default strategy for any YOLO→SAM pipeline operating on aerial imagery.

4.3 Unified Detection→Segmentation Pipeline Results

This section presents the results from the integrated Detection→Segmentation pipeline, which combines the YOLOv11x-OBB model for object detection, with the box-prompted SAM-ViT-B model for segmentation into a single end-to-end system. While the preceding baseline evaluations assessed each component independently against ground-truth annotations, the pipeline evaluation simulates a real-world performance where detection outputs directly serve as segmentation prompts. This setup enables quantification of error propagation between components and assessment of system-level behavior that cannot be observed through isolated component testing. The evaluation is conducted on the AerialFuseCV validation split (456 images, 26255 ground-truth instances across 15 object classes). YOLO predicts objects and SAM segments them. The resulting segmentation masks are matched to ground-truth via Hungarian algorithm matching, with $\text{IoU} \geq 0.1$ as matching threshold. Both detection-level metrics (Precision, Recall, F1) and segmentation-level metrics (mask IoU and mask DICE) are computed per matched pair. This dual-metric approach separates detection from segmentation quality, enabling identification of which pipeline stage limits overall performance for each object class.

4.3.1 Pipeline Final Configuration

The integrated pipeline final configuration (Figure 33) was determined by selecting the best-performing detection and segmentation models from the respective baseline experiments. The selection was performed based on pipeline-level performance rather than component-level metrics in isolation, as the two stages interact and computational bottlenecks differ between components.

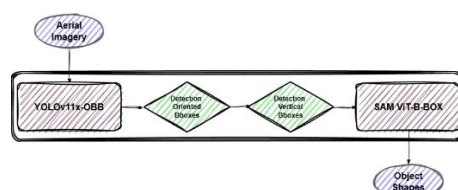


Figure 33: Detection→Segmentation pipeline implementation

YOLO model selection for unified pipeline: Based on the performance-efficiency analysis of section 4.1, **YOLOv11x-OB** was selected as the detection model for the integrated pipeline. Although it ties with YOLOv11l-OB for highest F1 score at 62.4%, YOLOv11x-OB achieves superior Precision (82.8% versus 81.1%), better bounding box localization quality (64.6% versus 63.9% IoU) and slightly tighter box fitting (71.8% versus 71.1% DICE) at an only 25% inference cost increase (59.8ms versus 47.8ms). The decision is based on the fact that the localization quality proved to be particularly important, as SAM's box-prompted segmentation directly depends on prompt accuracy. Tighter detection boxes lead to better spatial guidance for mask generation, while downgraded localization would equally degrade downstream segmentation quality, even when objects are correctly detected. The inference speed sacrifice is justified when considering the whole pipeline timewise performance. Detection time at 59.8ms represents only 8.2% of total pipeline latency, which is dominated by SAM segmentation at ~670ms per image. Selecting the YOLOv11l-OB instead of YOLOv11x-OB variant, would optimize detection speed by 12ms and reduce total pipeline time by only ~1.6%. This gain is not considered important enough to sacrifice even minimal gains in detection accuracy for it, as the process is fast already enough. YOLOv11x-OB therefore provides the optimal balance for pipeline deployment, maximizing detection quality to improve downstream segmentation rather than optimizing detection speed in isolation.

SAM model selection for Unified Pipeline: Based on the comprehensive box-prompted and point-prompted evaluation of section 4.2, **SAM-ViT-B-BOX** was selected as the segmentation model for the integrated pipeline. This configuration achieves 66.9% mean IoU and 78.5% mean DICE, which constitutes only a 0.8% below the largest ViT-H variant, while processing masks 29% faster (at 669ms per image) and substantially more memory-efficient, due to its significantly smaller model size (91M versus 630M parameters), which is an important factor when considering edge processing systems. The box-prompted strategy is selected for all classes, as it yields a ~20% IoU advantage over point prompts, according to what was demonstrated in section 4.2.3, while inference times between the two strategies remain comparable (~7-8% difference). This segmentation quality gap alone makes box prompts the only viable option for operational deployment where precise object delineation is required. SAM selection is based in deployment efficiency prioritization, over marginal quality gains. SAM-ViT-B segmentation performance is limited mainly by the detection prompt quality rather than the model capacity, as shown by the minimal 0.8% IoU difference across all three SAM sizes. On the other hand, the 274ms inference savings per image compared to ViT-H variant (943ms versus 669ms) is considered important enough, as it compounds to ~5 minutes of less processing time over the full validation split. This constitutes a meaningful gain for operational deployment scenarios, that may involve thousands of daily images processing.

The final pipeline design pairs the highest-accuracy detector (YOLOv11x-OB) with the most efficient segmenter (SAM-ViT-B-BOX), allocating computational budget to

the component where quality gains are largest. It implements a sequential three-stage architecture, as described in the methodology (section 3.4, Figure 17). For each input aerial image, YOLOv11x-OBB first predicts oriented bounding boxes with class labels and confidence scores. Each OBB detection is then converted to a VBB, as SAM does not natively support rotated box prompts. SAM-ViT-B processes each converted box as a prompt and generates a corresponding segmentation mask in a sequential manner. The instance masks are accumulated across all detections to produce the final segmentation result for the image. The pipeline’s flexible design supports mixed prompting strategies through a class-level configuration file, where each object class can be assigned either to be box or point prompted independently. This feature was implemented for future research purposes, as currently the experimental findings of sections 4.2.1 and 4.2.2, proved no benefit from used point prompts, for any of the classes on average. The OBB-to-VBB conversion step is acknowledged to constitute an additional source of spatial imprecision, as axis-aligned boxes inevitably include more background (than the original oriented boxes) for rotated objects. This conversion, though, is currently necessary given SAM’s interface constraints and the fact that SAM architectural modifications are not within the scope of this thesis.

Component	Model	Key Metric	Inference (ms)
Detection	YOLOv11x-OBB	62.4% F1	59.8 (8.2% of total time)
Segmentation	SAM-ViT-B-BOX	66.9% IoU	669.4 (91.8% of total time)
Pipeline Total			729.2

Table 26: Integrated Detection→Segmentation pipeline component configuration

Total pipeline theoretical inference time (based on individual component evaluation) averages ~730ms per image on the experimental hardware (RTX 3090 Ti), with SAM consuming ~92% and YOLO ~8% of that budget. This establishes the SAM segmentation stage as the clear processing bottleneck and any effort to improve pipeline throughput must focus on it. Potential optimization strategies may include researching faster SAM versions, SAM distillation, batch prompting support, or lighter decoder architectures, though these are beyond the scope of the current evaluation.

4.3.2 Pipeline Experimental Results

The integrated pipeline was evaluated on the AerialFuseCV validation split (456 images, 26255 ground-truth instances). YOLOv11x-OBB generated 20013 detection prompts, of which 16316 were successfully matched to ground-truth instances, yielding a match rate of 62.1%. The remaining 9939 ground-truth objects (37.9%) received no segmentation prompt due to detection failures, representing the primary coverage limitation of the end-to-end system. On overall (Table 27), the pipeline achieves 67.9% mean mask IoU and 79.5% DICE on matched prediction-ground-truth pairs. Detection-level metrics show 81.5% micro-averaged Precision, 62.1% Recall, and 70.5% F1 score. The high Precision indicates that most generated prompts correspond to actual objects (only 18.5% false

positive rate), while the moderate Recall reflects YOLO’s difficulty in detecting all object instances in aerial imagery, particularly small or rare classes. The 67.9% mask IoU on matched pairs is close to the 66.9% achieved by SAM-ViT-B-BOX in the baseline experiment with ground-truth prompts, suggesting that when YOLO correctly detects an object, the resulting predicted box provides spatial guidance of comparable quality to ground-truth annotations.

Metric	Value	Metric	Value
Precision (micro)	81.5%	Mask IoU	67.9%
Recall (micro)	62.1%	Mask DICE	79.5%
F1 (micro)	70.5%	Matched Pairs	16316
Total Prompts	20013	Total GT	26255

Table 27: Integrated Pipeline Overall Performance

Per-class pipeline results (Table 28) reveal a wide performance spectrum across the 15 object categories, with mask IoU ranging from 0.0% (swimming pools) to 87.1% (soccer ball fields). High-performing classes (IoU >80%) include soccer ball fields (87.1%), tennis courts (85.6%), and basketball courts (82.2%). These classes share large, geometrically regular shapes with strong internal contrast against surrounding terrain. Tennis courts achieve the highest F1 score (93.9%) and Recall (90.4%), indicating that YOLO detects nearly all instances and SAM segments them with high Precision. Soccer ball fields and basketball courts achieve high segmentation quality on detected instances but suffer from lower Recall (41.1% and 47.0% respectively), suggesting that YOLO misses many instances of these classes, possibly due to their relative rarity in the training set (141 and 115 GT instances). The moderate-performing classes (50–80% IoU) contain the majority of classes. Large vehicles (79.4%) and small vehicles (70.6%) benefit from high instance counts (3737 and 4489 GT) that provide YOLO with sufficient training examples, resulting in decent Recall (82.9% and 58.5%). Storage tanks achieve 71.8% IoU with strong Precision (87.8%) but low Recall (46.6%), indicating that detected tanks are segmented well but over half go undetected. Ships (63.2%), harbors (61.1%), and planes (54.1%) show moderate performance across both detection and segmentation dimensions, with Precision consistently exceeding Recall. Bridges (56.2%) achieve reasonable segmentation quality on matched pairs despite having the second-lowest Recall (27.6%), meaning that when a bridge is detected, SAM can segment it fairly well, but most bridges are missed entirely. Finally, the low-performing classes include helicopter (41.0% IoU) and swimming pool (0.0% IoU). Helicopters suffer from both low Recall (31.3%) and degraded segmentation quality, reflecting their small size and geometric similarity to other aerial objects. Swimming pools represent a complete pipeline failure, as YOLO produces zero TP detections across all 435 ground-truth instances, making segmentation impossible. This class was also poorly detected in the YOLO baseline experiment (0.4% F1 in standalone evaluation), confirming that the failure originates entirely in the detection stage rather than in SAM’s segmentation capability.

Class	Detection Stage			Segmentation Stage		TP	FP	FN	GT
	Prec (%)	Rec (%)	F1 (%)	IoU (%)	DICE (%)				
soccer-ball-field	65.2	41.1	50.4	87.1	92.2	58	31	83	141
tennis-court	97.7	90.4	93.9	85.6	91.5	678	16	72	750
basketball-court	70.1	47.0	56.2	82.2	89.1	54	23	61	115
large-vehicle	80.6	82.9	81.7	79.4	87.7	3097	744	640	3737
storage-tank	87.8	46.6	60.8	71.8	82.1	1091	152	1252	2343
small-vehicle	69.2	58.5	63.4	70.6	81.6	2626	1169	1863	4489
baseball-diamond	86.6	66.4	75.1	67.6	79.7	142	22	72	214
ship	83.7	63.0	71.9	63.2	76.2	5020	976	2946	7966
harbor	83.6	62.2	71.3	61.1	74.6	1640	322	996	2636
roundabout	88.9	35.4	50.6	60.5	73.7	64	8	117	181
ground-track-field	78.4	48.9	60.3	58.5	72.0	69	19	72	141
bridge	74.3	27.6	40.3	56.2	70.5	124	43	325	449
plane	91.1	63.0	74.5	54.1	69.4	1632	159	959	2591
helicopter	72.4	31.3	43.8	41.0	55.9	21	8	46	67
swimming-pool	0.0	0.0	0.0	0.0	0.0	0	5	435	435

Table 28: Integrated Pipeline Per-Class Results

An important observation from the per-class results is the decoupling between detection and segmentation performance. Classes with low Recall but high Precision like roundabout (35.4% Recall, 88.9% Precision, 60.5% IoU) and bridge (27.6% Recall, 74.3% Precision, 56.2% IoU) demonstrate that the dominant error source is YOLO’s FN detections and not SAM’s segmentation quality. When an object is detected, the resulting mask is generally of acceptable quality. Conversely, small vehicles show the highest FP count (1169) alongside moderate Precision (69.2%), indicating that detection confusion between vehicle-like objects generates unnecessary segmentation prompts that inflate computational cost without contributing useful masks.

The pipeline generated 3697 total FP prompts (18.5% of all prompts), each of which consumed SAM inference time to produce a mask for a non-existent object. Pipeline timing (Table 29) shows an average end-to-end processing time of 786.7ms per image, completing the full validation split (456 images) in ~6 minutes. This corresponds to a throughput of ~1.3 images per second on the experimental hardware (RTX 3090 Ti). The measured per-image time is higher than the theoretical component sum of 729.2ms (section 4.3.1) due to pipeline integration overhead including image loading, OBB-to-VBB coordinate conversion, prompt formatting, and result aggregation, which collectively add ~57ms per image. The detection stage averages 171.8ms per image in the pipeline context, compared to 59.8ms in standalone YOLO evaluation. This difference of 112ms reflects the end-to-end pipeline overhead that is attributed to the detection stage in the timing measurement, including image preprocessing and loading

from disk rather than from memory. The segmentation stage averages 609.5ms per image, slightly below the standalone SAM-ViT-B-BOX measurement of 669.4ms, likely because the pipeline processes only detected objects (average 43.9 prompts per image) rather than the full set of ground-truth instances used in the standalone benchmark. Segmentation remains the clear computational bottleneck at 77.5% of total pipeline time.

Stage	Avg per Image (ms)	Total (min)	Pipeline Share	Standalone (ms)
Detection	171.8	1.3	21.8%	59.8
Segmentation	609.5	4.2	77.5%	669.4
Overhead	5.4	<0.1	0.7%	—
Total	786.7	6.0	100%	—

Table 29: Integrated Pipeline Timing Breakdown

Analyzing some of the results examples (Figure 34), detection weaknesses can be clearly observed. A ship is correctly detected but accompanied by a set of FP detections of the adjacent piers, indicating a clear example of objects getting confused due to geometric (lengthy shapes), spectral (similar colors) and surrounding environment (sea) similarities. In addition, in difficult imagery, even classes like soccer fields, that have proven relatively easy to detect and segment, may face difficulties. The specific panchromatic satellite view, of a lower spatial resolution (0.7m), obviously presents obstacles to clear detection, due to its dark tones and overall smaller scale, where even large objects cover only a few of the total image pixels.

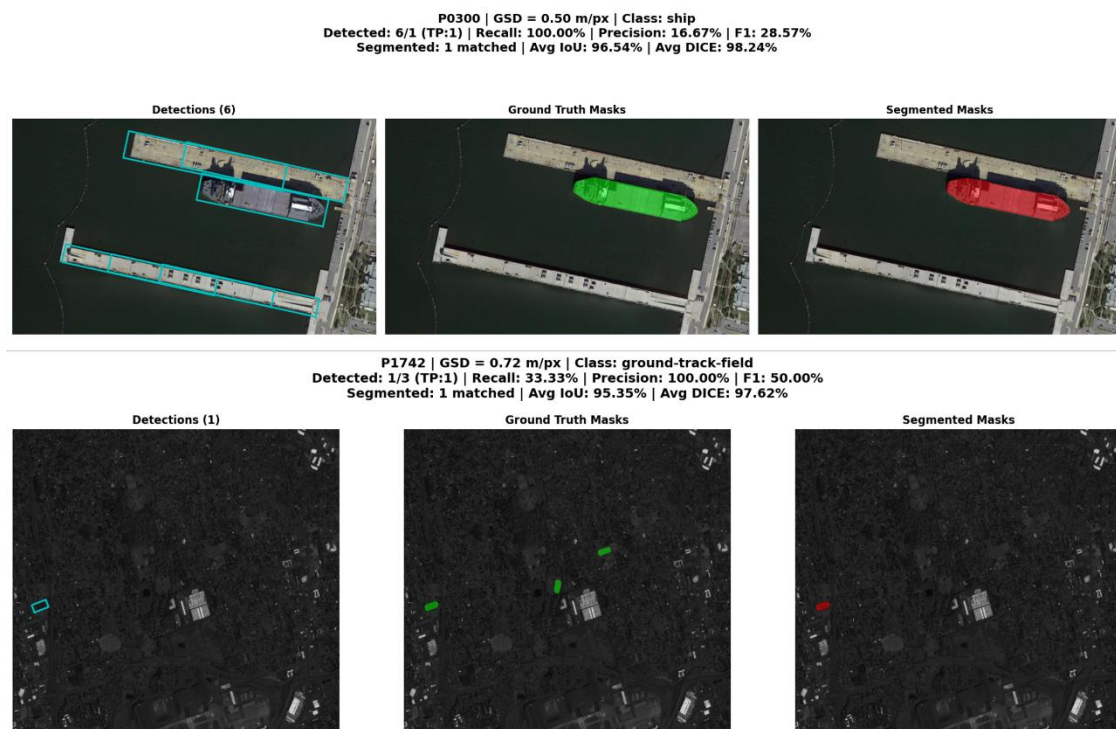


Figure 34: Integrated Detection→Segmentation pipeline detection failure examples

Of course, the already mentioned annotation discrepancies in the AerialFuseCV dataset, (inherited from DOTAv.1.0 and iSAID) still produce an effect in the total evaluation metrics. Distinct annotation discrepancy examples can be seen in Figure 35, where an un-annotated storage tank (top row) is correctly detected, but wrongly counts as FP, reducing Precision and the segmentation metrics. Also, a ground track field (medium row), correctly detected and segmented, is compared to a ground-truth mask that is wrong, as it describes both the ground track field and the soccer field that it includes, artificially reducing metrics. Another example regards a baseball diamond (bottom row), that is correctly detected and segmented but is again compared to a segmentation mask that spans beyond the diamonds area, possibly due to annotators error. The full impact of such annotation discrepancies is not possible to be quantified without an extensive dataset manual refinement process, lying outside the scope of this thesis. It is safe to assume, though, that it doesn't hurt the overall evaluation, but rather makes the reported performance a conservative estimate of the pipeline's true capability

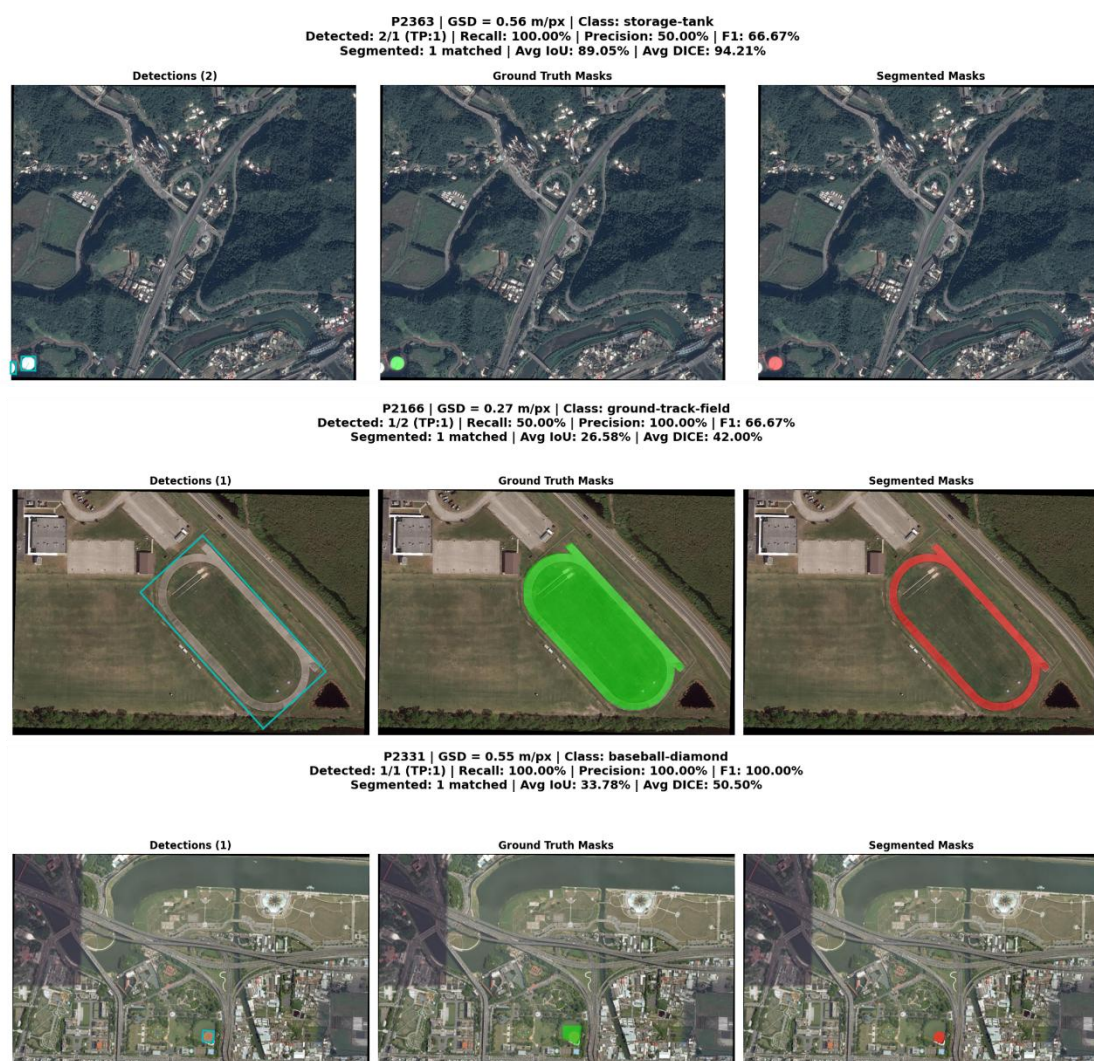


Figure 35: AerialFuseCV annotation discrepancies, inherited from DOTA v1.0 and iSAID datasets

4.3.3 Pipeline Comparative Analysis to Baseline Experimental Results

As a general comparison between the integrated pipeline’s segmentation quality and SAM baseline experiment (where ground-truth boxes served as prompts), can be obtained through the ΔIoU metric, as defined in section 3.4. Table 30 presents the per-class comparison alongside pipeline detection Recall, which determines what fraction of each class’s objects enter the segmentation stage at all.

Class	SAM Baseline IoU (%)	Pipeline IoU (%)	ΔIoU (%)	Pipeline Recall (%)
soccer-ball-field	81.2	87.1	+5.9	41.1
tennis-court	83.8	85.6	+1.8	90.4
basketball-court	75.4	82.2	+6.8	47.0
large-vehicle	74.9	79.4	+4.5	82.9
storage-tank	64.8	71.8	+7.0	46.6
small-vehicle	71.2	70.6	−0.6	58.5
baseball-diamond	67.3	67.6	+0.3	66.4
ship	67.5	63.2	−4.2	63.0
harbor	61.6	61.1	−0.5	62.2
roundabout	58.5	60.5	+2.1	35.4
ground-track-field	56.7	58.5	+1.7	48.9
bridge	54.7	56.2	+1.5	27.6
plane	50.0	54.1	+4.1	63.0
helicopter	40.5	41.0	+0.5	31.3
swimming-pool	69.7	0.0	−69.7	0.0
Overall	66.9	67.9	+1.0	62.1

Table 30: Pipeline vs SAM baseline segmentation quality (ΔIoU) and detection Coverage

The overall pipeline achieves 67.9% IoU compared to 66.9% for the SAM baseline, a positive ΔIoU of +1.0%. At the per-class level, 11 out of 15 classes show positive ΔIoU , with storage tanks (+7.0%), basketball courts (+6.8%), and soccer ball fields (+5.9%) exhibiting the largest gains. This counterintuitive result, where predicted boxes outperform ground-truth prompts, is explained by a selection effect. The pipeline only evaluates objects that YOLO successfully detects, and detected instances tend to be the larger, better-defined examples within each class. The SAM baseline, by contrast, evaluates all instances including small, occluded, or boundary-ambiguous objects that are harder to segment regardless of prompt quality. The pipeline’s IoU advantage therefore does not indicate superior prompting but rather a biased evaluation sample skewed toward easier instances.

Four classes show negative ΔIoU . Swimming pools (−69.7%) represent a special case of complete detection failure (0% Recall), making ΔIoU undefined in practice since no

masks are produced for comparison. Ships (−4.2%) show the only meaningful segmentation quality degradation, suggesting that YOLO’s predicted boxes for ships provide somewhat worse spatial guidance than ground-truth annotations, likely due to the difficulty of tightly bounding densely clustered ships in harbors. Small vehicles (−0.6%) and harbors (−0.5%) show negligible gaps within measurement noise.

The Pipeline Recall reveals the pipeline’s true limitation. While segmentation quality on detected objects matches or exceeds the baseline for most classes, detection Recall ranges from 0.0% (swimming pools) to 90.4% (tennis courts), meaning that between 10% and 100% of objects per class receive no segmentation mask at all. Seven classes have Recall below 50%, indicating that most of their instances go undetected. This confirms the finding from section 4.3.2, where detection coverage, not segmentation quality, is the dominant bottleneck of the integrated system. Improving YOLO’s Recall on low-coverage classes (bridges at 27.6%, helicopters at 31.3%, roundabouts at 35.4%) would directly increase the number of objects receiving high-quality masks, since SAM’s per-instance segmentation capability is not the limiting factor.

Chapter 5 Discussion - Conclusions & Future Work

This chapter interprets the experimental results of Chapter 4, identifies the positions that the evidence supports, acknowledges the limitations of the study, and outlines directions for future research. The discussion is organized around four themes that emerged from the experimental work: domain transfer behavior, the role of prompt quality, the detection bottleneck, and operational viability. Each theme represents not merely an observation from the data but a claim about how the components of a YOLO→SAM pipeline interacts when applied to aerial imagery.

5.1 Discussion of Experimental Findings

5.1.1 Domain Transfer Without Fine Tuning: Capabilities and Boundaries

The YOLO baseline experiments reveal a sharp asymmetry in how detection and segmentation models handle the domain gap between natural ground-level imagery and overhead aerial perspectives. All 24 VBB models, pretrained on COCO and evaluated on DOTA without any adaptation, achieved F1 scores below 0.3% across the three overlapping classes. This nearly complete failure cannot be attributed solely to the reduced class overlap between the COCO and DOTA taxonomies. The fundamental issue is that COCO-trained features encode ground-level object appearance, including frontal viewpoints, typical aspect ratios, and background context, none of which transfer to the top-down aerial view where the same objects present entirely different visual signatures. The VBB experiment therefore establishes that transfer learning from ground-level to aerial imagery fails categorically for detection when neither the representation format nor the training domain is adapted.

In contrast, the 10 OBB models pretrained on DOTA achieved F1 scores up to 62.4% across all 15 classes, a qualitative leap that demonstrates the necessity of domain-specific pretraining and rotation-aware representations for aerial object detection. The OBB advantage is not simply a matter of better training data. Oriented bounding boxes capture the actual spatial extent of rotated objects such as ships, bridges, and vehicles, which axis-aligned boxes inevitably overestimate by including substantial background regions. This geometric fidelity directly improves both localization quality and classification accuracy, as tighter boxes reduce the ambiguity that confuses downstream classification heads.

SAM exhibits a fundamentally different transfer behavior. Without any aerial-domain training, SAM-ViT-B achieves 66.9% mask IoU on aerial objects when provided with accurate box prompts. This success stems from the nature of the segmentation task itself. SAM was trained on the SA-1B dataset (11 million images, over 1 billion masks) to learn a general-purpose boundary delineation function, one that responds to edges, texture transitions, and contrast gradients rather than class-specific features. Aerial

objects, despite their unfamiliar viewpoint, still present clear boundary signals that SAM can exploit. The implication is that detection and segmentation have fundamentally different domain sensitivity profiles. Detection requires learning what objects look like in a specific domain, while segmentation requires learning how to trace boundaries, a more domain-agnostic capability. This distinction has practical significance for pipeline design: the detection stage demands domain adaptation investment, while the segmentation stage can operate effectively in a zero-shot configuration when adequately prompted.

5.1.2 Prompt Quality as the Dominant Variable

Across all SAM experiments, prompt type emerged as the single most influential factor on segmentation quality, dwarfing the effect of model size. Box-prompted SAM-ViT-B achieves 66.9% IoU, while point-prompted SAM-ViT-B achieves only 46.6%, a gap of 20.3%. By comparison, scaling from SAM-ViT-B (91M parameters) to SAM-ViT-H (630M parameters) within the box-prompted strategy improves IoU by only 0.8 %. In other words, switching from a point to a box prompt produces roughly 25 times more improvement than increasing model capacity by 7 times. This ratio held consistently across all 15 object classes, with no class benefiting from point prompts over box prompts.

The practical significance of this finding extends beyond the specific models tested. It suggests that for aerial imagery segmentation using SAM-family models, research effort and computational resources are better invested in improving prompt quality rather than in scaling model parameters. In the context of the integrated pipeline, this means that detection localization accuracy, measured by bounding box IoU and DICE, matters more than deploying a larger SAM variant. The selection of YOLOv11x-OBB over lighter alternatives was justified precisely by this principle: its 64.6% bbox IoU provides higher-quality prompts that translate directly into better segmentation masks, even though the detection-stage speed difference is operationally negligible.

An additional dimension of prompt quality degradation specific to this pipeline is the OBB-to-VBB conversion. SAM's architecture accepts only axis-aligned rectangular prompts, requiring each oriented bounding box to be converted into its minimum enclosing axis-aligned rectangle. For highly rotated objects like ships and bridges, this conversion substantially enlarges the prompt area, including background pixels that dilute the spatial guidance signal. The per-class results support this interpretation: ships, which are frequently elongated and rotated in harbor scenes, show the largest negative ΔIoU (-4.2%) between pipeline and SAM baseline, consistent with prompt degradation from the OBB-to-VBB step. Eliminating this conversion, either through SAM architectural modifications supporting rotated prompts or through rotation-aware prompt encoding, represents a concrete direction for improving pipeline segmentation quality.

5.1.3 The Detection Bottleneck

The pipeline's overall performance is fundamentally constrained not by segmentation quality but by detection coverage. Of the 26255 ground-truth object instances in the AerialFuseCV validation set, only 16316 (62.1%) received a segmentation prompt. The remaining 9939 objects were missed entirely by the detector, meaning that nearly 4 in 10 objects in the scene are invisible to the system regardless of how well SAM performs. This 62.1% coverage figure is the single most important metric for understanding the pipeline's real-world utility, because an undetected object receives no mask at all, a total failure that no downstream processing can recover.

The comparative analysis of section 4.3.3 reinforces this conclusion from a different angle. The pipeline achieves higher mask IoU than the SAM baseline on 11 out of 15 classes, with an overall ΔIoU of +1.0%. This seemingly positive result is actually evidence of selection bias: because the pipeline only evaluates objects that YOLO successfully detects, and detected instances tend to be the larger, better-defined examples within each class, the evaluated subset is systematically easier than the full population. The SAM baseline, which evaluates all instances using ground-truth prompts, includes the harder cases that the pipeline never attempts. The pipeline's apparent segmentation advantage therefore reflects what it avoids rather than what it achieves.

Eight of the 15 classes have pipeline Recall below 50%, meaning that the majority of instances are missed for over half the object categories. The distribution is sharply uneven: tennis courts reach 90.4% Recall while swimming pools achieve 0.0%, a complete detection failure across 435 ground-truth instances. This class-dependent reliability has direct operational implications. The pipeline can be deployed with confidence for well-detected classes (tennis courts, large vehicles) but would require supplementary detection mechanisms or human oversight for poorly-detected ones (bridges, helicopters, roundabouts). Importantly, improving detection Recall for these low-coverage classes would immediately translate into more segmented objects at the existing quality level, since SAM's per-instance segmentation performance is not the limiting factor.

5.1.4 Pipeline Viability for Operational Deployment

The measured end-to-end processing time of 786.7ms per image (approximately 1.3 images per second on RTX 3090 Ti) places the pipeline squarely in the batch-processing regime. For context, real-time UAV analysis typically requires 10–30 frames per second, an order of magnitude faster than the current pipeline achieves. The computational bottleneck is unambiguously located in the segmentation stage, which consumes 77.5% of per-image processing time. Detection contributes only 21.8% (including pipeline overhead), and the remaining 0.7% covers format conversion and result aggregation.

This timing breakdown dictates that any effort to improve throughput must target SAM inference rather than YOLO optimization.

Despite the throughput limitation, the pipeline demonstrates clear operational value for non-real-time aerial imagery analysis workflows. Processing the 456-image validation split requires approximately 6 minutes, a rate that is practical for post-flight UAV imagery analysis, satellite image batch processing, and offline geospatial mapping applications. The training-free nature of the pipeline is a significant operational advantage: no aerial-domain segmentation labels are needed, no fine-tuning is required, and the system can be deployed on new geographic regions or sensor types without retraining. This zero-shot deployment capability reduces the barrier to entry for organizations that need aerial object segmentation but lack the annotated datasets required to train end-to-end segmentation models from scratch.

The pipeline's class-dependent performance profile suggests that operational deployment should incorporate class-specific confidence thresholds rather than a uniform detection threshold. High-Precision classes like tennis courts (97.7% Precision, 90.4% Recall) can be processed with minimal human oversight, while low-Recall classes like roundabouts (88.9% Precision but only 35.4% Recall) would benefit from lower detection thresholds that accept more candidates at the cost of additional false positives, given that SAM's segmentation can still produce usable masks from imprecise prompts. This adaptive thresholding strategy leverages the asymmetry between detection Precision and Recall observed across classes, optimizing the coverage-Precision trade-off on a per-class basis rather than globally.

5.2 Conclusions

This thesis set out to investigate whether pretrained deep learning models can be combined into a training-free pipeline for object detection and semantic segmentation of aerial and satellite imagery. The work was organized around four research questions, each of which can now be answered on the basis of the experimental evidence presented in Chapters 3 and 4.

Detection domain transfer. The first research question asked how well pretrained detection models perform when the image domain shifts from ground-level to aerial views. The answer is sharply conditional on the representation format. 24 COCO-pretrained YOLO models using vertical bounding boxes failed on DOTA v1.0 dataset (yielding an $F1 < 0.3\%$ score), demonstrating that ground-to-aerial transfer is not viable without rotation-aware representations and domain-specific pretraining. In contrast, 10 DOTA-pretrained OBB models achieved up to 62.4% F1 score across all 15 classes, with YOLOv11x-OBB providing the best balance of detection quality (82.8% Precision, 64.6% Bbox IoU) and acceptable inference speed (59.8ms per image). Performance is highly class-dependent, ranging from 93.7% F1 for tennis courts to 0.4% for swimming

pools, indicating that scale, shape regularity, and training set frequency all govern per-class detection reliability in the aerial domain.

Segmentation domain transfer. The second question asked how well foundation segmentation models perform on aerial imagery. SAM demonstrates effective zero-shot transfer, achieving 66.9% mask IoU and 78.5% DICE coefficient (SAM-ViT-B with box prompts) without any aerial-domain training. The critical finding is that prompt type dominates performance, meaning box prompts outperform point prompts by approximately 20% in IoU, consistently across all model sizes and all 15 object classes, while model scaling from ViT-B (91M parameters) to ViT-H (630M parameters) improves IoU by only 0.8%. SAM's segmentation quality is primarily limited by prompt accuracy rather than model capacity, meaning that in any pipeline context, the quality of upstream detection directly governs downstream segmentation.

Dataset landscape and the AerialFuseCV contribution. The third question concerned current dataset availability for aerial object extraction. The literature review and experimental preparation confirmed that while established detection datasets (DOTA v1.0) and segmentation datasets (iSAID) exist independently, no publicly available dataset provides paired bounding box and pixel-level segmentation annotations for the same images across a common class taxonomy. This gap motivated the creation of AerialFuseCV, which fuses DOTA v1.0 detection annotations with iSAID segmentation masks. The dataset provides 26255 ground-truth instances across 15 classes with dual annotation types, enabling the combined detection-segmentation evaluation that this thesis required.

End-to-end pipeline feasibility. The fourth and central question asked whether a detection-segmentation pipeline is feasible for operational aerial imagery analysis. The integrated pipeline combining YOLOv11x-OBB with SAM-ViT-B-BOX achieves 67.9% mask IoU and 79.5% DICE on matched prediction-ground-truth pairs, with 81.5% micro Precision and 62.1% detection coverage. Processing speed of 786.7ms per image (approximately 1.3 images per second) is practical for batch workflows but not for real-time applications. The pipeline is feasible and produces usable segmentation outputs for most object classes without any segmentation-specific training, but its primary limitation is detection Recall rather than segmentation quality. Nearly 38% of ground-truth objects receive no mask at all, making detection improvement the most direct path to better system-level performance.

Beyond answering the research questions, this thesis makes four contributions:

- It provides a systematic 34-model YOLO benchmark on DOTA v1.0 spanning five architecture generations (YOLOv8 through YOLOv12) in both VBB and OBB configurations, establishing baseline detection performance on aerial imagery.

- It delivers a 6-configuration SAM benchmark evaluating 3 model sizes and 2 prompt strategies on aerial objects, systematically quantifying the impact of prompt quality, relative to model capacity, for zero-shot segmentation in overhead imagery. This constitutes a trade-off that has not been characterized in prior remote sensing studies under controlled, multi-class conditions.
- It presents a complete, reproducible Detection→Segmentation pipeline with a dual-metric evaluation framework that separates detection and segmentation performance, enabling precise attribution of pipeline failures to the responsible component.
- It introduces AerialFuseCV, a fused evaluation dataset derived from DOTA v1.0 and iSAID that provides paired detection and segmentation annotations for aerial imagery, supporting integrated pipeline analysis rather than standalone task evaluation.

The experimental evidence collected in this thesis demonstrates that combining pretrained detection and segmentation models into a training-free pipeline is a viable approach for aerial imagery analysis, with clear strengths in deployment flexibility and annotation cost avoidance, and equally clear weaknesses in detection coverage and processing speed. The identified bottlenecks, particularly detection Recall and OBB-to-VBB prompt conversion, are specific and addressable through the future research directions outlined in section 5.4. As detection architectures continue to improve and segmentation foundation models become faster and more versatile, the gap between zero-shot pipeline performance and fully supervised alternatives will likely narrow, making training-free approaches increasingly practical for operational geospatial applications.

5.3 Limitations and Work Assumptions

The findings of this thesis are subject to several limitations that affect the generalizability and interpretation of the reported results. These limitations are discussed below, grouped by their source, so it may become clear which conclusions hold in general and which of them are connected to the specific experimental conditions of this study. These limitations are inevitably leading to a set of working assumptions that define the conclusions drawn in this thesis and should be considered when interpreting the results or extrapolating them to different datasets, sensors, or operational settings.

Many important limitations come from the dataset itself. All experiments were conducted on DOTA v1.0 for detection annotations and iSAID for segmentation masks, fused into the AerialFuseCV dataset. While these are established benchmarks in the aerial imagery community, they represent a specific geographic distribution, sensor mix, and annotation style. Results on other aerial datasets such as xView, VisDrone, or proprietary UAV collections may differ, particularly for object classes whose visual appearance varies across regions and sensor modalities. The evaluation was performed

exclusively on the validation split of 456 images containing 26255 ground-truth instances. This provides robust statistics for high-frequency classes like ships (7966 instances) and small vehicles (4489 instances), but yields limited statistical power for rare classes such as helicopters (67 instances) and basketball courts (115 instances), where per-class metrics are more susceptible to individual instance variability.

In addition, the AerialFuseCV dataset, made for the integrated pipeline evaluation, inherits annotation inconsistencies from its parent datasets, as documented in section 4.3.2. These include unannotated objects that count as false positives when correctly detected, loose detection label, segmentation masks that extend beyond the actual object boundary, and occasional class labeling errors. The impact of these inconsistencies on the reported metrics is not possible to be precisely quantified without a complete manual re-annotation effort, which lies beyond the scope of this work. It is reasonable to assume, however, that the effect is conservative: annotation errors more frequently penalize correct predictions than reward incorrect ones, meaning the true pipeline performance is likely somewhat higher than what the metrics report.

Another limiting factor comes from the deliberate methodological decision of this thesis, to evaluate all models in their zero-shot or pretrained configuration without domain-specific fine-tuning. The YOLO OBB models were pretrained on DOTA v1.0 training data by their developers (Ultralytics), but no additional training was performed. SAM was used entirely in its original pretrained state. This choice enables a clean assessment of out-of-the-box capability, but it also means that the reported results represent a lower bound on what these architectures could achieve with targeted domain adaptation. In particular, the detection Recall bottleneck (62.1%) identified in section 5.1.3 could potentially be improved through fine-tuning on the specific aerial imagery distribution, though quantifying this improvement would require a separate experimental study.

The OBB-to-VBB conversion required by SAM's interface constitutes a structural limitation of the pipeline architecture. For rotated objects, the axis-aligned enclosing rectangle inevitably includes additional background pixels, degrading the spatial Precision of the segmentation prompt. This conversion is not a design choice, but a constraint imposed by SAM's current architecture, which does not natively accept rotated box coordinates. The magnitude of this degradation varies by class and is most pronounced for elongated, frequently rotated objects like ships and bridges. Future SAM variants or architectural modifications that support oriented prompts would eliminate this limitation entirely.

The Hungarian matching algorithm used for prediction-ground-truth assignment employs a relatively lenient IoU threshold of 0.1. This threshold was chosen to accommodate the spatial imprecision introduced by the OBB-to-VBB conversion and by annotation quality variation, but it means that some matched pairs may involve loosely overlapping predictions. Stricter thresholds (e.g., $\text{IoU} \geq 0.3$ or ≥ 0.5) would produce lower Recall and potentially different per-class rankings. The sensitivity of the

reported metrics to the matching threshold was not systematically evaluated and represents an area where additional analysis could be made.

During the experimentation phase, several research directions emerged, but intentionally excluded from this study, in order to maintain a focused scope. No comparison was made against end-to-end trained instance segmentation models such as Mask R-CNN, YOLACT, or SOLOv2, which would provide a useful baseline for contextualizing the pipeline's performance against supervised alternatives. Moreover, no multi-resolution controlled study was conducted. Although DOTA v1.0 images span a range of ground sample distances (0.1-3.0m), the evaluation did not systematically isolate GSD as an independent variable, in order to perform GSD focus experiments. Also, no temporal or video analysis was attempted, limiting the findings to static single-image processing. Finally, all experiments were conducted on a single desktop GPU (RTX 3090 Ti), and inference timing results may not generalize to other hardware configurations, particularly edge devices or cloud-based multi-GPU setups, relevant to production deployment scenarios.

5.4 Future Research Directions

The limitations and findings of this thesis suggest several directions for future research, organized below by expected impact and implementation feasibility. The most immediate extensions directly address the identified bottlenecks, while longer-term directions expand the scope of the pipeline approach to new domains and architectures.

The most impactful short-term improvement would be targeted fine-tuning of the YOLO detection stage on the DOTA training split, with an explicit emphasis on Recall optimization rather than overall detection accuracy. Although the YOLO-OBB models used in this study are already pretrained on DOTA, these pretrained checkpoints are optimized for general-purpose detection performance and balanced Precision–Recall trade-offs, not for the Recall-dominated requirements of a Detection→Segmentation pipeline. The detection Recall bottleneck (62.1% coverage) was identified as the primary pipeline limitation, and fine-tuning with adjusted loss weighting, confidence thresholds, or class rebalancing could directly address this weakness. Such fine-tuning could substantially improve Recall for currently low-coverage classes such as bridges (27.6%), helicopters (31.3%), and roundabouts (35.4%) without requiring any architectural changes to the pipeline. Given that the SAM segmentation results already demonstrate acceptable mask quality for detected instances, any Recall improvement at the detection stage would translate directly into a larger number of usable segmentation outputs.

On the segmentation side, evaluating newer SAM variants such as SAM2 (Ravi et al., 2024) and EfficientSAM (Xiong et al., 2023) would clarify whether the current segmentation speed bottleneck (77.5% of pipeline time) can be reduced without sacrificing mask quality. These models offer architectural optimizations specifically targeting inference efficiency, and their compatibility with the existing box-prompted

pipeline interface would make integration straightforward. Additionally, hybrid prompting strategies that combine box prompts with negative point prompts could improve segmentation quality in dense scenes like harbors and parking lots, where adjacent objects frequently confuse the mask decoder. Similar mechanisms are employed in PerSAM (R. Zhang et al., 2023b), where negative point prompts are used to explicitly suppress irrelevant regions and improve instance separation in crowded scenes. The pipeline’s existing class-level prompting configuration already supports this extension without severe architectural modifications.

A systematic evaluation of matching threshold sensitivity ($\text{IoU} \geq 0.1, 0.3, \text{ and } 0.5$) would quantify how robust the reported metrics are to this methodological choice. Similarly, expanding the evaluation to additional aerial datasets (xView, VisDrone, custom UAV captures) would test the generalizability of the findings beyond the DOTA/iSAID distribution.

Architecture-wise, the OBB-to-VBB conversion is the most important limitation of the current pipeline. Developing or adapting a SAM variant that natively accepts oriented bounding box prompts would eliminate the background inclusion problem that currently degrades segmentation quality for rotated objects. This could be achieved through modifications to SAM’s prompt encoder to accept rotated rectangles, or through a learned prompt transformation that maps oriented boxes to the internal representation space without information loss.

To address the computational waste from false positive detections, a confidence-based segmentation strategy could be explored. Currently, 18.5% of all generated prompts correspond to non-existent objects, each consuming SAM inference time without producing useful output. Filtering detections by a confidence threshold before passing them to SAM or implementing a lightweight verification step between detection and segmentation, could reduce unnecessary computation while maintaining coverage for genuine detections. Additionally, batch SAM inference, processing multiple prompts in a single encoder forward pass, could substantially reduce per-image segmentation time by amortizing the image encoding cost across all detections in an image.

On the broader side, comparing the proposed pipeline against end-to-end trained instance segmentation models such as Mask R-CNN (He et al., 2018), SOLOv2 (X. Wang et al., 2020), or recent transformer-based alternatives like Mask2Former (B. Cheng et al., 2022), would provide valuable context for evaluating the performance of the zero-shot approach relative to fully supervised baselines. Such a comparison would not aim to demonstrate superiority, but to quantify the trade-offs between annotation cost, training requirements, and achievable segmentation accuracy. Also, a controlled multi-resolution study, systematically varying GSD while holding other variables constant, would characterize the pipeline’s resolution sensitivity and identify minimum GSD requirements for reliable operation across different object classes.

For deployment-oriented research, benchmarking the pipeline on edge hardware (NVIDIA Jetson series, mobile GPUs) with quantized model variants would assess real-world deployment feasibility beyond desktop GPU conditions. Temporal extensions, applying the pipeline to sequential aerial imagery or video for object tracking and change detection, would expand the approach from static analysis to dynamic monitoring applications. Finally, cross-sensor evaluation on synthetic aperture radar (SAR) or multispectral imagery would test whether the pipeline's zero-shot segmentation capability generalizes beyond optical sensors to other modalities used in Earth observation.

In addition, a second deployment mode for the Detection→Segmentation pipeline, distinct from the real-time inference scenario that motivated the architectural choices in this thesis, is its use as an annotation-assistance tool for the construction of aerial segmentation datasets. The single-pass design used here trades mask quality against latency in favor of latency, as a single SAM call per detected box was performed, with no iterative refinement and no use of SAM's mask-prompt input. The opposite trade is appropriate when the goal is to produce ground-truth masks for a new dataset. Annotation latency is bounded by the human reviewer rather than by the model, so additional compute per object is essentially free, while mask quality is the property that determines whether the resulting dataset is usable.

Concretely, the pipeline could be extended along three axes for this purpose. First, SAM's mask-prompt input would be activated: the mask produced by the first SAM call would be fed back as a dense prompt to a second call, together with the original bounding box, allowing the model to refine boundary details that are typically lost in a single forward pass. Second, the OBB→VBB conversion that introduces background pixels for rotated objects could be partially compensated by adding negative point prompts at the corners of the axis-aligned box that fall outside the original OBB, suppressing the background regions that currently leak into the mask. Third, a lightweight human-in-the-loop interface would allow an annotator to add positive or negative point prompts to correct residual errors, with each correction triggering another SAM pass conditioned on the updated prompt set. The expected total cost per object is estimated in the order of one to two seconds, roughly three or four orders of magnitude faster than fully manual pixel-level annotation, while producing masks of substantially higher quality than the single-pass pipeline.

The aerial domain is particularly well suited to this approach, mainly for two reasons. First, existing aerial segmentation datasets are small relative to the scale of the underlying detection datasets, e.g. iSAID covers the 15 categories of DOTA v1.0 with extensive manual effort and no comparable resource exists for DOTA v1.5, DOTA v2.0, xView, or the various UAV datasets. Second, the AerialFuseCV construction methodology developed in this thesis already implements a partial version of this workflow, as detection boxes are matched against existing iSAID masks to verify spatial agreement. Replacing the iSAID-mask source with iteratively refined SAM masks

would allow the same workflow to be applied to detection datasets that have no segmentation labels at all, scaling up the available aerial segmentation data without requiring proportional manual annotation cost.

SAM-based annotation tooling already exists in the broader computer vision literature (for example, the workflows built around Grounding DINO and SAM, or the annotation features of commercial platforms such as Roboflow). What is not currently addressed in those tools is the set of constraints specific to overhead imagery: native handling of oriented bounding boxes rather than axis-aligned ones, robustness to crowded scenes containing dozens of small objects per image, and operation at the high spatial resolutions typical of aerial captures. An annotation-assistance pipeline tailored to these constraints, built on the Detection→Segmentation detector-prompter framework but operating in the high-quality rather than the low-latency regime, is therefore a natural direction for follow-up work and a candidate contribution distinct from the real-time pipeline evaluated in this thesis.

References

1. Bay, H., Tuytelaars, T., & Van Gool, L. (2006). SURF: Speeded Up Robust Features. In A. Leonardis, H. Bischof, & A. Pinz (Eds.), *Computer Vision – ECCV 2006* (pp. 404–417). Springer. https://doi.org/10.1007/11744023_32
2. Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection* (arXiv:2004.10934). arXiv. <https://doi.org/10.48550/arXiv.2004.10934>
3. Bolya, D., Zhou, C., Xiao, F., & Lee, Y. J. (2019). *YOLACT: Real-time Instance Segmentation* (arXiv:1904.02689). arXiv. <https://doi.org/10.48550/arXiv.1904.02689>
4. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020a). *End-to-End Object Detection with Transformers* (arXiv:2005.12872; Version 1). arXiv. <https://doi.org/10.48550/arXiv.2005.12872>
5. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020b). *End-to-End Object Detection with Transformers* (Version 3). arXiv. <https://doi.org/10.48550/ARXIV.2005.12872>
6. Chen, K., Liu, C., Chen, H., Zhang, H., Li, W., Zou, Z., & Shi, Z. (2023a). *RSPrompter: Learning to Prompt for Remote Sensing Instance Segmentation based on Visual Foundation Model* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.2306.16269>
7. Chen, K., Liu, C., Chen, H., Zhang, H., Li, W., Zou, Z., & Shi, Z. (2023b). *RSPrompter: Learning to Prompt for Remote Sensing Instance Segmentation based on Visual Foundation Model* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.2306.16269>
8. Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). *Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation* (arXiv:1802.02611). arXiv. <https://doi.org/10.48550/arXiv.1802.02611>
9. Cheng, B., Misra, I., Schwing, A. G., Kirillov, A., & Girdhar, R. (2022). *Masked-attention Mask Transformer for Universal Image Segmentation* (arXiv:2112.01527). arXiv. <https://doi.org/10.48550/arXiv.2112.01527>
10. Cheng, G., Han, J., Zhou, P., & Guo, L. (2014). Multi-class geospatial object detection and geographic image classification based on collection of part detectors. *ISPRS Journal of Photogrammetry and Remote Sensing*, 98, 119–132. <https://doi.org/10.1016/j.isprsjprs.2014.10.002>
11. Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, 1, 886–893 vol. 1. <https://doi.org/10.1109/CVPR.2005.177>
12. Demir, I., Koperski, K., Lindenbaum, D., Pang, G., Huang, J., Basu, S., Hughes, F., Tuia, D., & Raskar, R. (2018). *DeepGlobe 2018: A Challenge to Parse the Earth through Satellite Images*. <https://doi.org/10.48550/ARXIV.1805.06561>

13. Ding, J., Xue, N., Long, Y., Xia, G.-S., & Lu, Q. (2018a). *Learning RoI Transformer for Detecting Oriented Objects in Aerial Images* (arXiv:1812.00155). arXiv. <https://doi.org/10.48550/arXiv.1812.00155>
14. Ding, J., Xue, N., Long, Y., Xia, G.-S., & Lu, Q. (2018b). *Learning RoI Transformer for Detecting Oriented Objects in Aerial Images* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.1812.00155>
15. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* (arXiv:2010.11929). arXiv. <https://doi.org/10.48550/arXiv.2010.11929>
16. Du, B., Liu, L., Tao, D., Wang, D., Xu, M., Zhang, J., & Zhang, L. (2023). SAMRS: Scaling-up Remote Sensing Segmentation Dataset with Segment Anything Model. *Advances in Neural Information Processing Systems* 36, 8815–8827. <https://doi.org/10.52202/075280-0385>
17. Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., & Zisserman, A. (2010a). The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision*, 88(2), 303–338. <https://doi.org/10.1007/s11263-009-0275-4>
18. Everingham, M., Van Gool, L., Williams, C., Winn, J., & Zisserman, A. (2010b). The Pascal Visual Object Classes (VOC) challenge. *International Journal of Computer Vision*, 88, 303–338. <https://doi.org/10.1007/s11263-009-0275-4>
19. Garioud, A., De Wit, A., Poupée, M., Valette, M., Giordano, S., & Wattrelos, B. (2023). *FLAIR #2: Textural and temporal information for semantic segmentation from multi-source optical imagery*. <https://doi.org/10.48550/ARXIV.2305.14467>
20. Garioud, A., Gonthier, N., Landrieu, L., De Wit, A., Valette, M., Poupée, M., Giordano, S., & Wattrelos, B. (2023). *FLAIR: A Country-Scale Land Cover Semantic Segmentation Dataset From Multi-Source Optical Imagery* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2310.13336>
21. Girshick, R. (2015). *Fast R-CNN* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.1504.08083>
22. Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). *Rich feature hierarchies for accurate object detection and semantic segmentation* (arXiv:1311.2524). arXiv. <https://doi.org/10.48550/arXiv.1311.2524>
23. He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2021). *Masked Autoencoders Are Scalable Vision Learners* (arXiv:2111.06377). arXiv. <https://doi.org/10.48550/arXiv.2111.06377>
24. He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017). *Mask R-CNN* (arXiv:1703.06870; Version 1). arXiv. <https://doi.org/10.48550/arXiv.1703.06870>
25. He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2018). *Mask R-CNN* (arXiv:1703.06870). arXiv. <https://doi.org/10.48550/arXiv.1703.06870>
26. He, K., Zhang, X., Ren, S., & Sun, J. (2015). *Deep Residual Learning for Image Recognition* (arXiv:1512.03385). arXiv. <https://doi.org/10.48550/arXiv.1512.03385>
27. Hosang, J., Benenson, R., Dollár, P., & Schiele, B. (2016). What makes for effective detection proposals? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38(4), 814–830. <https://doi.org/10.1109/TPAMI.2015.2465908>

28. Jocher, et al. (2023). *Explore Ultralytics YOLOv8*. Ultralytics. <https://docs.ultralytics.com/models/yolov8/>
29. Jocher, G., Ayush Chaurasia, Stoken, A., Borovec, J., NanoCode012, Yonghye Kwon, Kalen Michael, TaoXie, Jiacong Fang, Imyhxy, Lorna, 曾逸夫(Zeng Yifu), Wong, C., Abhiram V, Montes, D., Zhiqiang Wang, Fati, C., Jebastin Nadar, Laughing, ... Jain, M. (2022). *ultralytics/yolov5: V7.0 - YOLOv5 SOTA Realtime Instance Segmentation* (Version v7.0) [Computer software]. Zenodo. <https://doi.org/10.5281/ZENODO.3908559>
30. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., Dollár, P., & Girshick, R. (2023). *Segment Anything* (arXiv:2304.02643). arXiv. <https://doi.org/10.48550/arXiv.2304.02643>
31. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25. https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html
32. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97. <https://doi.org/10.1002/nav.3800020109>
33. Lam, D., Kuzma, R., McGee, K., Dooley, S., Laielli, M., Klaric, M., Bulatov, Y., & McCord, B. (2018a). *xView: Objects in Context in Overhead Imagery* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.1802.07856>
34. Lam, D., Kuzma, R., McGee, K., Dooley, S., Laielli, M., Klaric, M., Bulatov, Y., & McCord, B. (2018b). *xView: Objects in Context in Overhead Imagery* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.1802.07856>
35. Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X., & Wei, X. (2022). *YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications* (arXiv:2209.02976). arXiv. <https://doi.org/10.48550/arXiv.2209.02976>
36. Li, K., Wan, G., Cheng, G., Meng, L., & Han, J. (2020). Object detection in optical remote sensing images: A survey and a new benchmark. *ISPRS Journal of Photogrammetry and Remote Sensing*, 159, 296–307. <https://doi.org/10.1016/j.isprsjprs.2019.11.023>
37. Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). *Focal Loss for Dense Object Detection* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.1708.02002>
38. Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., & Dollár, P. (2015a). *Microsoft COCO: Common Objects in Context* (arXiv:1405.0312). arXiv. <https://doi.org/10.48550/arXiv.1405.0312>
39. Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., & Dollár, P. (2015b). *Microsoft COCO: Common Objects in Context* (arXiv:1405.0312). arXiv. <https://doi.org/10.48550/arXiv.1405.0312>

40. Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., Jiang, Q., Li, C., Yang, J., Su, H., Zhu, J., & Zhang, L. (2023). *Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection* (Version 5). arXiv. <https://doi.org/10.48550/ARXIV.2303.05499>
41. Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2015). *SSD: Single Shot MultiBox Detector*. <https://doi.org/10.48550/ARXIV.1512.02325>
42. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows* (arXiv:2103.14030). arXiv. <https://doi.org/10.48550/arXiv.2103.14030>
43. Long, J., Shelhamer, E., & Darrell, T. (2015). *Fully Convolutional Networks for Semantic Segmentation* (arXiv:1411.4038). arXiv. <https://doi.org/10.48550/arXiv.1411.4038>
44. Long, Y., Gong, Y., Xiao, Z., & Liu, Q. (2017). Accurate Object Localization in Remote Sensing Images Based on Convolutional Neural Networks. *IEEE Transactions on Geoscience and Remote Sensing*, 55(5), 2486–2498. <https://doi.org/10.1109/TGRS.2016.2645610>
45. Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*, 60(2), 91–110. <https://doi.org/10.1023/B:VISI.0000029664.99615.94>
46. Maggiori, E., Tarabalka, Y., Charpiat, G., & Alliez, P. (2017). Can semantic labeling methods generalize to any city? The inria aerial image labeling benchmark. *2017 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, 3226–3229. <https://doi.org/10.1109/IGARSS.2017.8127684>
47. Mumuni, F., & Mumuni, A. (2024). *Segment Anything Model for automated image data annotation: Empirical studies using text prompts from Grounding DINO* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.2406.19057>
48. Osco, L. P., Wu, Q., Lemos, E. L. de, Gonçalves, W. N., Ramos, A. P. M., Li, J., & Junior, J. M. (2023). *The Segment Anything Model (SAM) for Remote Sensing Applications: From Zero to One Shot* (arXiv:2306.16623). arXiv. <https://doi.org/10.48550/arXiv.2306.16623>
49. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). *Learning Transferable Visual Models From Natural Language Supervision* (arXiv:2103.00020). arXiv. <https://doi.org/10.48550/arXiv.2103.00020>
50. Ravi, N., Gabeur, V., Hu, Y.-T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K. V., Carion, N., Wu, C.-Y., Girshick, R., Dollár, P., & Feichtenhofer, C. (2024). *SAM 2: Segment Anything in Images and Videos* (arXiv:2408.00714). arXiv. <https://doi.org/10.48550/arXiv.2408.00714>
51. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2015). *You Only Look Once: Unified, Real-Time Object Detection* (Version 5). arXiv. <https://doi.org/10.48550/ARXIV.1506.02640>

52. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection* (arXiv:1506.02640). arXiv. <https://doi.org/10.48550/arXiv.1506.02640>
53. Redmon, J., & Farhadi, A. (2018). *YOLOv3: An Incremental Improvement* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.1804.02767>
54. Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks* (Version 3). arXiv. <https://doi.org/10.48550/ARXIV.1506.01497>
55. Ren, S., He, K., Girshick, R., & Sun, J. (2016). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks* (arXiv:1506.01497). arXiv. <https://doi.org/10.48550/arXiv.1506.01497>
56. Ren, S., Luzi, F., Lahrichi, S., Kassaw, K., Collins, L. M., Bradbury, K., & Malof, J. M. (2023). *Segment anything, from space?* (Version 4). arXiv. <https://doi.org/10.48550/ARXIV.2304.13000>
57. Ronneberger, O., Fischer, P., & Brox, T. (2015a). *U-Net: Convolutional Networks for Biomedical Image Segmentation* (arXiv:1505.04597). arXiv. <https://doi.org/10.48550/arXiv.1505.04597>
58. Ronneberger, O., Fischer, P., & Brox, T. (2015b). *U-Net: Convolutional Networks for Biomedical Image Segmentation* (arXiv:1505.04597). arXiv. <https://doi.org/10.48550/arXiv.1505.04597>
59. Shermeyer, J., & Van Etten, A. (2018). *The Effects of Super-Resolution on Object Detection Performance in Satellite Imagery* (Version 3). arXiv. <https://doi.org/10.48550/ARXIV.1812.04098>
60. Sun, X., Wang, P., Yan, Z., Xu, F., Wang, R., Diao, W., Chen, J., Li, J., Feng, Y., Xu, T., Weinmann, M., Hinz, S., Wang, C., & Fu, K. (2022). FAIR1M: A benchmark dataset for fine-grained object recognition in high-resolution remote sensing imagery. *ISPRS Journal of Photogrammetry and Remote Sensing*, 184, 116–130. <https://doi.org/10.1016/j.isprsjprs.2021.12.004>
61. Tian, Z., Shen, C., & Chen, H. (2020). *Conditional Convolutions for Instance Segmentation* (arXiv:2003.05664). arXiv. <https://doi.org/10.48550/arXiv.2003.05664>
62. Tian, Z., Shen, C., Wang, X., & Chen, H. (2020). *BoxInst: High-Performance Instance Segmentation with Box Annotations* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2012.02310>
63. Ultralytics. (n.d.). *Ultralytics YOLO11*. Retrieved February 3, 2026, from <https://docs.ultralytics.com/models/yolo11/>
64. Ultralytics. (2024). *Ultralytics YOLO11*. <https://docs.ultralytics.com/models/yolo11/>
65. Van Etten, A., Lindenbaum, D., & Bacastow, T. M. (2018). *SpaceNet: A Remote Sensing Dataset and Challenge Series* (Version 3). arXiv. <https://doi.org/10.48550/ARXIV.1807.01232>
66. Van Rijsbergen, C. J. (1979). Information Retrieval, 2nd ed. *Journal of the American Society for Information Science*. <https://doi.org/10.1002/asi.4630300621>

67. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need* (arXiv:1706.03762; Version 1). arXiv. <https://doi.org/10.48550/arXiv.1706.03762>
68. Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., & Ding, G. (2024). *YOLOv10: Real-Time End-to-End Object Detection* (arXiv:2405.14458). arXiv. <https://doi.org/10.48550/arXiv.2405.14458>
69. Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2022). *YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors* (arXiv:2207.02696). arXiv. <https://doi.org/10.48550/arXiv.2207.02696>
70. Wang, C.-Y., Yeh, I.-H., & Liao, H.-Y. M. (2024). *YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information* (arXiv:2402.13616). arXiv. <https://doi.org/10.48550/arXiv.2402.13616>
71. Wang, D., Zhang, J., Du, B., Xu, M., Liu, L., Tao, D., & Zhang, L. (2023a). *SAMRS: Scaling-up Remote Sensing Segmentation Dataset with Segment Anything Model* (Version 4). arXiv. <https://doi.org/10.48550/ARXIV.2305.02034>
72. Wang, D., Zhang, J., Du, B., Xu, M., Liu, L., Tao, D., & Zhang, L. (2023b). *SAMRS: Scaling-up Remote Sensing Segmentation Dataset with Segment Anything Model* (Version 4). arXiv. <https://doi.org/10.48550/ARXIV.2305.02034>
73. Wang, J., Xu, C., Yang, W., & Yu, L. (2021). *A Normalized Gaussian Wasserstein Distance for Tiny Object Detection* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.2110.13389>
74. Wang, J., Zheng, Z., Ma, A., Lu, X., & Zhong, Y. (2021). *LoveDA: A Remote Sensing Land-Cover Dataset for Domain Adaptive Semantic Segmentation* (Version 6). arXiv. <https://doi.org/10.48550/ARXIV.2110.08733>
75. Wang, X., Zhang, R., Kong, T., Li, L., & Shen, C. (2020). *SOLOv2: Dynamic and Fast Instance Segmentation* (arXiv:2003.10152). arXiv. <https://doi.org/10.48550/arXiv.2003.10152>
76. Xia, G.-S., Bai, X., Ding, J., Zhu, Z., Belongie, S., Luo, J., Datcu, M., Pelillo, M., & Zhang, L. (2019). *DOTA: A Large-scale Dataset for Object Detection in Aerial Images* (arXiv:1711.10398). arXiv. <https://doi.org/10.48550/arXiv.1711.10398>
77. Xia, J., Yokoya, N., Adriano, B., & Broni-Bediako, C. (2022). *OpenEarthMap: A Benchmark Dataset for Global High-Resolution Land Cover Mapping* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2210.10732>
78. Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2021). *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers* (arXiv:2105.15203). arXiv. <https://doi.org/10.48550/arXiv.2105.15203>
79. Xie, X., Cheng, G., Wang, J., Yao, X., & Han, J. (2021). *Oriented R-CNN for Object Detection* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2108.05699>
80. Xiong, Y., Varadarajan, B., Wu, L., Xiang, X., Xiao, F., Zhu, C., Dai, X., Wang, D., Sun, F., Iandola, F., Krishnamoorthi, R., & Chandra, V. (2023). *EfficientSAM: Leveraged Masked Image Pretraining for Efficient Segment Anything* (arXiv:2312.00863). arXiv. <https://doi.org/10.48550/arXiv.2312.00863>

81. Yang, X., & Yan, J. (2020). Arbitrary-Oriented Object Detection with Circular Smooth Label. *Computer Vision – ECCV 2020*, 677–694. https://doi.org/10.1007/978-3-030-58598-3_40
82. Zamir, S. W., Arora, A., Gupta, A., Khan, S., Sun, G., Khan, F. S., Zhu, F., Shao, L., Xia, G.-S., & Bai, X. (2019). *iSAID: A Large-scale Dataset for Instance Segmentation in Aerial Images* (arXiv:1905.12886). arXiv. <https://doi.org/10.48550/arXiv.1905.12886>
83. Zhang, C., Han, D., Qiao, Y., Kim, J. U., Bae, S.-H., Lee, S., & Hong, C. S. (2023). *Faster Segment Anything: Towards Lightweight SAM for Mobile Applications* (arXiv:2306.14289). arXiv. <https://doi.org/10.48550/arXiv.2306.14289>
84. Zhang, R., Jiang, Z., Guo, Z., Yan, S., Pan, J., Ma, X., Dong, H., Gao, P., & Li, H. (2023a). *Personalize Segment Anything Model with One Shot* (arXiv:2305.03048). arXiv. <https://doi.org/10.48550/arXiv.2305.03048>
85. Zhang, R., Jiang, Z., Guo, Z., Yan, S., Pan, J., Ma, X., Dong, H., Gao, P., & Li, H. (2023b). *Personalize Segment Anything Model with One Shot* (arXiv:2305.03048). arXiv. <https://doi.org/10.48550/arXiv.2305.03048>
86. Zhao, X., Ding, W., An, Y., Du, Y., Yu, T., Li, M., Tang, M., & Wang, J. (2023). *Fast Segment Anything* (arXiv:2306.12156). arXiv. <https://doi.org/10.48550/arXiv.2306.12156>
87. Zhu, P., Wen, L., Bian, X., Ling, H., & Hu, Q. (2018). *Vision Meets Drones: A Challenge* (Version 2). arXiv. <https://doi.org/10.48550/ARXIV.1804.07437>
88. Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2020). *Deformable DETR: Deformable Transformers for End-to-End Object Detection* (Version 4). arXiv. <https://doi.org/10.48550/ARXIV.2010.04159>

Appendices

Appendix - A: All YOLO Models Evaluation with Vertical Bounding Boxes - Per Class Performance

Model	Class	F1 (%)	Recall (%)	Precision (%)	BBox IoU (%)	BBox Dice (%)
YOLOv8n	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.09	0.05	9.09	69.36	81.74
	small-vehicle	0.29	0.15	20.00	42.43	54.56
YOLOv8s	ship	0.22	0.11	33.33	16.05	27.66
	large-vehicle	0.05	0.02	3.45	66.67	80.00
	small-vehicle	0.69	0.35	17.72	48.33	61.96
YOLOv8m	ship	0.04	0.02	16.67	54.88	70.12
	large-vehicle	0.09	0.05	6.25	82.42	89.88
	small-vehicle	0.61	0.31	16.33	73.41	84.33
YOLOv8l	ship	0.07	0.03	21.43	69.15	80.50
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.65	0.33	25.37	78.74	87.93
YOLOv8x	ship	0.16	0.08	70.00	80.76	88.38
	large-vehicle	0.04	0.02	1.39	56.15	71.92
	small-vehicle	0.18	0.09	7.94	71.27	82.67
YOLOv9t	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.05	0.02	5.26	62.00	76.54
	small-vehicle	0.11	0.06	21.43	75.25	85.87
YOLOv9s	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.05	0.02	2.44	64.52	78.43
	small-vehicle	0.51	0.26	17.86	55.77	66.87
YOLOv9m	ship	0.02	0.01	8.33	42.39	59.54
	large-vehicle	0.22	0.11	5.81	54.93	69.48
	small-vehicle	0.43	0.22	9.78	59.27	71.17
YOLOv9c	ship	0.02	0.01	20.00	57.24	72.80
	large-vehicle	0.05	0.02	0.00	0.00	0.00
	small-vehicle	0.55	0.28	25.00	68.48	78.31
YOLOv9e	ship	0.07	0.03	60.00	69.64	81.44
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.07	0.04	6.67	74.32	85.26
YOLOv10n	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.04	0.02	25.00	72.59	84.12
YOLOv10s	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.00	0.00	0.00	0.00	0.00

Model	Class	F1 (%)	Recall (%)	Precision (%)	BBox IoU (%)	BBox Dice (%)
	small-vehicle	0.55	0.28	20.34	76.22	86.13
YOLOv10m	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.22	0.11	7.69	36.84	46.61
YOLOv10x	ship	0.02	0.01	14.29	76.95	87.00
	large-vehicle	0.05	0.02	2.56	73.04	84.42
	small-vehicle	0.94	0.48	24.14	64.10	74.51
YOLOv11n	ship	0.02	0.01	25.00	50.66	67.25
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.65	0.33	24.66	59.35	70.95
YOLOv11s	ship	0.04	0.02	25.00	15.69	27.02
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.22	0.11	13.64	27.28	40.13
YOLOv11m	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.22	0.11	20.00	68.09	79.82
YOLOv11l	ship	0.04	0.02	28.57	69.40	81.86
	large-vehicle	0.05	0.02	1.67	77.77	87.50
	small-vehicle	0.15	0.07	17.39	68.99	81.17
YOLOv11x	ship	0.11	0.06	50.00	78.24	87.58
	large-vehicle	0.05	0.02	3.23	40.48	57.64
	small-vehicle	0.11	0.06	10.00	71.68	83.53
YOLOv12n	ship	0.00	0.00	0.00	0.00	0.00
	large-vehicle	0.09	0.05	9.09	49.22	65.03
	small-vehicle	0.07	0.04	9.09	54.05	68.64
YOLOv12s	ship	0.04	0.02	50.00	37.23	54.02
	large-vehicle	0.05	0.02	2.94	47.43	64.35
	small-vehicle	0.07	0.04	16.67	73.18	84.51
YOLOv12m	ship	0.02	0.01	33.33	67.88	80.87
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.07	0.04	12.50	81.47	89.79
YOLOv12l	ship	0.13	0.07	66.67	45.61	58.23
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.22	0.11	24.00	59.59	70.51
YOLOv12x	ship	0.07	0.03	50.00	57.35	66.36
	large-vehicle	0.00	0.00	0.00	0.00	0.00
	small-vehicle	0.15	0.07	30.77	47.55	59.78

Appendix - B: All YOLO Models Evaluation with Oriented Bounding Boxes - Per Class Performance

Model	Class	F1 (%)	Recall (%)	Precision (%)	BBox IoU (%)	BBox Dice (%)
YOLOv8n-OBB	plane	68.0	55.5	88.0	75.8	85.6
	ship	69.6	58.3	86.3	73.4	84.0
	storage-tank	48.8	33.4	90.5	74.0	84.4
	baseball-diamond	57.4	44.4	81.2	71.0	82.7
	tennis-court	92.9	90.0	96.1	90.8	95.0
	basketball-court	54.8	40.9	83.1	87.1	92.8
	ground-track-field	51.3	41.7	66.7	80.5	88.9
	harbor	81.6	79.2	84.1	67.1	79.2
	bridge	32.9	21.1	74.2	59.2	73.6
	large-vehicle	81.6	78.8	84.7	79.5	88.2
	small-vehicle	56.3	47.8	68.4	73.7	84.4
	helicopter	37.4	23.3	94.4	72.5	83.8
	roundabout	38.9	26.8	70.6	71.4	82.1
	soccer-ball-field	48.3	37.9	66.7	84.4	91.1
	swimming-pool	0.9	0.5	7.7	55.7	69.1
YOLOv8s-OBB	plane	73.4	61.6	91.0	78.2	87.4
	ship	71.9	60.0	89.7	74.4	84.7
	storage-tank	52.9	37.3	91.3	74.0	84.5
	baseball-diamond	57.0	44.9	78.0	71.6	83.2
	tennis-court	93.2	89.7	97.0	91.9	95.6
	basketball-court	61.3	46.2	91.0	86.5	92.0
	ground-track-field	52.5	43.1	67.4	84.7	91.5
	harbor	84.3	82.9	85.7	69.0	80.7
	bridge	41.0	28.9	70.5	60.3	74.3
	large-vehicle	83.5	79.3	88.3	80.6	89.0
	small-vehicle	59.0	50.0	72.1	75.0	85.3
	helicopter	34.7	23.3	68.0	75.0	85.6
	roundabout	45.4	31.8	79.2	70.6	81.6
	soccer-ball-field	45.0	36.6	58.3	86.6	92.5
	swimming-pool	1.3	0.7	18.8	44.8	61.4
YOLOv8m-OBB	plane	76.8	67.2	89.6	78.8	87.8
	ship	72.9	61.3	89.9	75.1	85.2
	storage-tank	56.8	40.8	93.3	73.3	84.0
	baseball-diamond	68.8	60.7	79.3	72.8	83.9
	tennis-court	94.4	90.7	98.4	92.4	95.9
	basketball-court	59.3	44.7	88.1	91.1	95.2
	ground-track-field	56.5	50.0	64.9	84.1	91.0

Model	Class	F1 (%)	Recall (%)	Precision (%)	BBox IoU (%)	BBox Dice (%)
	harbor	86.1	85.3	86.9	70.6	81.8
	bridge	42.6	30.8	68.8	61.7	75.3
	large-vehicle	83.8	80.6	87.1	81.2	89.3
	small-vehicle	59.9	51.5	71.4	76.2	86.1
	helicopter	53.5	37.0	96.4	72.8	84.1
	roundabout	48.1	34.6	78.5	72.0	82.6
	soccer-ball-field	47.3	37.9	63.0	89.0	94.0
	swimming-pool	3.2	1.8	12.1	52.6	67.7
YOLOv8l-OB	plane	79.3	71.0	89.8	79.4	88.1
	ship	72.9	61.1	90.2	76.1	85.9
	storage-tank	57.7	41.9	92.6	74.5	84.8
	baseball-diamond	68.2	62.1	75.6	74.9	85.4
	tennis-court	93.6	90.0	97.6	92.7	96.1
	basketball-court	61.8	47.7	87.5	90.7	95.0
	ground-track-field	60.8	54.9	68.1	84.0	91.0
	harbor	86.7	84.7	88.9	71.1	82.2
	bridge	44.3	33.4	65.7	63.7	76.8
	large-vehicle	83.8	80.3	87.7	81.5	89.5
	small-vehicle	62.4	53.6	74.5	76.4	86.3
	helicopter	56.9	42.5	86.1	74.5	85.1
	roundabout	50.9	39.7	71.0	69.3	80.5
	soccer-ball-field	50.4	41.2	64.9	87.8	93.2
	swimming-pool	0.0	0.0	0.0	0.0	0.0
YOLOv8x-OB	plane	78.8	70.8	89.0	80.0	88.6
	ship	73.1	61.1	91.0	76.0	85.9
	storage-tank	57.3	41.1	94.4	75.3	85.4
	baseball-diamond	73.3	66.8	81.2	74.1	84.9
	tennis-court	93.9	90.9	97.2	92.6	96.1
	basketball-court	62.1	47.7	88.7	91.6	95.5
	ground-track-field	60.7	51.4	74.0	83.6	90.9
	harbor	87.8	86.8	88.7	71.2	82.3
	bridge	44.6	33.4	67.1	62.5	75.9
	large-vehicle	83.9	79.9	88.4	81.8	89.7
	small-vehicle	63.4	55.3	74.4	75.7	85.8
	helicopter	49.5	34.2	89.3	75.4	85.8
	roundabout	52.2	40.2	74.2	70.8	81.6
	soccer-ball-field	50.0	43.1	59.5	86.3	92.2
	swimming-pool	0.0	0.0	0.0	0.0	0.0
YOLOv11n-OB	plane	64.9	50.7	90.1	77.3	86.8

Model	Class	F1 (%)	Recall (%)	Precision (%)	BBox IoU (%)	BBox Dice (%)
	ship	70.3	58.2	88.6	72.9	83.7
	storage-tank	49.3	33.5	93.3	73.9	84.4
	baseball-diamond	52.6	38.3	83.7	70.5	82.3
	tennis-court	92.1	87.6	96.9	91.3	95.3
	basketball-court	56.2	43.2	80.3	87.5	93.0
	ground-track-field	55.9	43.1	79.5	82.0	89.8
	harbor	82.3	77.9	87.2	67.7	79.8
	bridge	33.3	21.6	73.5	60.6	74.5
	large-vehicle	81.9	78.4	85.8	79.3	88.1
	small-vehicle	57.2	46.1	75.2	74.8	85.2
	helicopter	31.1	19.2	82.4	75.6	86.0
	roundabout	38.9	26.8	70.6	73.5	83.9
	soccer-ball-field	46.4	35.3	67.5	84.4	91.1
	swimming-pool	0.4	0.2	20.0	68.0	81.0
YOLOv11s-OBb	plane	71.2	58.4	91.1	78.4	87.5
	ship	72.0	60.1	89.9	74.7	84.9
	storage-tank	54.1	37.9	94.6	73.9	84.4
	baseball-diamond	62.1	50.5	80.6	73.0	84.1
	tennis-court	92.8	88.6	97.4	92.2	95.9
	basketball-court	57.9	43.2	87.7	89.4	94.1
	ground-track-field	59.0	50.0	72.0	82.9	90.4
	harbor	84.6	83.6	85.7	69.2	80.9
	bridge	39.2	26.9	71.8	61.2	74.9
	large-vehicle	83.3	78.7	88.5	80.6	89.0
	small-vehicle	59.0	51.1	69.8	75.4	85.6
	helicopter	50.5	35.6	86.7	76.4	86.4
	roundabout	49.0	34.6	83.8	70.9	81.9
	soccer-ball-field	48.8	38.6	66.3	87.4	93.1
	swimming-pool	0.9	0.5	12.5	66.7	80.0
YOLOv11m-OBb	plane	73.6	60.9	93.1	80.1	88.7
	ship	73.0	61.1	90.8	75.4	85.4
	storage-tank	55.7	39.5	94.5	74.4	84.7
	baseball-diamond	66.5	57.9	78.0	72.6	83.7
	tennis-court	93.8	89.7	98.3	92.5	96.0
	basketball-court	58.2	43.2	89.1	91.0	95.2
	ground-track-field	57.6	47.2	73.9	85.1	91.7
	harbor	86.2	84.6	87.9	70.5	81.8
	bridge	39.5	28.2	65.8	63.9	77.0
	large-vehicle	84.5	79.9	89.6	81.2	89.4

Model	Class	F1 (%)	Recall (%)	Precision (%)	BBox IoU (%)	BBox Dice (%)
	small-vehicle	62.0	52.5	75.6	76.2	86.2
	helicopter	46.0	31.5	85.2	76.6	86.6
	roundabout	48.2	33.5	85.7	71.0	82.0
	soccer-ball-field	48.1	36.6	70.0	88.2	93.4
	swimming-pool	0.4	0.2	3.4	74.3	85.2
YOLOv11l-OBB	plane	75.9	64.2	92.7	80.3	88.8
	ship	73.2	61.2	91.0	75.8	85.7
	storage-tank	56.9	40.5	95.5	75.6	85.6
	baseball-diamond	66.3	57.9	77.5	74.4	85.1
	tennis-court	93.7	89.5	98.3	92.9	96.3
	basketball-court	62.7	47.7	91.3	91.1	95.3
	ground-track-field	58.0	49.3	70.3	83.9	91.0
	harbor	87.3	84.8	89.9	71.3	82.4
	bridge	41.6	29.5	70.3	63.0	76.3
	large-vehicle	84.7	80.0	89.9	81.3	89.4
	small-vehicle	62.0	51.5	78.0	76.3	86.3
	helicopter	56.1	41.1	88.2	73.6	84.6
	roundabout	51.7	37.4	83.8	70.8	81.6
	soccer-ball-field	51.6	41.8	67.4	86.4	92.3
	swimming-pool	0.4	0.2	3.2	71.6	83.4
YOLOv11x-OBB	plane	78.2	66.7	94.3	81.0	89.2
	ship	73.5	61.5	91.5	76.1	85.9
	storage-tank	57.5	41.1	95.6	75.9	85.8
	baseball-diamond	73.0	64.5	84.1	75.1	85.5
	tennis-court	93.7	89.6	98.1	92.8	96.2
	basketball-court	61.2	48.5	83.1	90.8	95.1
	ground-track-field	60.3	48.6	79.5	84.6	91.4
	harbor	88.0	85.3	90.8	71.7	82.7
	bridge	39.0	26.5	73.7	65.7	78.3
	large-vehicle	85.1	79.8	91.1	81.6	89.6
	small-vehicle	62.2	52.8	75.5	76.3	86.2
	helicopter	54.9	38.4	96.6	74.4	85.1
	roundabout	52.6	36.9	91.7	71.7	82.2
	soccer-ball-field	47.9	37.9	65.2	88.2	93.4
	swimming-pool	0.0	0.0	0.0	0.0	0.0

Appendix - C: All SAM Models Segmentation Evaluation per Class Performance

SAM Variant	Prompt	Class	IoU (%)	Dice (%)
SAM-ViT-H	Box	plane	52.3	66.9
		ship	68.6	80.3
		storage-tank	66.7	78.8
		baseball-diamond	70.3	81.6
		tennis-court	83.7	90.3
		basketball-court	76.4	84.9
		ground-track-field	56.7	70.2
		harbor	62.7	75.8
		bridge	54.5	69.0
		large-vehicle	75.0	84.7
		small-vehicle	71.3	82.7
		helicopter	41.3	55.6
		roundabout	60.3	72.6
		soccer-ball-field	81.4	88.6
		swimming-pool	69.1	79.9
	Point	plane	44.3	55.7
		ship	47.7	59.1
		storage-tank	32.0	43.0
		baseball-diamond	46.2	57.8
		tennis-court	74.0	82.8
		basketball-court	40.0	48.2
		ground-track-field	35.3	44.3
		harbor	35.7	47.0
		bridge	11.7	16.8
		large-vehicle	65.0	75.0
		small-vehicle	61.8	73.6
		helicopter	30.5	39.2
		roundabout	36.5	43.4
		soccer-ball-field	47.2	56.5
		swimming-pool	57.5	67.5

SAM Variant	Prompt	Class	IoU (%)	Dice (%)
SAM-ViT-L	Box	plane	51.7	66.2
		ship	68.1	79.9
		storage-tank	65.9	77.5
		baseball-diamond	66.5	78.8
		tennis-court	83.9	90.5
		basketball-court	77.1	85.1
		ground-track-field	56.9	70.3
		harbor	62.7	75.9
		bridge	54.5	69.0
		large-vehicle	75.1	84.8
		small-vehicle	71.2	82.6
		helicopter	41.1	55.4
		roundabout	58.6	71.5
		soccer-ball-field	81.8	88.9
		swimming-pool	68.8	79.7
	Point	plane	44.0	55.7
		ship	47.9	59.4
		storage-tank	32.3	43.3
		baseball-diamond	47.2	58.5
		tennis-court	74.6	83.5
		basketball-court	38.4	46.6
		ground-track-field	35.2	44.4
		harbor	34.4	45.4
		bridge	12.8	18.3
		large-vehicle	65.2	75.4
		small-vehicle	62.7	74.5
		helicopter	29.4	37.7
		roundabout	36.7	43.9
		soccer-ball-field	47.8	57.3
		swimming-pool	59.3	69.5

SAM Variant	Prompt	Class	IoU (%)	Dice (%)
SAM-ViT-B	Box	plane	50.0	65.0
		ship	67.5	79.5
		storage-tank	64.8	77.2
		baseball-diamond	67.3	79.5
		tennis-court	83.8	90.5
		basketball-court	75.4	84.5
		ground-track-field	56.7	70.2
		harbor	61.6	75.2
		bridge	54.7	69.1
		large-vehicle	74.9	84.6
		small-vehicle	71.2	82.6
		helicopter	40.5	55.0
		roundabout	58.5	72.1
		soccer-ball-field	81.2	88.6
		swimming-pool	69.7	80.4
	Point	plane	41.6	53.6
		ship	47.1	58.4
		storage-tank	30.8	41.3
		baseball-diamond	45.7	57.0
		tennis-court	75.1	83.5
		basketball-court	38.2	46.8
		ground-track-field	34.5	44.3
		harbor	35.9	48.1
		bridge	11.1	16.6
		large-vehicle	63.1	73.7
		small-vehicle	59.7	72.1
		helicopter	27.9	36.4
		roundabout	32.2	38.6
		soccer-ball-field	52.7	62.8
		swimming-pool	60.3	70.3